



謝林海 (Linhai Xie)
Kellogg College
牛津大學

提交以申請哲學博士學位之論文

2019 年夏季學期 (Trinity Term)

本論文獻給我的父母，感謝他們無私且不可或缺的支持

致謝

在博士生涯的尾聲，我對許多人滿懷感激。最值得我感謝的是我的指導教授 Niki 和 Andrew。他們勤勉的指導磨練了我也成就了我，嚴厲中卻充滿溫柔。聆聽他們那些「瘋狂」的想法是最令人愉悅的事情之一，儘管起初這些想法常讓我感到驚恐。然而，我逐漸意識到他們是透過這些想法在教導我，如同《悲慘世界》中主教教導被捕的尚萬強那樣，引領我從更高的層次看待事物。在他們的共同指導下，Niki 更多地關注我的研究方向，憑藉她長期累積的學術經驗，耐心地與我討論所有的可能性。另一方面，Andrew 總能從我拙劣的表達中捕捉到核心概念，並協助我在論文中將其精煉。感謝他們，否則這篇論文中的許多成果在誕生前就已夭折。

其次，我要感謝 Sen Wang 博士和 Yishu Miao 博士。Sen 引領我踏入機器人領域的第一步，如同黑夜大海中指引水手的金星，照亮了我在這片未知領域的道路。那些他在辦公室坐在我身後的難忘日子，是我機器人研究進步最神速的巔峰時期。Yishu 是我在機器學習領域的導師兼合作夥伴，他催化了我對 Deep Reinforcement Learning 的研究，並將其與機器人技術結合。現在他成了我在 MO Intelligence 實習的老闆，那是一個非常棒的工作環境。Sen 和 Yishu 也是我的朋友與導師，我深切感激他們在牛津生活期間給予我的幫助。

接下來，我要感謝 Xiaoxuan、Zhihua 以及 CyberPhysical Systems Group 的其他同事。他們對工作的熱忱和令人振奮的成就，激勵我更深入地鑽研研究。此外，辦公室裡沒完沒了的軼事和有趣的八卦也為我帶來了歡笑與快樂。感謝我在牛津的每一位朋友，我懷念所有的火鍋聚會、酒吧小酌，以及我們在 DotA 的線上重聚。我也要感謝我的女朋友 Yuanyuan，在艱難時期給予我溫暖的擁抱，以及我的父母，感謝他們不可或缺且無私的支持。

導航是移動機器人最基本的能力之一，使其能夠從起點移動到終點。傳統方法高度依賴預先定義的地圖，而獲取地圖在時間和人力上都耗費巨大。此外，地圖僅在採集當下是準確的，且會隨著環境變遷而逐漸失效。我們認為，必須具備高品質地圖的這項嚴格要求，從根本上限制了機器人系統在動態世界中的實現可能性。在本論文中，我們受無地圖導航 (mapless navigation) 範式與深度強化學習 (Deep Reinforcement Learning, DRL) 近期發展的啟發，探討如何開發實用的機器人導航技術。

DRL 面臨的主要問題之一是需要具備數百萬次重複試驗的多樣化實驗設置。這顯然無法透過真實機器人經由試錯 (trial and error) 來達成，因此我們改從模擬環境中學習。這引發了第一個基本問題，即如何彌合從模擬到真實環境之間的現實差距 (reality gap)，這將在第 3 章中討論。我們專注於單眼視覺障礙物規避 (monocular visual obstacle avoidance) 這項挑戰，將其作為低階導航原語 (navigation primitive)。我們開發了一種在模擬世界中訓練，卻能良好泛化至真實世界的 DRL 方法。

另一個限制 DRL 技術應用於真實世界移動機器人領域的問題，是訓練後的策略具有高度變異性 (high variance)。由於搜尋空間複雜且維度高，這會導致收斂效果不佳以及整體獎勵偏低。在第 4 章中，我們利用簡單的古典控制器為 DRL 的局部導航任務提供引導，避免純粹的隨機初始探索。我們證明了這種新穎的加速方法能大幅降低樣本變異性，並顯著提高可達到的平均獎勵。

我們考慮的最後一個挑戰是無地圖導航中的稀疏視覺引導。在第 5 章中，我們提出了一種基於少量航點圖像進行導航的創新方法，這與傳統基於影片的「教導與重複」(teach and repeat) 模式截然不同。我們證明了在模擬環境中學習到的策略可以直接轉移到現實世界，並且能夠在僅有極少環境描述的情況下，良好地泛化至未見過的場景。

針對障礙物規避、局部引導和全域導航等關鍵問題，我們開發並測試了多種創新方法，以實現實用機器人導航的願景。我們展示了如何將 DRL 作為一種強大的無模型 (model-free) 方法來解決這些問題。

目錄

1 緒論 ...	1
1.1 研究動機 ...	1
1.2 研究挑戰與問題 ...	3
1.3 貢獻 ...	6
2 背景知識 ...	9
2.1 強化學習 (Reinforcement Learning) ...	9
2.1.1 馬可夫決策過程 (Markov Decision Process) ...	9
2.1.2 核心要素 ...	10
2.1.3 與其他學習範式的比較 ...	11
2.1.4 歷史發展 ...	12
2.1.5 方法 ...	13
2.1.6 深度強化學習演算法 ...	14
2.1.6.1 Deep Q Network ...	14
2.1.6.2 REINFORCE 演算法 ...	16
2.1.6.3 深度確定性策略梯度 (Deep Deterministic Policy Gradients) ...	18
2.2 基於地圖的機器人導航 ...	19
2.2.1 感測 ...	20
2.2.2 定位與地圖構建 ...	21
2.2.2.1 定位 ...	21
2.2.2.2 地圖表示 ...	22
2.2.2.3 同步定位與地圖構建 ...	22
2.2.3 路徑規劃 (Path-Planning) ...	23
2.3 無地圖機器人導航 (Mapless Robot Navigation) ...	24
2.3.1 基於測距感測器的系統 (Ranging Sensor Based Systems) ...	25
2.3.1.1 基於邊緣檢測的方法 (Edge-Detection Based Methods) ...	25
2.3.1.2 位能場法 (Potential Field Methods) ...	25
2.3.1.3 動態視窗法 (Dynamic Window Based Methods) ...	26
2.3.2 基於視覺的系統 (Vision Based Systems) ...	26
2.3.2.1 基於光流的方法 (Optical Flow Based Approaches) ...	27
2.3.2.2 基於外觀匹配的方法 ...	28
2.3.2.3 反應式視覺方法 ...	29
2.3.2.4 基於特徵追蹤的方法 ...	30
2.4 討論 ...	30
2.4.1 從模擬到現實世界 ...	30
2.4.2 加速模擬訓練 ...	31
2.4.3 學習全域無地圖導航 ...	32
3 視覺避障：從模擬到現實世界 ...	34
3.1 前言 ...	34

3.2 相關工作 ...	35
3.3 問題定義 ...	37
3.4 模型與訓練細節 ...	37
3.5 實驗 ...	40
3.5.1 不同模型的訓練效率 ...	40
3.5.2 真實世界測試 ...	41
3.5.2.1 靜態影像的動作預測 ...	41
3.5.2.2 三種不同場景下的測試 ...	43
3.5.2.3 凌亂環境下的測試 ...	44
3.5.2.4 與基準方法的比較 ...	45
3.6 結論 ...	46
4 局部導航：利用引導加速訓練 ...	47
4.1 簡介 ...	47
4.2 相關工作 ...	48
4.3 問題定義與假設 ...	50
4.4 AsDDPG：結合確定性引導的學習 ...	50
4.4.1 啟發式控制器 ...	52
4.4.2 網路架構 ...	52
4.4.3 Critic-DQN ...	53
4.4.4 訓練 ...	55
4.5 AsDDPG 實驗 ...	56
4.5.1 使用多種超參數加速訓練程序 ...	57
4.5.2 控制器參數的影響 ...	58
4.5.3 在複雜環境與稀疏獎勵下進行訓練 ...	62
4.5.4 真實世界測試 ...	64
4.6 SGuidance：透過隨機引導進行學習 ...	65
4.7 模型 ...	66
4.7.1 啟發式控制器 (Heuristic Controllers) ...	67
4.7.2 隨機切換 (Stochastic Switch) ...	67
4.7.2.1 折棍程序 (Stick-breaking) ...	67
4.7.2.2 帶有基準值的 REINFORCE 演算法 (REINFORCE Algorithm with Baselines) ...	68
4.7.3 演算法 ...	69
4.8 SGuidance 實驗 ...	69
4.8.1 訓練環境與設定 ...	71
4.8.2 模擬環境中的導航 ...	72
4.8.2.1 結合隨機引導的強化學習 ...	72
4.8.2.2 使用不同的獨立控制器 ...	74
4.8.2.3 使用不同的切換機制 ...	74
4.8.2.4 與 Thompson Sampling 的進一步比較 ...	75
4.8.2.5 隨機切換函數的建構 ...	78
4.8.2.6 複雜環境與簡單獎勵 ...	79
4.8.2.7 無引導推論 ...	80
4.8.2.8 不同 γ 的學習 ...	80
4.8.3 真實世界環境中的導航 ...	81
4.9 結論 ...	83
5 全域導航：具備稀疏方向引導與視覺參考的無地圖導航 ...	84
5.1 緒論 ...	84
5.2 相關工作 ...	86
5.3 任務描述 ...	87
5.3.1 任務分解 ...	87
5.3.2 指令子任務 ...	87
5.3.3 控制子任務 ...	88
5.4 網路架構 ...	88
5.4.1 基於注意力機制的指揮器 ...	88
5.4.2 控制器 ...	90
5.5 訓練 ...	90
5.5.1 自監督式指揮官訓練標籤 ...	90
5.5.2 訓練指揮官 ...	91
5.5.3 控制策略的強化學習 ...	93
5.6 實驗 ...	94
5.6.1 虛擬世界中的模型消融研究 ...	94
5.6.2 真實世界測試 ...	97

5.7 結論 ...	99
6 結論與未來展望 ...	100
參考文獻 ...	104

圖表目錄

1.1 一幅漫畫（基於 YimelsGreat 的插畫），想像了機器人與人類緊密合作的近未來。它也展示了本論文中提到的一些核心概念：1) 將學習到的策略 (policy) 從模擬環境遷移到現實世界。2) 透過額外的引導加速學習過程。3) 學習在稀疏的方向指令和視覺參考下進行導航。 ...	4
2.1 在 RL 問題中，學習代理人 (agent) 在每個時間步觀察環境狀態並執行動作以影響環境。根據不同的動作，環境會回傳一個數值獎勵作為給代理人的回饋，用以評估該行為是否符合預期。 ...	11
2.2 三類無模型 (model-free) RL 方法，即基於策略 (policy-based)、基於價值 (value-based) 以及動作及評論者 (actor-critic) 方法。 ...	13
2.3 基於地圖導航與無地圖導航系統的組成。局部目標 (local goals) 是沿著全域規劃器 (global planner) 所提供之規劃路徑上的連續路標點 (waypoints)。 ...	19
2.4 GraphSLAM 的一個簡單實例。藍色三角形代表機器人在不同時間步的位姿 (poses)，紅色圓圈表示構成環境地圖的不同地標 (landmarks)。此外，實線是機器人的軌跡，而虛線則表示機器人對地標的觀測 ...	23
2.5 此圖展示了動態視窗法 (Dynamic Window Approach)。白色圓形和矩形為障礙物，藍色矩形為機器人，虛線則是不同控制動作下的取樣軌跡。根據預測，機器人將選擇直行以避開兩側的障礙物。 ...	26
2.6 光流 (Optical Flow) 的輸出。綠色箭頭顯示移動像素的方向，而點則代表靜態像素。 ...	27
2.7 [154] 中的地面分割。左欄影像為輸入，其中黑色多邊形內的區域是用於學習地面紋理的參考區域。右欄則展示了分割結果。 ...	29
3.1 基於單眼影像並透過深度強化學習實現障礙物規避的網路架構。首先建構一個全卷積神經網路 (Fully Convolutional Neural Network) 以從原始 RGB 影像預測深度。隨後接續一個深度 Q 網路，該網路由一個卷積網路和一個對決網路 (Dueling Network) 組成，用以平行預測角速度動作與線速度動作的 Q 值。 ...	37
3.2 給定一批訓練資料，包含目前狀態 x_t 、動作 a 、獎勵 r 以及結果狀態 x_{t+1} ，D3QN 的訓練程序如圖所示。 $\oplus, \ominus$ 與 $\otimes$ 分別代表逐元素的加法、減法與乘法運算。 ...	38
3.3 用於訓練的兩個 Gazebo 模擬世界。 ...	40
3.4 三種模型的平滑學習曲線，顯示機器人獲得的平均獎勵。 ...	41
3.5 在不同室內環境（例如：圖書館、博物館、閣樓與臥室，由上至下）進行的實驗。下方的長條圖展示了網路針對每個線速度與角速度動作所預測的 Q 值，其中紅色長條表示網路以貪婪策略 (Greedy) 選擇的動作。請注意，前兩個長條代表線速度動作，其餘則代表轉向動作。 ...	42
3.6 三種不同場景下的真實世界測試。影像串流下方的曲線顯示了機器人在每一步所選擇的轉向動作。 ...	43
3.7 在具有不同障礙物數量與佈置的房間內進行真實世界測試。矩形代表箱子，而星號和圓圈分別代表椅子和垃圾桶。 ...	44
3.8 動態環境中的真實世界測試。灰色障礙物為固定不動，彩色障礙物則表示可移動障礙物在各個時間點的運動路徑。障礙物上的數字表示變化的順序。 ...	45
3.9 左圖為雜亂環境中的特徵檢測結果，其餘兩張則為走廊環境中的結果。 ...	46
4.1 使用 Actor-Critic 強化學習方法訓練深度神經網路，以學習機器人導航的局部規劃。Critic-DQN 網路會評估外部控制器（例如 PID 控制器）與策略網路 (Policy Network) 的表現，並根據情況從中選擇較佳的動作。所有產生的學習樣本皆儲存於經驗回放池 (Replay Buffer) 中。因此，策略網路可以透過模仿外部控制器或檢視自身的策略來進行自我提升。 ...	51
4.2 網路架構。網路層以矩形表示。橘色箭頭表示網路層與其他組件（例如輸入狀態和簡單控制器的輸出）之間的連接關係。最終動作是根據 Critic-DQN 預測的 Q 值來選取的。 ...	52
4.3 用於訓練的三個 Stage 模擬世界。灰色矩形為障礙物，藍色矩形為機器人。每個回合 (Episode) 的目標位置皆為隨機生成。 ...	56
4.4 不同網路超參數下的平滑學習曲線。此圖展示了網路在每個訓練步驟的平均表現。請注意，MoveBase 被選為基準方法 (Baseline)，其平均表現得分約為 0.5。 ...	57
4.5 在兩種不同模擬世界中，整個訓練過程中單次訓練回合 (episode) 內策略網路的使用情況。 ...	58
4.6 機器人在訓練不同階段配合各種比例控制器 (proportional controllers) 所學習到的策略。每張圖片分別展示了訓練初期、中期和後期，在 200 個訓練回合中機器人的軌跡，以及外部控制器與策略網路之間的切換結果。 ...	60
4.6 機器人在訓練不同階段配合各種比例控制器 (proportional controllers) 所學習到的策略。每張圖片分別展示了訓練初期、中期和後期，在 200 個訓練回合中機器人的軌跡，以及外部控制器與策略網路之間的切換結果。 ...	61
4.7 每個訓練回合的最終結果（抵達目標、碰撞與逾時）以及平滑後的學習曲線。 ...	63
4.8 使用密集 (dense) 與稀疏 (sparse) 獎勵函數所學習到的策略，其中稀疏獎勵策略花費 43 步 (8.6 秒) 到達目標，而密集獎勵策略則花費 53 步 (10.6 秒)。 ...	63
4.9 Gazebo ...	64
4.10 真實世界 ...	64
4.11 機器人在真實世界實驗中的軌跡。黃色矩形為障礙物，藍色圓圈表示連續目標點。 ...	64
4.12 提議框架的架構。它由三個部分組成：感知 (perception)、控制 (control) 和隨機切換 (stochastic switch)。感知部分處理觀測值並生成相應的輸入表示。控制部分包含一個標準的 DDPG 控制器和若干獨立控制器——在此案例中，包含一個比例-積分-微分 (PID) 控制器和一個避障 (OA) 控制器。隨機切換則決定應選擇哪個控制器的動作來進行導航。 ...	66
4.13 (a) 4 個灰色矩形為障礙物，藍色正方形代表機器人。機器人上安裝了稀疏雷射，其偵測區域以綠色區域表示。(b) 顯示	

了由 ROS Stage 模擬的更複雜環境。(c) ROS Gazebo 也被用於訓練可遷移至真實世界環境的模型。Turtlebot 2 (一種地面機器人平台) 被用作配備深度攝影機的移動平台。 ... 71

4.14 我們的模型與其他基準模型對比所獲得的總獎勵。曲線代表 5 次重複訓練過程的平均值，透明區域則表示結果的變異量。 ... 73

4.15 透過將不同的啟發式控制器與 DDPG 結合後所獲得的平滑總獎勵。SGuidance (PID+OA) 同時利用了 PID 和 OA 控制器，而 SGuidance (OA) 和 SGuidance (PID) 則分別僅採用其中之一。SGuidance (movebase) 由 movebase 引導，而 DDPG (Vanilla) 則是沒有啟發式引導的基準 DDPG。 ... 74

4.16 使用不同切換機制進行學習所獲得的累積獎勵。Stochastic 代表我們預設的隨機切換設定。Argmax 與 Uniform 分別是提議的 argmax 切換與均勻分佈切換。ThompsonSampling 則是使用高斯先驗實現的 Thompson Sampling ... 75

4.17 在三個不同學習階段中，使用各個控制器 (DDPG、PID 或 OA) 的空間密度 (上方為 SGuidance，下方為 ThompsonSampling)。每一列代表單一控制器的使用情況，每一欄則表示訓練的早期 (0 – 10k 步)、中期 (40k-50k 步) 或晚期 (80k-90k 步) 階段。請注意，所有灰色區域皆為障礙物。 ... 76

4.18 分別應用 SGuidance 與 ThompsonSampling 時，在一個回合 (episode) 中達到不同最終狀態之比率的圖示。每一欄代表一個訓練時期，每個長條圖中的不同區段則表示該時期內不同控制器的使用比例。 ... 77

4.19 使用不同隨機切換函數之影響的圖示。左側 γ 軸顯示所有方法的總獎勵，右側 γ 軸顯示獨立控制器的總使用次數。 ... 78

4.20 在圖 4.13b 所示的複雜環境中，使用稀疏獎勵 (sparse rewards) 的實驗結果。除了每一步的時間懲罰外，機器人僅在碰撞或到達目的地時才會獲得獎勵。前兩行代表在部署或不部署引導 (guidance) 的情況下，每個導航回合 (episode) 中不同結束狀態 (碰撞、逾時或成功) 的比例。最後一行則顯示了每個回合的總獎勵。 ... 79

4.21 SGuidance 在不同折扣因子 (discount factor) γ 下，各個控制器的使用率。每張圖表繪製了在不同 γ 下，三種控制器的使用率。 ... 81

4.22 (a) SGuidance 與 DDPG 模型在 ROS Gazebo 模擬器中獲得的總獎勵比較。請注意，由於 Gazebo 模擬了物理特性，與 ROS stage 相比，其控制策略更難學習，且訓練需要更長的時間。(b) 真實世界場景。使用 TurtleBot 作為移動平台，並在房間內放置數個紙箱作為障礙物。(c) 房間佈局與障礙物 (黑色區域)。藍色曲線代表機器人的軌跡，目標點以紅圈標示，其中的數字表示順序。 ... 82

5.1 具有稀疏方向引導 (sparse directional guidance) 的視覺導航任務範例。機器人會根據當前的觀測結果，從提供的引導中自動進行選擇。 ... 85

5.2 SnapNav 的網路架構由兩個模組組成。首先，指揮官 (commander) 透過尋找與當前觀測結果正確匹配的內涵，來關注特定的引導指令，接著發布高階指令。隨後，控制器 (controller) 根據該指令、估計的深度圖像以及前一次預測的動作，來預測低階機器人動作。請注意，指令以縮寫「CMD」表示，「EMB」代表線性嵌入層 (linear embedding layer)，「GRU」表示閘控循環單元 (Gated Recurrent Unit) [25]，而 $\oplus$ 則是串接 (concatenation) 運算。

5.3 影片首先根據估計的光流 (optic flow) 進行分割。接著，在代理人 (agent) 實際轉向之前，會先分配方向變化指令 (黃色和綠色區段) 的偽標籤 (pseudo labels)。重要的是，這些標籤應隨機標記，而非根據地面真值 (ground-truth)。「停止 (Stop)」指令僅在影片的最後部分 (紫色部分) 進行標記，其餘影格則設定為預設指令「前進 (go forward)」 (藍色區域)。 ...

5.4 ROS Gazebo 中的隨機環境及其地圖範例。系統設計了一個神諭指揮官 (oracle commander)，在機器人到達轉彎點和終點前提供正確指令。紅圈標示出一個具挑戰性的情況：該路口未完全受障礙物限制，容易使機器人產生混淆。 93

5.5 採樣影片中編碼圖像的歐幾里得距離矩陣 (Euclidean distance matrix)。 96

5.6 DRQN、RDPG 與 DDPG 的平滑學習曲線。每種演算法皆在 ROS Gazebo 中使用 oracle commander 進行 100 萬步的訓練。 ... 97

5.7 真實世界測試範例。圖例由上至下分別為：起始位置、引導記錄位置、示範階段路徑、測試階段的隨機障礙物、機器人觀測值以及測試中的機器人軌跡。注意力位置以機器人軌跡中的不同顏色表示，並對應引導中動作的顏色。紅色與綠色圓圈強調了示範與測試之間環境視覺外觀與幾何形狀的變化。與測試。 ... 98

第 1 章

緒論

1.1 研究動機

在這個人工智慧興起的時代，移動機器人、無人機和自動駕駛汽車的自主運行，基本上都取決於是否存在一個可靠且智慧的導航系統。移動機器人的導航問題已經被研究了四十多年 [41]。其目標在概念上非常簡單：在避開障礙物的同時，尋找從機器人當前位置到目標點的最佳路徑。然而，要在各種情況下使系統穩健地運作，在實務上是非常困難的，這將在下文中討論。

為了安全地到達目標位置，傳統的機器人導航系統被明確地分為三個部分：地圖構建與定位模組 (mapping and localisation module)、全域路徑規劃器 (global path planner) 以及局部規劃器 (local planner)。局部規劃器遵循全域規劃器所建議的一系列航點 (waypoints)，而一個良好的全域規劃主要取決於精確的地圖和良好的位置估計。上述導航系統基本上是建立在環境的幾何描述 (例如地圖) 以及機器人的動力學模型之上。

使用地圖能帶來多項優勢。透過將高維度的觀測資料 (例如攝影機影像) 投影到地圖中的三維位姿，能使整個規劃與控制系統在運算上變得可行。此外，全域路徑的最佳性也能輕易得到保證。然而，這也存在一些限制。首先，為環境建構精確的地

圖既耗時又費力，且往往需要專家知識。其次，從長期角度來看，維護與更新地圖的成本可能更高，特別是在面對動態變化時。第三，控制效能完全取決於機器人的數學模型，但該模型通常會被簡化或線性化，最終削弱了導航系統的魯棒性。

無地圖導航 (Mapless navigation) 作為一種替代方案，被廣泛認為是減輕導航系統對地圖依賴的補救措施，因為它通常建立感測輸入與機器人動作之間的直接映射。遺憾的是，在沒有地圖的情況下，進行最佳路徑的全域規劃極其困難。因此，無地圖導航更常應用於沒有明確目的地的任務（例如避障），或是目的地在機器人局部座標系中為已知的任務（例如局部導航）。在此背景下，無地圖導航類似於基於行為的導航 (behaviour based navigation) [87]，後者不具備基於環境先驗知識的高階推理過程。然而，兩者在全域導航方面有很大差異，即機器人無法在其局部座標系中獲取目的地資訊。為了實現基於行為的導航通常無法處理的全域導航，無地圖導航系統通常會被提供某種環境資訊。這可以是如視覺示教與重複 (Visual Teach and Repeat, VT&R) [37] 中的影片，這些影片相對容易與攝影機觀測結果匹配，但在記憶體和通訊方面顯得過於密集且低效。值得注意的是，局部導航與全域導航的目標都是透過最短路徑到達目的地，唯一的區別在於是否給定每個時間步長下目的地與機器人之間的相對位姿。自然語言 [24] 作為最高效的溝通媒介之一，也被探索作為無地圖導航的引導方式，但其稀疏性以及語言指令與感測觀測之間的跨模態問題，很大程度上阻礙了其在即時控制中的應用。

作為一種新興範式，深度強化學習 (Deep Reinforcement Learning, DRL) 已被證明能夠精通如圍棋 [135] 等複雜任務，並能處理來自各個領域的觀測數據，例如影像 [109]、雷射掃描 [146] 和語言 [133]。作為一種純數據驅動的方法，DRL 在多方面優於傳統方法。其一在於擺脫了手工設計的控制規則，這些規則通常並非最優，因為它們很大程度上取決於設計者有限的經驗以及對任務的認知。AlphaGo Zero [137] 對人類棋手的超越，已明確證明了純數據驅動的方法可以突破人類策略的局限。另一個優勢在於其具備從高維度領域學習的能力，無論是在感測器觀測還是機器人的物理模型方面，這對於啟發式方法 (heuristic approaches) 而言，在資訊理論和計算上都非常困難。

與其他機器學習機制（如監督式學習或模仿學習）相比，強化學習更適用於具有複雜獎勵結構以及對環境有強烈連續互動需求的問題 [66]。無地圖導航 (Mapless navigation) 是一個典型的例子，代理人 (agent) 可能只有在行經長距離、且未發生碰撞或超時的情況下成功抵達目的地，最終才能獲得單一的正向獎勵。最終的獎勵是途中多次動作的結果，包括接近目的地和避開障礙物。這些挑戰使得 DRL 難以訓練，因為其搜尋空間極其龐大。

然而，DRL 在實際機器人任務中的應用也帶來了重大挑戰，這些挑戰將在下一節中討論。尋求這些挑戰的解決方案是本論文的最終目標，願景是創造出實用的 DRL 方法來實現無地圖視覺導航，使其具備高效的訓練能力、泛化性，並能部署在真實機器人上。

1.2 研究挑戰與問題

儘管 DRL 在虛擬遊戲中展現了令人驚嘆的性能，但在充滿雜訊且動態的現實世界問題中，要取得相當的成就仍然具有挑戰性。

最顯著的問題在於其樣本使用效率低下。在複雜的現實環境中，DRL 深受高變異性 (high variance) 挑戰的困擾。這由多種原因造成 [66]，其中之一是高維度真實機器人系統所需的數據與運算量呈指數級增長，這被稱為「維度詛咒」 (Curse of Dimensionality) [11]。為了充分探索龐大的狀態與動作空間，必須透過與現實環境的互動累積大量的學習樣本。因此，大多數機器人強化學習問題無法直接利用現實世界的樣本進行訓練，因為機器人硬體昂貴且容易損壞。此外，從真實機器人獲取訓練回合 (episodes) 速度緩慢且需要人工干預。因此，在模擬器中進行訓練程序，然後將學到的策略應用於現實場景 [146, 181] 是一個更可行的選擇。透過加速模擬可以縮短訓練週期，且虛擬世界在發生碰撞後可以輕鬆重置而不會造成實際損壞。另一方面，使用模擬也會帶來建模不足 (under-modelling) 和模型不確定性的問題，阻礙我們將學到的策略直接從虛擬世界轉移到現實世界，這通常被稱為「現實差距」 (reality gap)。因此，關鍵挑戰之一在於如何彌合虛擬與現實世界在感測觀測（如視覺外觀）和物理現象（如機器人動力學）方面的差距。

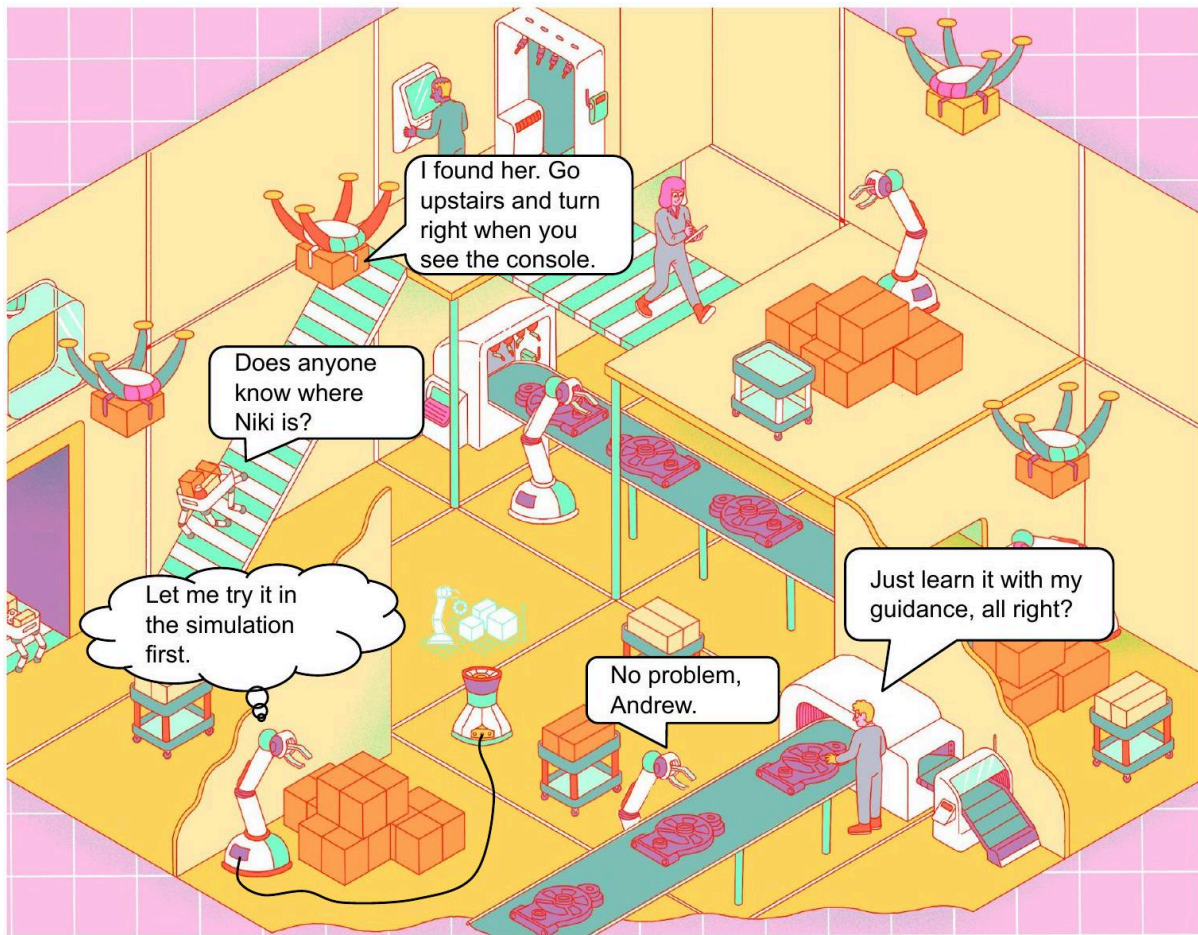


圖 1: 圖 1.1: 一幅漫畫 (基於 YimelsGreat 的插圖), 想像了機器人與人類密切合作的近未來。它也展示了本論文中提到的一些核心概念: 1) 將學到的策略從模擬環境轉移到現實世界。2) 透過額外的引導加速學習過程。3) 學習在稀疏的方向指令和視覺參考下進行導航。

另一個重要的挑戰是將學習到的經驗從已知場景遷移到未知場景的泛化能力。在我們的案例中, 這指的是在具有陌生幾何形狀的環境中導航的能力, 在這種情況下, 機器人無法透過回想訓練環境中記憶的路線來快速到達目的地。對於避障或區域導航等原始任務來說, 這相對容易, 因為可以從局部觀測中學習到最佳動作; 但對於全域導航來說, 這並非易事, 必須提供額外的引導資訊。

目標規範 (Goal specification) 是另一個重大挑戰。由於在強化學習中, 預期行為是透過獎勵函數隱含地指示, 因此合適的獎勵函數至關重要, 但在實際的機器人問題中可能相當難以設計。例如, 其中一個矛盾存在於稀疏 (sparse) 與密集 (dense/frequent) 獎勵之間。

獎勵。為了根據專家的先驗知識影響行為而給予密集獎勵, 可以大幅加快訓練速度, 但最終可能會促使代理人學習到該任務的次佳策略 (sub-optimal policy)。另一方面, 稀疏獎勵函數要求學習者進行大量的隨機探索, 因為它只有在觀察到各種負面獎勵後才能獲得正面經驗, 也就是說, 它必須先學習「如何不去做」這項任務 (負面獎勵), 而不是直接學習「如何做」這項任務 (正面獎勵)。這會導致 DRL 策略的收斂速度非常緩慢。

本論文旨在提出一個應用於現實場景的 DRL 無地圖導航系統, 並致力於解決以下問題:

1. 我們該如何將學習到的策略 (policy) 從虛擬環境遷移到真實環境?

這是利用模擬器訓練 DRL 模型的關鍵問題, 因為它影響了學習到的策略最終是否對真實機器人問題具有價值。然而, 如前所述, 在模擬器中訓練是快速訓練 DRL 網路唯一可行的方法。這可以從兩個角度來考慮: 遷移感知 (perception) 或動作 (action), 具體取決於所使用的模擬器。感知代表機器人如何從其觀測值 (observations) 中理解周圍環境, 而動作則反映了在給定觀測值下所選擇的控制訊號。就物體的外觀、多樣性以及物理現象的抽象化而言, 模擬世界是真實世界的高度簡化模型。因此, 模擬機器人的感知與行為方式與真實機器人有所不同。如何縮小甚至消除這一差距是第 3 章的主要探討主題。

2. 我們該如何加速在模擬環境中的訓練?

雖然模擬器相對於真實世界已大幅加快了訓練過程, 但它們仍需要大量的訓練回合 (episodes) 才能學習到良好的策略。提高數據效率並降低訓練變異量 (variance), 是我們改進 DRL 演算法的第二種方法。

即使在運行速度比真實世界快上數倍的模擬器中, 訓練出一個優秀的網路仍可能需要數天的時間。在機器學習社群中, 大量的研究集中於引入能良好泛化到不同問題的通用偏置 (general bias) [101, 127, 2, 45]。然而, 這些演算法並未考慮特定任

務的情境資訊，而這些資訊對於訓練而言可能非常有價值。

以我們的案例來說，機器人導航任務確實存在現有的解決方案，可以輔助 DRL 訓練。雖然這些解決方案未必是最優的，但在大多數情況下，它們的表現仍優於隨機探索策略。如何充分利用並受益於這些先驗方法，並將其與 DRL 緊密結合以加速訓練，是第 4 章所討論的問題。

3. 我們該如何進行長距離的無地圖導航？

對於短距離導航任務（如避障或已知目的地的局部導航），無地圖導航並不困難，因為機器人的局部觀測資訊已足以做出動作決策。然而，長距離導航的問題則有所不同。它需要一個全域規劃程序來選擇正確的局部動作，而這通常依賴於對環境的描述。受到人類詢問前往目的地路線時的行為啟發，我們在第 5 章討論了移動代理人如何學習在稀疏的方向指引和視覺參考下進行導航。

1.3 貢獻

在本節中，我們首先將本論文各章節的貢獻總結如下：

在第 3 章中，我們提出了一種兩階段深度神經網路，以實現基於單眼視覺的障礙物規避。這是首次將 D3QN 網路架構應用於學習機器人任務，並對提升學習效率做出了貢獻。作為一項重大且具創新性的貢獻，本研究首次利用估計的深度資訊來彌補模擬環境與真實世界之間單眼視覺感知的差距。我們進行了大量的實驗，展示了該視覺障礙物規避系統在各種真實場景中的高效能。此項工作發表於：

Linhai Xie, Sen Wang, Andrew Markham, and Niki Trigoni. Towards monocular vision based obstacle avoidance through deep reinforcement learning. Robotics: Science and System 2017 workshop on New Frontiers for Deep Learning in Robotics.

2. 在第四章中，我們提出了一個新的框架，能有效地整合啟發式知識（heuristic knowledge），以克服使用 DDPG 學習策略時的高變異性問題。這是首次將專家或初級的傳統控制器「軟性」地結合到學習過程中。我們證明了它們不僅能在早期階段加速底層策略（off-policy）DRL 方法（即 DDPG）的學習過程，且當學習到的策略表現優於這些控制器時，它們可以被完全捨棄，讓 DDPG 透過自我探索繼續學習。我們提出了確定性（AsDPG）與隨機性（SGuidance）技術，並比較了它們各自的優點。這兩種演算法都大幅加速並穩定了 DDPG 的訓練，其中 SGuidance 達到了接近預言機（near-oracle）的效能。此外，我們展示了所提出的切換機制如何根據輸入和預期獎勵動態地學習選擇哪種控制器，其表現優於僅根據預期獎勵進行學習的切換機制（例如湯普森採樣，Thomson sampling）。我們還證明了這項技術對於稀疏獎勵（sparse rewards）特別重要，在這種情況下，原始的 DDPG 無法學會如何完成整個回合。此項工作已發表如下：

Linhai Xie, Sen Wang, Stefano Rosa, Andrew Markham, and Niki Trigoni. Learning with training wheels: speeding up training with a simple controller for deep reinforcement learning. In 2018 IEEE International Conference on Robotics and Automation (ICRA), pages 6276-6283. IEEE, 2018

Linhai Xie, Yishu Miao, Sen Wang, Phil Blunsom, Zhihua Wang, Changhao Cheng, Andrew Markham, and Niki Trigoni. Learning with stochastic guidance for robot navigation. Under review at IEEE Transactions on Neural Networks and Learning Systems.

3. 在第 5 章中，我們提出了一種新型的視覺導航系統，使機器人能夠在僅有極稀疏引導的情況下，於未知環境中進行導航。我們設計了一種雙層架構的網路，每個模組都可以透過不同的學習機制，以及來自各個領域與來源的數據進行訓練。這賦予了導航系統更高程度的自主性，並能輕易地將在模擬環境中學到的策略轉移至現實世界。高層級指揮器（high-level commander）是透過一種結合多種學習訊號的新型自我監督訓練方法所訓練而成。最後，實驗證明訓練後的網路能在現實世界的實驗中穩健地進行導航。此項工作發表於以下論文：

Linhai Xie, Andrew Markham, and Niki Trigoni. SnapNav: Learning mapless visual navigation with sparse directional guidance and visual reference. Accepted at 2020 IEEE International Conference on Robotics and Automation (ICRA).

我在博士生涯期間的其他貢獻也列舉如下：

Linhai Xie, Sen Wang, Andrew Markham, and Niki Trigoni. “GraphTinker: Outlier rejection and inlier injection for pose graph SLAM.” In 2017 IEEE/RSJ

International Conference on Intelligent Robots and Systems (IROS), pp. 6777-6784. IEEE, 2017.

2. Linhai Xie, Yishu Miao, Sen Wang, Phil Blunsom, Zhihua Wang, Changhao Cheng, Andrew Markham, and Niki Trigoni. “Learning with Stochastic Guidance for Navigation.” NeuraIPS 2018 workshop on Infer2Control.

3. Zhihua Wang, Stefano Rosa, Linhai Xie, Bo Yang, Sen Wang, Niki Trigoni, and Andrew Markham. “Defo-Net: Learning body deformation using generative adversarial networks.” In 2018 IEEE International Conference on Robotics and Automation (ICRA), pp. 2440-2447. IEEE, 2018.

4. Lu Chris Xiaoxuan, Yang Li, Peijun Zhao, Changhao Chen, Linhai Xie, Hongkai Wen, Rui Tan, and Niki Trigoni. “Simultaneous localization and mapping with power network electromagnetic field.” In Proceedings of the 24th Annual International Conference on Mobile Computing and Networking, pp. 607-622. ACM, 2018.

5. Changhao Chen, Yishu Miao, Chris Xiaoxuan Lu, Linhai Xie, Phil Blunsom, Andrew Markham, and Niki Trigoni. “MotionTransformer: Transferring Neural Inertial Tracking between Domains.” In Proceedings of the AAAI Conference on Artificial Intelligence, vol. 33, pp. 8009-8016. 2019.

6. Hu, Qingyong, Bo Yang, Linhai Xie, Stefano Rosa, Yulan Guo, Zhihua Wang, Niki Trigoni, and Andrew Markham. “RandLA-Net: Efficient semantic segmentation of large-scale point clouds.” arXiv preprint arXiv:1911.11236 (2019).

第 2 章

背景知識

在本章中，我們將全面概述強化學習（Reinforcement Learning）與機器人導航的基礎知識及近期研究。首先，我們探討了幾種深度強化學習（Deep Reinforcement Learning, DRL）解決方案，並討論其主要優缺點。接著，我們討論了基於地圖（map-based）的機器人導航之公式化定義與局限性，隨後介紹了無地圖（mapless）導航的動機與近期研究。

本章結構安排如下：第 2.1 節描述強化學習問題的公式化定義及其解決方案。第 2.2 節與第 2.3 節分別回顧基於地圖的機器人導航與無地圖機器人導航。最後，第 2.4 節討論了目前最先進的 DRL 導航方法的局限性。

2.1 強化學習

強化學習（Reinforcement Learning, RL）是機器學習的一個分支，研究如何從互動中學習，這被認為是人類最自然的學習經驗。當一個出生時不具備先驗知識的嬰兒觀察環境並與之互動時，他們可以迅速累積關於物理世界中因果關係的資訊。這是在沒有任何指導的情況下，僅透過與環境的互動（即動作與感官觀測）來實現的。這種學習機制在我們的生活中永無止境，且是許多智慧代理人（intelligent agents）的基礎。

2.1.1 馬可夫決策過程 (Markov Decision Process)

在理論公式中，強化學習 (RL) 問題被建模為一個馬可夫決策過程 (MDP)，其將狀態映射到動作，旨在最大化一個數值型的獎勵訊號 (reward signal)。因此，我們首先需要概述 MDP 的原理 [10]。

馬可夫性質 (Markov property) 指出，在給定當前狀態的情況下，未來狀態與過去狀態無關。更具體地說，在從時間 0 到 T 的狀態序列 $x_0, x_1, x_2, \dots, x_T$ 中，我們說狀態 $x_t \in X$ 具有馬可夫性質，若且唯若

$$P(x_{t+1} | x_t) = P(x_{t+1} | x_0, x_1, \dots, x_t),$$

其中 X 為狀態空間，而 P 為轉移機率函數，用以描述給定當前與歷史狀態下，下一個狀態的分佈情形。基於此特性，馬可夫過程 (Markov process) 是一種無記憶的隨機過程，即滿足馬可夫動力學的一系列隨機狀態 $x_0, x_1, \dots$ 。當我們根據轉移機率函數 P 對此過程進行採樣時，所得的序列稱為一個回合 (episode)。此外，藉由引入作為價值判斷的獎勵函數 r ，我們可得到馬可夫獎勵過程 (Markov reward process)。在此，獎勵函數 r 用於評估採樣的回合，其方式是將每次狀態轉移時的即時獎勵 r_t 累加為 $\sum_{\tau=1}^T r_\tau$ 。最後，透過在轉移函數中引入動作 $a \in A$ 作為 $P(x_{t+1} | x_t, a_t)$ ，即可定義馬可夫決策過程 (Markov Decision Process)。因此，MDP 亦可表示為一個多元組 (X, A, P, r) ，由狀態空間 X 、動作空間 A 、轉移機率函數 P 以及獎勵函數 r 所組成。此外，從時間 0 到 T 採樣的序列 $x_0, a_1 x_1, \dots, a_T, x_T$ 被稱為軌跡 (trajectory) s 。

2.1.2 核心要素

強化學習 (RL) 問題的核心要素如圖 2.1 所示。在時間 $t \in [0, T]$ ，學習者（稱為代理人，agent）根據代表環境當前狀態的狀態 x_t 採取動作 $a_t \in \mathcal{A}$ 。執行動作後，機器人會根據獎勵函數獲得由環境給予的獎勵 r_t ，並轉移至下一個狀態 x_{t+1} 。請注意，代理人在時間 t 採取的每個動作都可能影響未來所有後續的觀測、動作與獎勵，因此當前動作的有意義回饋可能會延遲多個步驟。此決策過程的目標是達成最大的折扣累積未來獎勵 $R = \sum_{\tau=t}^T \gamma^{\tau-t} r_\tau$ ，這意味著從當前狀態到達目標狀態的最優動作序列。在此， γ 被定義為折扣因子 (discount factor)。

除了上述定義外，我們在此想介紹強化學習中其他一些基本概念。

策略 π 是代理人根據當前觀測值 x_t 來決定動作 a_t 的策略。它直接將狀態映射到動作：

$$a_t = \pi(x_t)$$

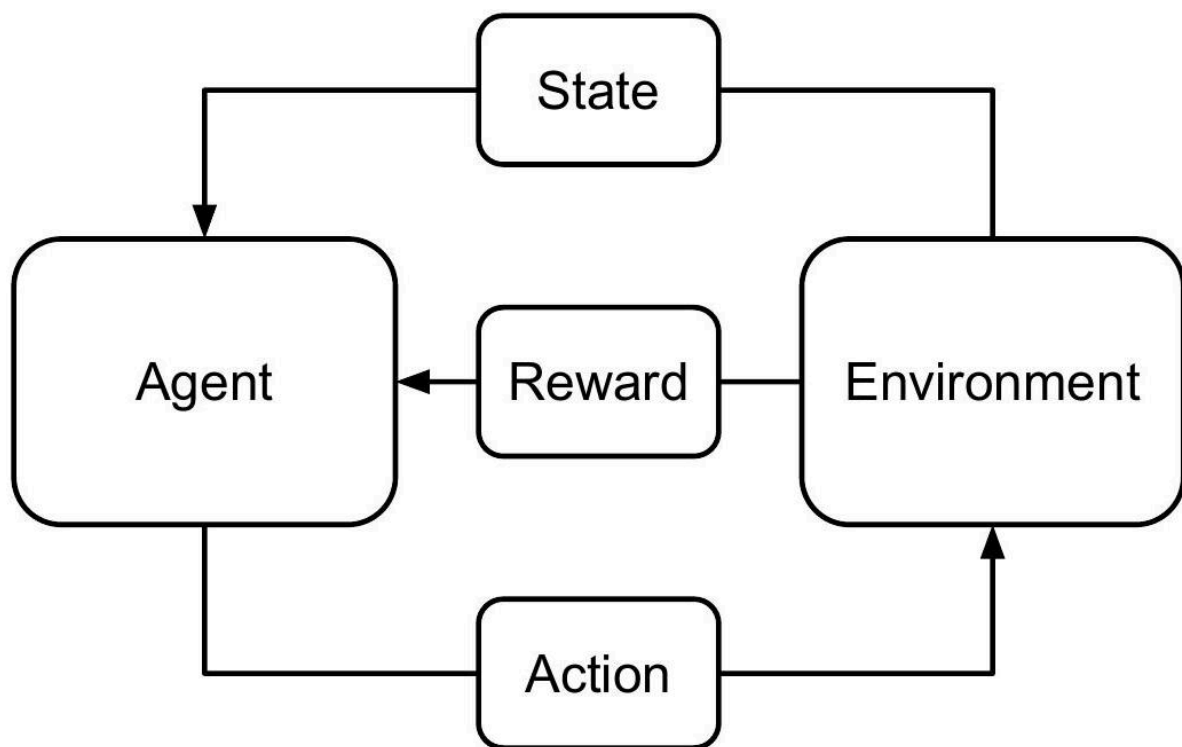


圖 2：圖 2.1：在 RL 問題中，學習代理人在每個時間步觀測環境狀態，並執行動作以影響環境。根據不同的動作，環境會回傳一個數值獎勵作為給代理人的回饋，用以評估該行為是否為預期行為。

在此公式中，我們假設的是確定性策略（deterministic policy），但它也可以是隨機性策略（stochastic policy），即動作是從分布中取樣而得。此外，動作可以是連續的或離散的。連續動作通常會導致學習者的搜尋空間大得多，且在大多數情況下更難學習。

數值 V 是未來折扣獎勵的期望總和 $\mathbb{E} \left[\sum_{\tau=t}^T \gamma^{\tau-t} r_{\tau} \mid \pi \right]$ ，其中折扣因子 γ （也用於 R ）用於降低長期未來獎勵的權重，並避免在無窮序列中出現無限大的數值。數值 V 是在給定特定策略 π 和狀態 x 的情況下定義為 $V_{\pi}(x)$ 。Q 值與數值 V 相似，但不同之處在於 Q 值 $Q_{\pi}(x, a)$ 進一步將動作定義為額外參數。由於獎勵 r_t 是衡量在狀態 x_t 下採取動作 a_t 有多大益處的指標，我們可以認為數值 $V_{\pi}(x)$ 代表在給定狀態 x 下對策略 π 的評估。Q 值 $Q_{\pi}(x, a)$ 則進一步衡量在狀態 x 下特定動作 a 是否良好。

2.1.3 與其他學習範式的比較

RL 與廣義的監督式學習（supervised learning）或非監督式學習（unsupervised learning）不同。前者是最廣泛使用的學習方法，旨在給定標記數據（labelled data）的情況下，尋找從輸入到輸出變數的映射函數。標記過的學習樣本將每個最佳輸出與輸入變數連結起來。訓練過程是透過最小化代理器（agent）的預測值與每個輸入狀態標籤之間的誤差來進行。憑藉足夠的訓練範例，代理器可以將學到的知識泛化到未見過的情況。這是一種高效的學習方法，但不適用於互動式問題，因為這類問題通常沒有先知（oracle）能提供最佳解決方案。相反地，在非監督式學習中，學習者的目標是尋找未標記數據中的隱藏結構，然而這並無助於達成透過一系列動作實現目標的任務。因此，RL 是從互動中學習最合適的範式。它也適用於許多複雜的機器人應用，在這些應用中，由於高維度問題，往往無法獲得最佳軌跡。因此，必須透過與環境互動來探索解決方案。

RL 面臨一個特有的挑戰——利用與探索（exploitation and exploration）的兩難困境。一方面，代理器傾向於「利用」那些它估計能獲得高獎勵的動作；另一方面，那些高獎勵的動作只能透過嘗試未經「探索」的動作來發現。此外，獎勵函數（reward function）的設計也至關重要。在獎勵稀疏且延遲的情況下（例如：下棋時僅將最終結果作為獎勵），代理器可能要在經過數百萬次嘗試後才能捕捉到正向的學習訊號。然而，使用密集但人工設計的獎勵函數進行學習，可能會阻礙代理器進步到最佳策略（optimal policy）。

2.1.4 歷史

RL 的歷史可以分為三個脈絡，我們根據 [141] 對其進行簡要描述。第一個脈絡源自於最佳控制（optimal control）[58]，其主要方法是使用價值函數（value functions）和動態規劃（dynamic programming）[12]。Bellman 還引入了最佳控制問題的離散隨機版本，稱為馬可夫決策過程（Markovian Decision Process, MDP）[10]，隨後 Ron Howard 提出了策略迭代（policy iteration）方法 [56] 來解決 MDP。這些概念和理論為現代 RL 鋪平了道路。另一個脈絡稱為試錯法（trial-and-error），這也是我們最熟悉的。它是建立在最初由 Edward Thorndike 稱為「效果律」（Law of Effect）[149] 的真實「強化」學習概念之上。簡單來說，就是增加選擇具有良好回饋之動作的傾向，並減少對具有不良回饋之動作的偏好。沿著這個脈絡，「強化學習」一詞首次出現，並且在 [97] 中研究了 RL 中的幾個重要問題，例如信用分配（credit assignment）。

最後一個脈絡是時序差分學習 (temporal-difference learning)。其核心思想是學習在兩個連續時間步長下，對某個值 (例如未來總獎勵) 估計之間的差異。Klopf [65]、Sutton [140] 和其他研究者的想法為完善這一脈絡做出了巨大貢獻。

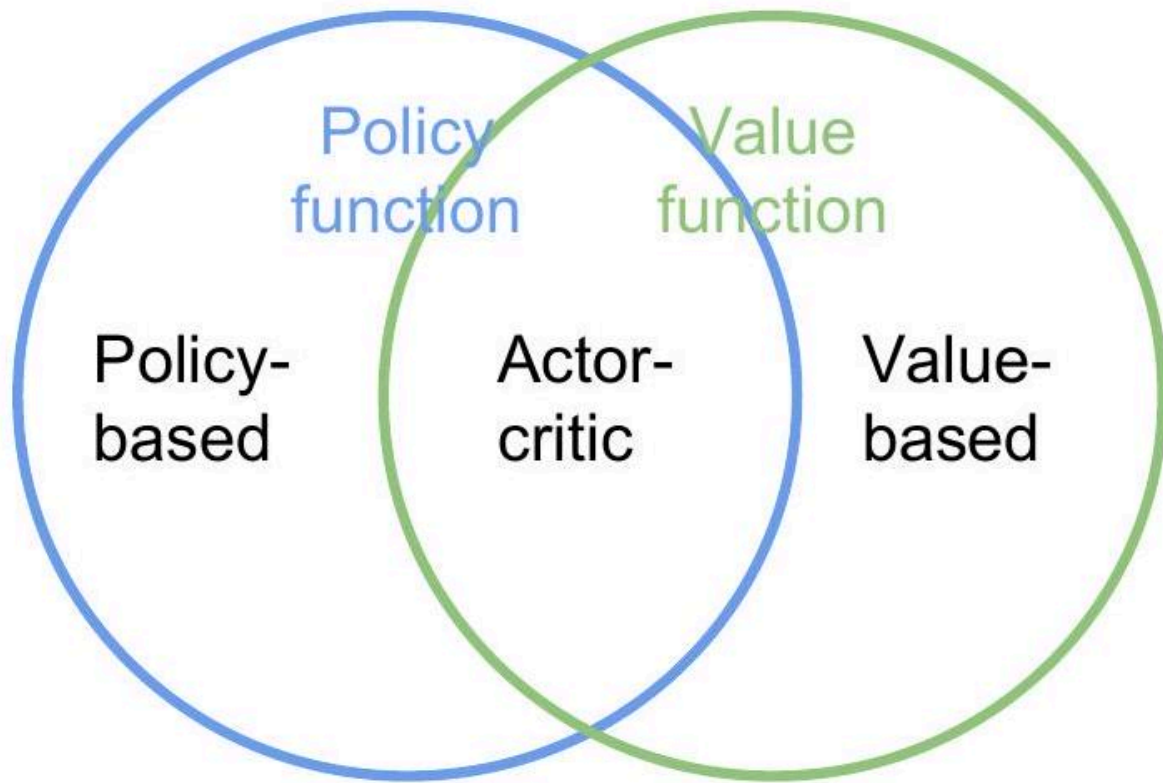


圖 3：圖 2.2：三類無模型 (model-free) RL 方法，即基於策略 (policy-based)、基於價值 (value-based) 和行動者-評論家 (actor-critic) 方法。

最後，透過結合時序差分和試錯法這兩個脈絡，行動者-評論家 (actor-critic) 架構於 1983 年被引入 [8]。此外，著名的 Q-learning 方法在 [163] 中被提出，將時序差分和最佳控制這兩個脈絡結合在一起。

近年來，深度學習帶動了深度強化學習 (Deep Reinforcement Learning, DRL) 的研究熱潮。在高複雜度領域中的數值估計與策略搜尋問題，皆透過理論上能逼近任何函數的深度神經網路得到了緩解。在本論文中，我們利用了三種 DRL 演算法，並將在不同章節中詳細介紹，分別是：Deep Q-learning [103]、Deep Deterministic Policy Gradient [79] 以及「深度」版本的 REINFORCE 演算法 [164]。

2.1.5 方法

在本論文中，我們專注於無模型 (model-free) 方法。這是因為隨著深度神經網路近期的發展，無模型強化學習 (RL) 已被證明在高維度狀態空間中非常有效。然而，在高維度狀態空間中學習現實世界環境的模型，在計算上的成本非常昂貴。

無模型方法可分為三個不同類別，即基於數值 (value-based)、基於策略 (policy-based) 以及動作及評論者 (actor-critic) 方法。

策略型方法 (Policy-based approaches)，例如 REINFORCE [164]，會直接優化策略函數 π 。其主要方法是估計策略函數相對於總獎勵的梯度。透過遵循這些梯度，策略將得到改進以獲得更高的獎勵。價值型方法 (Value-based methods) 則從相反的角度學習，因為它們僅估計價值或 Q-values，例如 Q-learning [163]。在已知未來獎勵總和的情況下，策略會變得簡單，即選擇能夠使未來獎勵最大化的動作。

策略型方法的主要優點是具有較好的收斂特性。這是因為策略函數會隨著梯度連續更新，從而不斷改進策略，並保證至少能收斂到局部最小值 [114]。另一個優點是其在連續或高維度動作空間中的天然相容性。在價值型方法中通常使用的最大化操作 (maximisation operation)，當有數百萬個動作可供選擇時，其計算成本可能高得令人望而卻步。價值函數必須準確估計每個動作的 Q-value，才能最終找到最佳動作。最後一個優點是策略型方法可以學習隨機策略 (stochastic policies)，在某些情況下，隨機策略的表現優於確定性策略，這將在第 5 章進一步討論。然而，與價值型方法相比，其缺點也不容小覷。策略型方法更容易陷入局部最小值，且其學習過程受高變異數 (high variance) [114, 67] 的影響，這也是第 4 章的主要研究動機。

第三類被稱為 Actor-Critic，它同時使用價值函數和策略函數。這兩個函數在學習過程中會迭代更新。Critic 函數會朝著更準確的價值或 Q-values 估計進行更新。接著，策略函數會根據 Critic 函數建議的方向進行優化，並學習在不同情況下的更佳動作。從 Critic 函數計算出的梯度比策略型方法中的梯度更穩定，因為後者是從取樣軌跡的總獎勵中估計的，而前者則是基於 Critic 估計的期望值 [67]。

2.1.6 深度強化學習演算法

我們現在將討論本論文中實作並經過廣泛測試的三種不同 DRL 演算法。

2.1.6.1 Deep Q Network

DQN [104] 是一種典型的基於價值 (value-based) 的 DRL 演算法。給定策略 $a_t = \pi(x_t)$ ，狀態-動作對 (x_t, a_t) 的 Q 值 (Q-value) 定義如下

$$Q^\pi(x_t, a_t) = \mathbb{E}[R_t \mid x_t, a_t, \pi]$$

其中 $R_t = \sum_{\tau=t}^T \gamma^{\tau-t} r_\tau$ 是從時間 t 開始的總未來獎勵，並由 γ 進行折現。Q 值函數可以使用 Bellman 方程式計算

$$Q^\pi(x_t, a_t) = \mathbb{E}[r_t + \gamma \mathbb{E}[Q^\pi(x_{t+1}, a_{t+1}) \mid x_{t+1}, a_{t+1}, \pi] \mid x_t, a_t, \pi].$$

透過每次選擇最佳動作 $Q^*(x_t, a_t) = \max_{a_t} Q(x_t, a_t)$ 的確定性貪婪策略 (deterministic greedy policy)，我們可以得到最佳 Q 值函數

$$Q^*(x_t, a_t) = \mathbb{E}\left[r_t + \gamma \max_{a_{t+1}} Q(x_{t+1}, a_{t+1}) \mid x_t, a_t\right]$$

這表示我們在時間 t 所能獲得的最佳 Q 值，是當前獎勵 r_t 加上時間 $t+1$ 可獲得的折現最佳 Q 值，而不是直接在龐大的狀態空間上計算 Q 值函數。

接著，以 θ^Q 為參數的深度神經網路被用作 Q 值估計器 $Q(x_t, a_t \mid \theta^Q)$ 。給定一個特定的學習樣本 (x_t, a_t, r_t, x_{t+1}) ，用於優化網路參數的損失函數公式如下：

$$\mathcal{L}(\theta^Q) = [y - Q(x_t, a_t \mid \theta^Q)]^2$$

其中目標為 $y = r_t + \gamma \max_{a_{t+1}} Q(x_{t+1}, a_{t+1} \mid \theta^Q)$ 。請注意，由於貪婪策略 (greedy policy) π 在所有可能的動作中包含最大化 (max) 運算，因此 DQN 僅適用於離散動作空間。

在上述原始 DQN 的基礎上，雙重網路 (double network) [156]、優先經驗回放 (prioritised experience replay) [128] 以及競爭網路 (dueling network) [161] 是三種能大幅提升其效能的技術。

Double DQN：在上述 DQN 的公式中，貪婪策略使用相同的 Q 值來評估並選擇動作，如 Hasselt 等人 [156] 所指出，這會導致過於樂觀的價值估計。為了緩解此問題，DDQN 採用了以下目標：

$$y' = r_t + \gamma \max_{a_{t+1}} Q(x_{t+1}, a_{t+1} \mid \theta^{Q'})$$

其中 $\theta^{Q'}$ 是目標 Q 網路 (target Q network) 的參數，該網路會緩慢地追蹤 θ^Q 。因此，所有可能的動作都由目標 Q 網路進行評估，而貪婪選擇仍由原始 Q 網路做出。

優先經驗回放 (Prioritised experience replay) 作為一種離策略 (off-policy) 演算法，DQN 將學習過的經驗樣本儲存在回放緩衝區 (replay buffer) 中，並隨機均勻地抽取一批數據進行訓練。[128] 提出為那些能帶來較大學習進展的樣本分配較高的採樣機率，而這種對學習進展的重要性是透過 Q 值估計的誤差來近似的。該方法在多款 Atari 遊戲中展現了更快的學習速度以及更好的效能。

深度遞迴 Q 網路 (Deep Recurrent Q Learning, DRQN) [49] 由 Hausknecht 等人提出，旨在解決部分可觀測馬可夫決策過程 (Partially Observable MDP, POMDP)；在這種情況下，DQN 會因為狀態觀測不完整而導致效能下降。其與 DQN 的核心差異在於利用遞迴神經網路 (RNN) 的優勢，將歷史資訊納入 Q 值估計中。

對鬥網路 (Dueling network) 基於「並非所有動作在每個狀態 s 下都需要被估計」的直覺，Wang 等人 [161] 提出了對鬥網路架構。在 DQN 中，卷積層之後僅構建一條全連接層流來估計每個「動作-狀態」對的 Q 值。然而，在對鬥網路中，則構建了兩條全連接層流分別計算價值函數 (value function) 與優勢函數 (advantage function)，最後再將兩者結合以計算 Q 值。優勢函數 $A(x_t, a_t \mid \theta^\alpha)$ 的定義為：

$$Q(x_t, a_t \mid \theta^\alpha, \theta^\beta) = V(x_t \mid \theta^\beta) + A(x_t, a_t \mid \theta^\alpha)$$

其中 θ^α 與 θ^β 分別代表 Q 網路中優勢流與價值流的參數。接著為了體現可識別性 (unidentifiable problem)，對鬥網路最終採用了以下公式：

$$Q(x_t, a_t | \theta^\alpha, \theta^\beta) = V(x_t | \theta^\beta) + \left[A(x_t, a_t | \theta^\alpha) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(x_t, a'_t | \theta^\alpha) \right]$$

其中 $\mathcal{A}$ 代表動作空間。

2.1.6.2 REINFORCE 演算法

REINFORCE [164] 是一種基於策略 (policy-based) 的演算法，其在光譜上的位置與 DQN 完全相反。本質上，

REINFORCE 透過強化取樣具有高預期總獎勵 $\mathbb{E} \left(\sum_{t=0}^T r_t \right)$ 之動作 $p = P^\pi(a_t | x_t, \theta^\pi)$ 的機率，來學習隨機策略。請注意， θ^π 代表策略網路的參數，且在此推導中，我們首先暫不考慮折扣因子 γ 。目標函數的定義如下

$$J(\theta^\pi) = V_0 = \mathbb{E} \left[\sum_{t=0}^T r_t | \pi \right] = \sum_s \sum_{t=0}^T P(x_t, a_t | s) r_t$$

其中 s 是 $x_0, a_1, x_1, a_2, \dots, x_T, a_T$ 的單一軌跡，其取決於策略 π ，而 $P(x_t, a_t | s)$ 表示在給定 s 的情況下， x_t 與 a_t 發生的機率。接著透過對兩邊進行微分，我們可以得到以下等式：

$$\begin{aligned} \nabla_{\theta^\pi} J(\theta^\pi) &= \sum_s \sum_{t=0}^T \nabla_{\theta^\pi} P(x_t, a_t | s) r_t \\ &= \sum_s \sum_{t=0}^T P(x_t, a_t | s) \frac{\nabla_{\theta^\pi} P(x_t, a_t | s)}{P(x_t, a_t | s)} r_t \\ &= \sum_s \sum_{t=0}^T P(x_t, a_t | s) \nabla_{\theta^\pi} \log P(x_t, a_t | s) r_t \\ &= \mathbb{E} \left[\sum_{t=0}^T \nabla_{\theta^\pi} \log P(x_t, a_t | s) r_t | s \right] \end{aligned}$$

在訓練過程中，使用 Monte Carlo 方法並透過 N 個隨機取樣的軌跡來近似上述梯度。

$$\nabla_{\theta^\pi} J(\theta^\pi) \sim \frac{1}{N} \sum_{n=1}^N \sum_{t=0}^T \nabla_{\theta^\pi} \log P(x_t, a_t | s) r_t$$

此外，透過展開 $\nabla_{\theta^\pi} \log P(x_t, a_t | s)$ 並捨棄不依賴於 θ^π 且對其梯度為 0 的項目，我們得到：

$$\nabla_{\theta^\pi} \log P(x_t, a_t | s) = \sum_{t'=0}^t \nabla_{\theta^\pi} \log P^\pi(a_{t'} | x_{t'}, \theta^\pi)$$

因此，

$$\begin{aligned} \sum_{t=0}^T \nabla_{\theta^\pi} \log P(x_t, a_t | s) r_t &= \sum_{t=0}^T r_t \left(\sum_{t'=0}^t \nabla_{\theta^\pi} \log P^\pi(a_{t'} | x_{t'}, \theta^\pi) \right) \\ &= \sum_{t=0}^T \nabla_{\theta^\pi} \log P^\pi(a_t | x_t, \theta^\pi) \left(\sum_{t'=t}^T r_{t'} \right) \end{aligned}$$

在納入折扣因子 γ 後，

$$\begin{aligned} \nabla_{\theta^\pi} J(\theta^\pi) &\sim \frac{1}{N} \sum_{n=1}^N \sum_{t=0}^T \nabla_{\theta^\pi} \log P^\pi(a_t | x_t, \theta^\pi) \left(\sum_{t'=t}^T \gamma^{t'-t} r_{t'} \right) \\ &= \frac{1}{N} \sum_{n=1}^N \sum_{t=0}^T \nabla_{\theta^\pi} \log P^\pi(a_t | x_t, \theta^\pi) R_t \end{aligned}$$

最後，策略網路的參數 θ^π 透過以下方式更新：

$$\theta^\pi \leftarrow \theta^\pi + \frac{\partial}{\partial \theta^\pi} J(\theta^\pi)$$

變異數縮減 (Variance Reduction) 由於梯度是使用蒙地卡羅方法 (Monte-Carlo method) 估算的，在較長的片段 (episode) 中可能會產生很大的雜訊。因此，相較於直接應用公式 2.6，一些期望值為零的基準值 (baselines) [142] b_t 被提出，用以降低梯度估算的變異數，如下所示：

$$\frac{1}{N} \sum_{n=1}^N \sum_{t=0}^T \nabla_{\theta^\pi} \log P^\pi(a_t | x_t, \theta^\pi) (R_t - b_t)$$

其中基準值 b_t 可以是近期獲得獎勵的移動平均值，或是透過將 $R_t - b_t$ 的期望值優化至趨近於 0，由神經網路學習而得。

2.1.6.3 深度確定性策略梯度 (Deep Deterministic Policy Gradients)

DDPG [79] 屬於動作評分 (actor-critic) 方法，是基於價值 (value-based) 與基於策略 (policy-based) 方法的結合。它由兩個網路組成：動作網路 (actor network) $\pi(x_t | \theta^\pi)$ 與評論網路 (critic network) $Q(x_t, a_t | \theta^Q)$ ，分別針對給定的 x_t 確定性地估計動作 a_t 及其 Q 值。

與 REINFORCE 演算法相似，動作網路也是透過策略梯度 (policy gradients) 進行更新。然而，如 [136] 中所證明，它應用了以下方程式作為策略梯度：

$$\begin{aligned} \nabla_{\theta^\pi} J(\theta^\pi) &\sim \mathbb{E} [\nabla_{\theta^\pi} Q(x_t, a_t | \theta^Q) | x_t, a_t = \pi(x_t | \theta^\pi)] \\ &= \mathbb{E} [\nabla_{a_t} Q(x_t, a_t | \theta^Q) \nabla_{\theta^\pi} \pi(x_t, \theta^\pi) | x_t] \end{aligned}$$

這意味著方程式 2.6 中所經歷的總未來獎勵 R_t 是由評論網路估計的 Q 值，且其對 θ^π 的導數遵循連鎖律 (chain rule)，可分解為兩個部分。據此，評論網路的學習過程也與 DQN 相似但略有不同，其中方程式 2.4 中的目標值 y 被修改為：

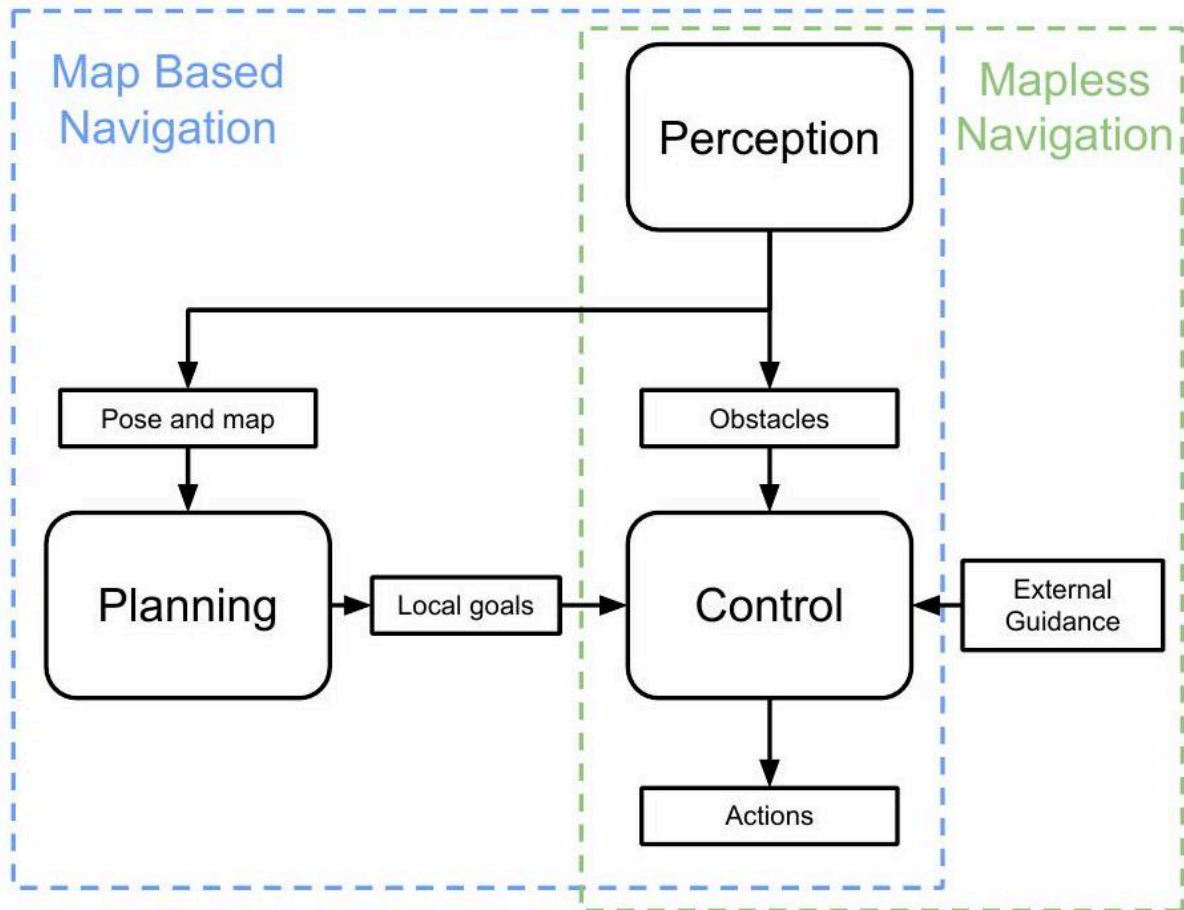


圖 4：圖 2.3：地圖導航系統與無地圖導航系統的組成。局部目標 (local goals) 是全域規劃器 (global planner) 所提供的規劃路徑中，一系列的連續中繼點 (waypoints)。

$$y = r_t + \gamma Q(x_{t+1}, \pi(x_{t+1} | \theta^\pi) | \theta^Q),$$

因為 Critic 網路負責評估 Actor 網路當前的策略，而非在所有可能的動作中搜尋最佳 Q 值。

遞迴 DPG (RDPG) 與 DRQN 類似，是 DDPG 的遞迴版本，其在 Critic 網路與 Actor 網路中皆包含 RNN，並透過隨時間反向傳播 (BPTT) 進行更新。遞迴網路有效地減輕了某些任務中部分可觀測性 (partial observability) 的問題，並使 DPG 能夠處理需要跨多個時間步長保留資訊的任務。

請注意，此處我們僅從理論層面解釋這些 DRL 演算法，其實作細節將在各章節中進一步介紹。

2.2 基於地圖的機器人導航

機器人導航是一個經典的機器人學問題，在眾多產業中都有廣泛的應用。我們並未對這個極其龐大的領域進行詳盡的綜述，而是僅針對室內環境中輪式機器人的導航問題進行探討。

通常導航的目標是透過一條無碰撞的路徑，以最低的時間成本到達目的地。如圖 2.3 所示，它可以分為三個子任務，即感知 (perception)、規劃 (planning) 與控制 (control)。感知任務涉及感測、環境表示以及機器人定位，回答「我在哪裡」的問題；路徑規劃則是搜尋通往目的地的最佳路徑，解決「要去哪裡」的問題；最後，控制系統根據規劃導引機器人，同時防止碰撞，告訴機器人「如何去」。請注意，由於控制階段不需要地圖，我們將在下一節（第 2.3 節）將其作為局部導航 (local navigation) 進行介紹。

2.2.1 感測

理解機器人在環境中的狀態，通常著重於提取機器人與環境中物體之間的幾何關係，這仰賴於機器人的車載感測器。因此，測量機器人與障礙物之間的距離是最直接的感測方法，因為原始感測數據可直接用於避障和建構環境地圖。為此，LiDAR [74, 130, 176]、超音波感測器 [34, 107] 和深度攝影機 [106, 26] 是常見的選擇。

LiDAR 技術利用光脈衝或雷射光束來確定感測器與物體之間的距離。光束傳播到物體並反射回光源，接著利用經過的時間來計算距離。超音波測距則是基於向目標表面發射聲能脈衝，並測量回波返回所需的時間。以 Microsoft Kinect [177] 為例，深度攝影機由紅外線雷射投影儀結合單色 CMOS 感測器組成，可在任何環境光條件下擷取 3D 視訊數據。其缺點之一是強烈的陽光會因為過多的紅外線造成干擾，導致 Kinect 紅外線感測器完全失效。

視覺 [29] 作為測距感測器的替代方案，是另一種在機器人導航中被廣泛利用的感測模態，它能擷取環境中物體的外觀資訊。雖然無法直接從單張攝影機影像中獲得幾何資訊，但在已知攝影機內參和外參的情況下，可以從一對影像中推導出來。此外，考慮到商用攝影機的普及性以及 RGB 影像中所包含的語義資訊，基於視覺的導航在機器人導航研究中具有呈指數級增長的影響力。

2.2.2 定位與地圖構建

在本節中，我們將回顧不構建環境地圖的純定位方法、地圖的不同表示方式，以及同步定位與地圖構建 (SLAM) 方法。

2.2.2.1 定位

在許多基於地圖的機器人導航系統中，環境預先已在度量地圖 (metric map) 中描述，且機器人隨時可以存取。因此，要了解機器人在環境中的狀態，僅需要一個定位系統來提供機器人在地圖座標系中的位姿 (pose)。由於本論文僅關注機器人在室內環境中的導航，本節將回顧室內定位系統 (IPS) 的相關研究。我們根據 IPS 是否需要環境的先驗知識來進行分類。

我們首先回顧那些假設具備環境先驗知識 (prior knowledge) 的系統。對於利用指紋識別 (fingerprinting) 技術的系統，無論採用何種感測器，都必須對覆蓋區域內指定位置的接收訊號強度 (RSS) 進行調查。接著，透過比較測量值與資料庫中指紋的相似度，即可獲得代理器的定位。Wifi 是常見的選擇，如 [132] 所示。然而，另一項研究 [6] 則利用了不同的感測模態，包括聲音、光線、顏色，甚至是使用者的動作。指紋識別是一種完全由數據驅動的方法，這意味著收集到的數據直接決定了其效能。一旦環境發生變化，就必須重新進行調查程序，而這種獲取環境資訊的過程通常成本高昂。此外，還有許多系統應用三邊測量 (lateration) 和三角測量 (angulation) 方法，這些方法需要已知訊號發射器 (如存取點 AP) 的位置。因此，透過應用訊號傳播的物理模型，系統可以利用從多個或單個發射器接收到的訊號來計算使用者的位置 [131, 68]。其中一些系統可以達到很高的精度，但其限制之一是某些材料以及環境中物品的重新排列會影響無線電波的傳播，進而影響測量精度。同樣地，光學追蹤器 (如 Vicon 系統 [92]) 也屬於這一類。它利用多個紅外線攝影機來追蹤安裝在移動物體上的反射標記，解析度可達毫米級。

相反地，對於不需要環境先驗知識的系統，它們僅利用代理器感測器所收集的資訊。其中一個具代表性的系統是視覺定位 [29, 126]，由於攝影機成本低且精度高，這在機器人領域非常受歡迎。其瓶頸包括運算能力有限，且有時對環境變化 (例如光線變化) 的魯棒性不足。另一種在無基礎設施室內定位系統 (infrastructure-free IPS) 中常用的感測器是雷射測距儀 [81, 82]，其在精度方面優於其他感測器。但它也具有高能耗和高金錢成本的缺點。除了上述兩種無基礎設施 IPS 之外，其他系統可能會利用磁場感測器 [158] 或慣性感測器 [167]，這些感測器通常在製造成本與測量精度之間存在嚴重的權衡。

2.2.2.2 地圖表示法

環境可以透過不同形式的地圖來描述。機器人導航中常用的表示方法包括網格地圖 (grid map)、地標 (landmarks) 和點雲 (point clouds)。

網格地圖表示法最早於 1987 年提出 [34]，並已廣泛應用於基於距離感測器的導航系統。世界被建模為環境中連續位置空間上的細粒度網格。每個網格都包含一個確定性數值 (certainty value)，表示該位置被物體佔據的可能性。

地標表示法 [76] 用於環境中觀測到的特徵較為稀疏且可被識別的情況，例如信標（beacons）。在這種情況下，地圖會記錄每個特徵（即地標）的位置和不確定性。

點雲 [165] 是一種適用於 3D 空間的表示方式，包含環境中所有偵測到的特徵點位置。它在採用 3D LiDAR 或攝影機感測器的導航系統中非常受歡迎。

2.2.2.3 同步定位與地圖構建 (Simultaneous Localisation and Mapping)

同步定位與地圖構建 (Simultaneous Localisation and Mapping, SLAM) [33] 是指機器人必須在未知環境中逐步建立一致的地圖，同時在該地圖上進行自身定位。請注意，機器人的軌跡和地圖都是在線 (on-line) 估算的，不需要任何關於位置和環境的先驗知識。這使得 SLAM 在機器人社群中佔有重要地位，因為它提供了讓機器人實現真正自主化的手段。

SLAM 以不同的形式進行建模與求解，其中機率 SLAM (probabilistic SLAM) [150] 通常被認為是接受度最廣泛的建模方式。目前在理論和概念層面上，它已被視為一個已解決的問題，並擁有多種解決方案，例如擴展卡爾曼濾波 (EKF-SLAM) [7]、Rao-Blackwellized 粒子濾波

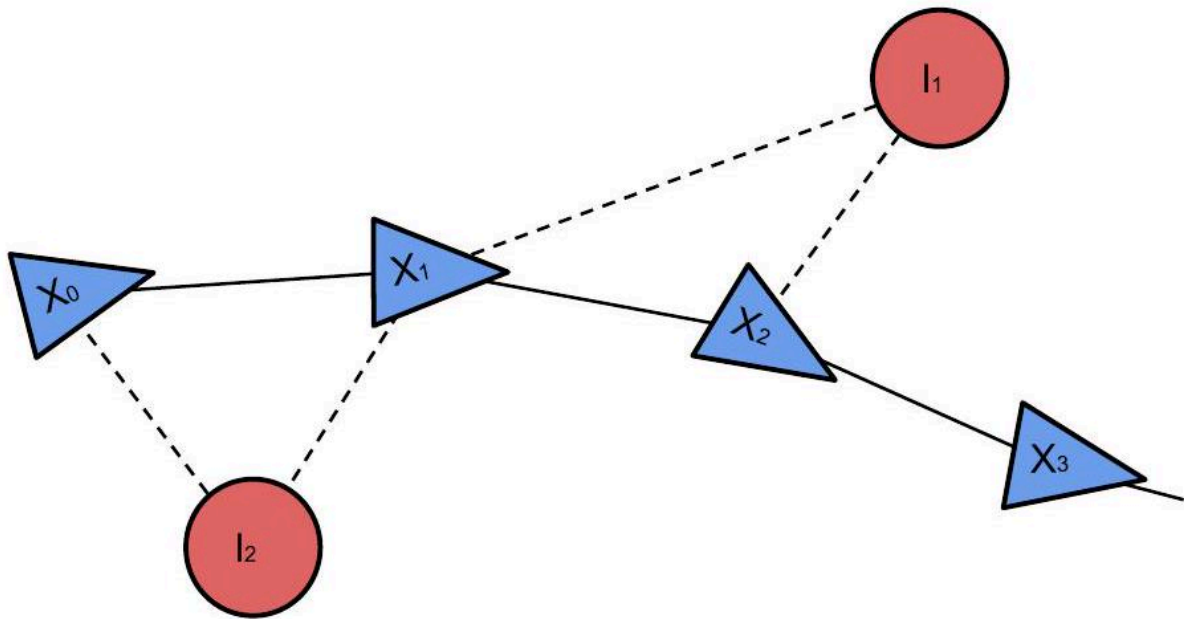


圖 5：圖 2.4：GraphSLAM 的一個簡單實例。藍色三角形代表機器人在不同時間步的位姿 (poses)，紅色圓圈表示構成環境地圖的不同地標 (landmarks)。此外，黑色實線是機器人的軌跡，而虛線則表示機器人對地標的觀測 (observations)。

濾波器 (FastSLAM) [105] 與圖論 (GraphSLAM) [151]。然而，在考慮大規模問題的實際應用時，仍存在許多尚未解決的重大問題。

計算複雜度是首要問題。基於狀態的 SLAM 問題公式化涉及對聯合狀態的估計，該狀態由機器人位姿 (pose) 和觀測到的靜止地標位置組成，其中每個地標代表地圖中的一個特徵點。因此，當機器人行經長距離路徑且地圖中記錄的地標數量不斷增加時，狀態向量會呈線性增長，而其對應的共變異數矩陣 (covariance matrix) 則呈平方增長。另一個問題則是動態環境。現實世界的環境包含許多動態物體，例如人類，以及其他可移動或變動的物體，如椅子和門。這些都違反了大多數 SLAM 演算法中的靜態世界假設，並可能在透過資料關聯 (data association) 更新地圖時導致衝突。例如，如果機器人在長途跋涉後回到地圖中已記錄的地點，它會嘗試透過將當前觀測結果與地圖進行匹配來進行閉環 (close the loop)，以減少漂移。若該地點發生了重大變化，可能會導致關聯失敗，進而產生不一致的地圖。

2.2.3 路徑規劃 (Path-Planning)

當機器人能夠自我定位且目標點也在地圖中被標定時，即可開始尋找通往目的地的路徑。路徑規劃演算法用於在特定條件下，以最低成本生成一條通往目的地的無碰撞路徑。

準則，例如時間、能量或路徑長度。根據是否提供完整的環境描述，可將其歸納為兩大類：離線 (off-line) 與在線 (on-line)。

在離線路徑規劃問題中，由於已給出完整的環境地圖，因此搜尋最佳路徑相對於在線問題較為容易。Lozano-Perz 與 Wesley 在早期以組態空間 (configuration space)、可視圖 (visibility graph) 和單元分解 (cell decomposition) 等概念開創了此領域 [85, 84]。這些方法在簡單環境中可能很有效，但在複雜場景中計算成本很高。因此，一些優化工具如遺傳演算法 (genetic algorithms) 和粒子群優化 (particle swarm optimisation) 被用來產生更有效率的解決方案 [166, 117]。

在線路徑規劃在機器人導航中有更廣泛的應用，因為它可以與 SLAM 演算法緊密配合，讓機器人在穿越環境的同時建立地圖。人工位能場 (Artificial potential fields) 是主要的方法之一 [60]，此外還有碰撞錐 (collision cones) [23]。與離線問題類似，傳統的在線路徑規劃方法後來也透過計算效率更高的優化工具得到了改進 [155, 96, 91]。

關於路徑規劃方法的更全面回顧，可參閱 [118]。

2.3 無地圖機器人導航

在本節中，我們將探討無地圖導航 (mapless navigation) 在室內環境中的應用。如圖 2.3 所示，與基於地圖的導航不同，無地圖導航既不需要預先建立環境描述，也不需要進行線上地圖構建。這意味著機器人僅透過當前或近期的觀測值來感知世界，且歷史資訊 (如果使用的話) 不會被關聯整合為地圖。請注意，機器人仍有可能將當前的感測觀測值形成局部地圖 (local map)。因此，無地圖導航主要解決的是避障問題，這是機器人導航的核心骨幹。同時，如果路徑規劃器 (path planner) 在其局部座標系中提供了一個目標點，系統也會朝向該局部目的地前進。但如果不是以目標為導向，它則僅僅是沿著可通行區域移動。

在本節中，我們回顧了採用兩種不同感測模態的無地圖導航系統，即基於測距感測器 (ranging sensor) 與基於視覺的系統。由於感測方式不同，這些導航系統各自專注於其特定的問題。由於測距感測器在障礙物偵測方面非常直觀，機器人研究人員在早期就對其進行了探索，並提出了許多高效的控制原則。而對於視覺導航系統而言，障礙物偵測始終被視為核心難點。

2.3.1 基於測距感測器的系統

在基於測距感測器的系統中，障礙物偵測相對容易，因為機器人與環境中物體之間的距離可直接從原始感測數據中獲得。

2.3.1.1 基於邊緣偵測的方法

基於邊緣偵測 [14, 71] 的途徑是最早期的研究方法。演算法首先利用超音波感測器偵測障礙物的兩個垂直邊緣，並將連接這些邊緣的直線定義為障礙物的邊界之一。隨後，機器人會繞過該障礙物並抵達目的地。

2.3.1.2 位能場法 (Potential Field Methods)

這類方法是基於虛擬力 (imaginary force) 的概念 [60]。它包含指向目標的引力以及來自障礙物的斥力之總和。在 [69] 中，Krogh 提議在障礙物附近應考慮機器人的速度。隨後，Krogh 與 Thorpe [70] 將其與全域路徑規劃器整合，並提出了「廣義位能場 (generalised Potential Field)」。上述位能場的應用皆依賴於我們擁有環境描述的假設。但在 [4, 19] 中，每個超音波感測器輸入都會產生一個斥力。當這些斥力的總和達到閾值時，機器人會停止並轉向引力所建議的方向。透過這種方式，所提出的位能場能夠應用於無地圖導航問題。

虛擬力場 (Virtual force field) 方法 [15] 將力場與局部網格地圖相結合。然而，它需要在導航過程中建立線上確定性網格地圖 (certainty grid map) 以記錄環境中的障礙物。向量場直方圖 (Vector field histograms) [16] 是基於虛擬力場提出的，旨在提高運算效率、魯棒性，並對錯誤讀數不敏感。其核心思想是透過適合處理不準確感測數據的直方圖網格，使用障礙物的統計表示法，並容納多個感測器讀數的融合。

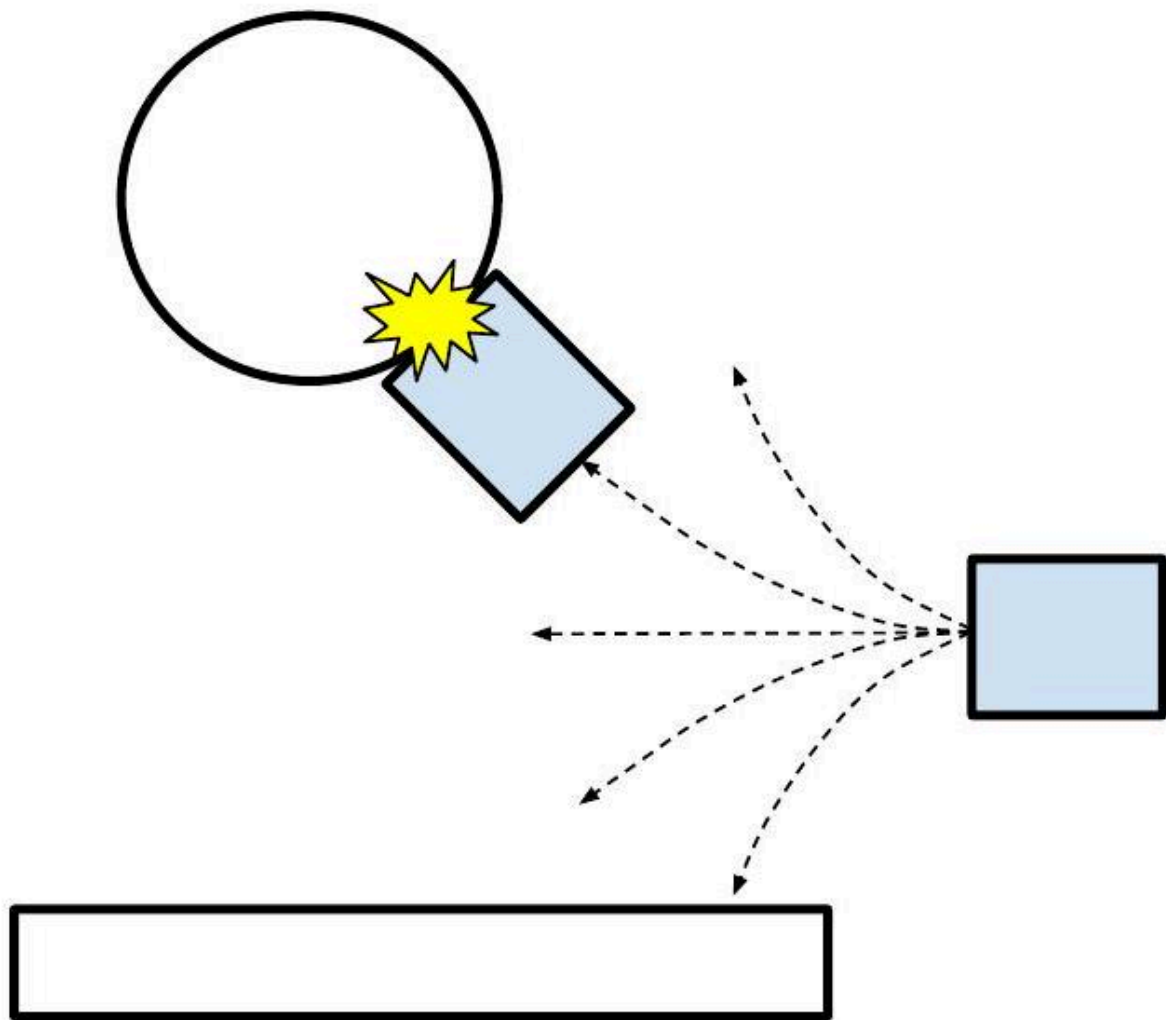


圖 6：圖 2.5：此圖展示了動態窗口法 (dynamic window approach)。白色圓圈和矩形是障礙物，藍色矩形是機器人，虛線是不同控制動作下的採樣軌跡。根據預測，機器人將選擇直行以避開兩側的障礙物。

2.3.1.3 動態視窗法 (Dynamic Window Based Methods)

由 Fox 等人 [36] 提出的動態視窗法 (Dynamic Window Approach) 是避障領域的另一項開創性工作。如圖 2.5 所示，它在機器人的控制空間中進行離散採樣。對於每個採樣速度，它會從機器人的當前狀態進行前向模擬 (forward simulation)，以預測如果施加該採樣速度一段（短）時間後會發生什麼情況。接著，系統會使用包含與障礙物的距離、與目標的距離、與全域路徑的距離以及速度等特徵的指標，對前向模擬產生的每條軌跡進行評估。非法軌跡（即與障礙物碰撞的軌跡）會被捨棄。最後，它會選擇得分最高的軌跡，並計算出對應給機器人的速度。此方法直接源自機器人的動力學，特別設計用於處理受限的速度與加速度。

文獻 [18] 對此進行了改進，結合了規劃與即時避障演算法，為行動機器人產生運動指令，使其在未知環境中安全導航的同時完成任務。在此基礎上，Damas 與 SantosVictor [27] 考慮了障礙物的速度，提出了一種在動態環境中有效的方法。

2.3.2 視覺導向系統 (Vision Based Systems)

由於感測觀測值中沒有明確的幾何資訊，因此使用視覺進行機器人導航比應用測距感測器更加困難。其核心



圖 7：圖 2.6：光流（optical flow）的輸出。綠色箭頭顯示移動像素的方向，而點則代表靜態像素。

問題在於如何從影像中偵測障礙物或可通行區域。在本節中，我們將重點回顧使用傳統電腦視覺技術的現有方法。近期透過深度學習實現的方法將在後續章節中提及。

2.3.2.1 基於光流的方法

光流（Optical flow）最初由 Gibson [40] 提出，並成為電腦視覺中的基本概念。如圖 2.6 所示，它基於兩個假設來追蹤影像序列中像素的相對運動。第一個假設是物體的像素強度在連續幀之間不會改變，第二個假設是相鄰像素具有相似的運動。這兩個假設使我們能夠匹配兩幀中的像素。接著，計算兩幀中配對像素之間相對位移或旋轉的方向與大小，以表示每個特徵點相對於攝影機的相對速度。請注意，光流也可以透過追蹤每一幀中偵測到的特徵點來進行稀疏估計，而無需考慮上述假設。基於「近處障礙物移動快，而遠處障礙物移動相對較慢」的直覺，這使機器人能夠在走廊等簡單環境中進行導航。

Santos-Victor 與 Sandini [125] 提出了一種方法，透過模擬蜜蜂的視覺皮層，利用兩台視野不重疊的發散式立體攝影機（divergent stereo cameras）所產生的光流來進行導航。其核心思想是比較兩側攝影機計算出的平均光流速度差異，並以此建構比例積分微分（PID）控制器 [5] 的誤差訊號以產生動作。

在 [21] 中，光流發散度（optical flow divergence）是從單眼廣角攝影機的連續觀測中提取的。基於此，計算出在各個方向與物體碰撞的時間，並相應地產生控制訊號。

Talukder 等人研究了基於立體攝影機提取的光流（optical flow）來檢測動態物體的方法 [147]。其核心概念是移動物體會導致光流的不連續性，且其量值（magnitude）相對於背景像素會發生變化。這項工作隨後透過結合光流與立體攝影機的深度資訊得到了進一步改良 [148]。

在 [63] 中，透過計算影像序列的平面法向量（plane normal）來分割出地面。首先從影像中獲取稠密光流（dense optical flow），接著用於計算影像子區域的法向量。

2.3.2.2 基於外觀匹配（Appearance Matching）的方法

另一種視覺方法採用兩階段解決方案。首先，在訓練階段透過記錄觀察到的影像或提取的顯著特徵來記憶環境。接著在導航階段，藉由將當前觀察到的資訊與「記憶」進行匹配，機器人可以進行自我定位，並產生正確的動作以穿梭於相同的環境中。我們將這類方法稱為外觀匹配。

在 [89] 中，Matsumoto 等人在訓練階段儲存了路徑上的影像序列以及相鄰兩張影像之間的方向關係。而在測試階段，機器人透過將當前觀測到的影像與記錄的影像序列進行比對，來偵測其位置與轉向角。在 [88] 中，此方法進一步擴展為使用全景攝影機（omnidirectional camera），以獲得更好的匹配準確度與強健性。

在 [179] 中，記錄的影像被編碼為多維直方圖，用以描述外觀特徵，例如顏色、邊緣密度、梯度量值 (gradient magnitude) 和紋理。這大幅降低了記憶體消耗並提升了檢索效率。

Remazeilles 等人 [119] 提出在離線過程中將環境建模為影像圖形庫 (image graph base)。接著在每次導航任務中，檢索相關影像以構成機器人的預定路徑。最後，利用序列影像中匹配點的二維座標，並基於位能場法 (potential field approach) 產生控制訊號。

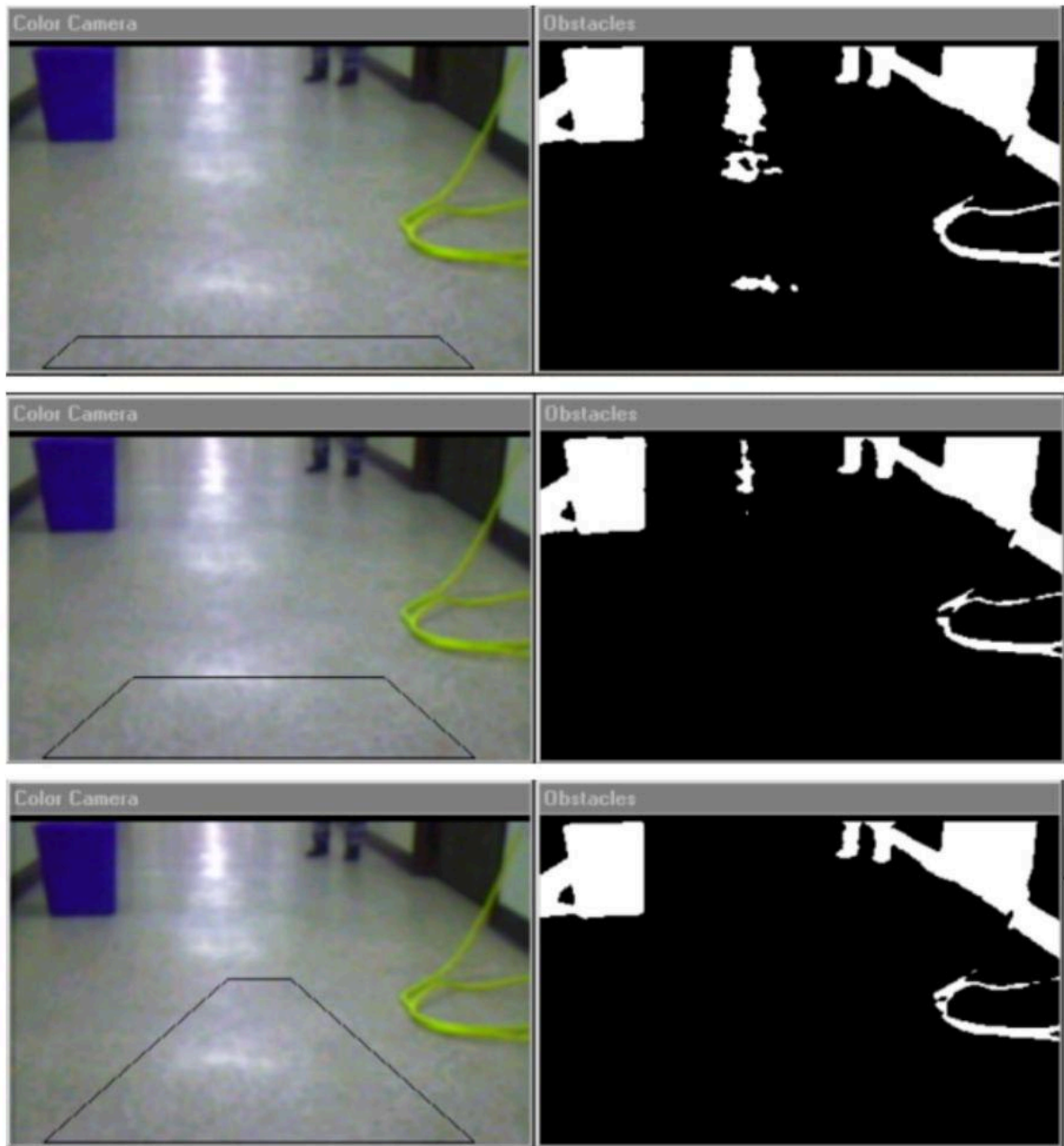


圖 8：圖 2.7：[154] 中的地面分割。左欄影像為輸入，其中黑色多邊形內的區域是建議用於學習地面紋理的參考區域。右欄則展示了分割結果。

2.3.2.3 反應式視覺方法 (Reactive Visual Approaches)

這類方法通常專注於根據影像的某些定性特徵來偵測可移動區域，並能避免計算精確的數值（例如點的距離和座標）。

Shakey [111] 是該領域的先驅系統之一。它在人工環境中進行導航，利用邊緣偵測器將障礙物與無紋理的地板區分開來。[54] 中也出現了類似的方法，使機器人能夠在真實的室內環境中進行導航。

上述方法在具有紋理地板的環境中會遇到問題。Ulrich 與 Nourbakhsh 等人在 [154] 中解決了這個問題。其解決方案是在訓練階段，學習影像中建議參考區域內地面層的顏色特徵，如圖 2.7 所示。因此，機器人在導航過程中能夠將影像中的像素分類為地板或障礙物。

2.3.2.4 基於特徵追蹤的方法

在電腦視覺中追蹤顯著特徵 (salient features) 對視覺導航領域做出了重大貢獻。第一步是使用特徵檢測器 (feature detectors) 在成對的影像中尋找特徵點，例如邊緣 (edge) [22, 47]、角點 (corner) [47, 120] 或斑點 (blob) [9, 95]。接著，將兩張影像中的特徵進行對應匹配。當擁有足夠的匹配特徵時，即可估算單應性矩陣 (homography matrix)，並最終計算出兩張影像之間的相對旋轉與位移。

[113] 提出了基於單應性 (H-based) 的追蹤器，用於追蹤室內環境中的角點。該追蹤器採用平面單應性 (planar homographies)，並以地面上的 5 個角點進行初始化。此研究在 [78] 中得到擴展，透過識別地面平面上方的特徵點來偵測障礙物。[83] 也提到了一種相似的方法，在假設地面為局部平面的前提下，透過搜尋無法註冊 (registered) 於地面平面上的特徵點來偵測障礙物。此外，在 [180, 28] 中，單應性矩陣被計算並用於尋找地面平面，其基礎是先前使用 Harris corner detector [47] 所追蹤的影像角點。

2.4 討論

最近的研究已開始嘗試將基於 DRL 的導航系統部署在實體機器人上，並提出了一些尖端解決方案來解決 1.2 節中提到的核心挑戰。然而，這些方案僅在有限的情況下解決了上述問題，一個能夠泛化至不同場景並部署於各種實體機器人的實用導航系統尚未被具體建立。

2.4.1 從模擬到現實世界

領域隨機化 (Domain randomisation) 是解決此問題的一種可能方案 [152, 122]。該技術的核心思想是強制策略在環境視覺外觀隨機化的情況下進行學習。雖然此方法簡單，但最終的效能很大程度上取決於隨機變數分佈的正確選擇 [108]。這裡的困境在於，高度變化的隨機化會增加任務的複雜度並延長訓練時間，而過窄的分佈則可能無法涵蓋目標現實領域。

另一個分支則透過調整合成領域與現實領域之間的視覺觀測值來彌合「現實差距 (reality gap)」。Bousmalis [17] 與 Stein [139] 皆建議透過渲染使用生成模型 (虛擬轉真實) 產生更逼真的視覺觀測結果。與領域隨機化 (domain randomisation) 相比，這些方法更為直觀，但由於在訓練過程中必須在優化 DRL 策略的同時生成逼真圖像，因此會產生額外的運算負擔。如 [175, 57] 所述，此問題可透過在真實世界的推論階段，於適應操作 (adaptation operation) 之前加入反向適應方向 (真實轉虛擬) 來解決。然而，另一個大幅降低此類方法實用性的問題在於，訓練好的適應網路是針對特定的虛擬/真實環境對所量身打造的，缺乏對具有明顯視覺變化環境的泛化能力。這在視覺外觀極具多樣性的室內環境中變得更加明顯。

在第 3 章中，我們率先提出了一種更具實用性且以任務為導向的解決方案。它透過將視覺觀測投影到另一種模態 (即深度資訊)，來最小化合成數據與真實數據之間的差異。這種方法在訓練和測試階段都易於實作。大多數機器人模擬器 (例如 ROS Stage¹、ROS Gazebo² 和 Carla [31]) 都提供了地面真值 (ground truth) 深度資訊。同時，在真實世界進行測試時，可以利用現成的深度預測網路 [75, 1] 從視覺觀測中估計深度，而這些網路通常能泛化到各種室內場景。使用深度資訊作為橋樑的另一個原因是，環境中機器人與物體之間直接提供的距離資訊，有助於更輕鬆地學習避障功能。

2.4.2 加速模擬環境中的訓練

處理 DRL 樣本使用效率低下的問題，一直是機器學習社群中一個長期存在的課題。為了保持對機器學習領域各種任務的泛化能力，許多理論性 [162, 127] 和系統性 [101] 的方法被引入。這些方法透過優化學習機制本身來提高訓練效率，但並未考慮任何額外資訊。然而，與廣義的機器學習領域不同，機器人任務 (如無地圖導航) 有許多傳統方法可以作為學習策略的先驗知識 (prior knowledge)。

從演示中學習 (Learning from demonstrations, 或稱模仿學習) [32] 結合了強化學習 (RL) 中的互動式學習模式以獲得魯棒性，並透過強制網路模仿專家行為，利用監督式學習中高效的學習訊號。

然而，這種方法受限於專家控制器 (expert controllers)，而這些控制器本身在機器人領域通常難以建立。

另一種方法如 [170] 所示，是將傳統控制器嵌入到導航系統中。它僅透過強化學習來學習規劃策略，以便在不同情況下為底層控制器分配不同的參考訊號。雖然這種方法在早期階段能有效加速學習過程，但其最終效能會受限於底層的啟發式控制器 (heuristic controller)。

因此，在第 4 章中，我們介紹了一種新穎的框架，其中所有類別的啟發式控制器 (無論是專家級還是簡單級) 僅被視為引導，用以降低離策 (off-policy) DRL 方法 (例如 DDPG) 學習過程中的變異量。在此框架下，底層的 DRL 方法可以在早期階段快速向引導控制器學習，並在其策略佔優勢時自動捨棄它們，隨後繼續透過自我探索進行學習。所使用的啟發式控制器可以是初階的或專家級的，因為這種「軟性」引導的效能僅會影響學習過程的加速率，而不會限制底層 DRL 方法的學習。

2.4.3 學習全域無地圖導航

將學習到的導航策略泛化 (Generalising) 至未曾見過的环境，在 DRL 中仍是一個尚未解決的問題，因此將其應用於全域無地圖導航也具有極大的難度。許多開創性的研究 [99, 181, 98] 並未考慮這個問題，因為其最短路徑策略是透過深度神經網路記憶所有的環境資訊來實現的。其他一些研究則必須提供度量地圖 (Metric map) 或拓撲地圖 (Graphic map)，才能將策略轉移到新環境中 [46, 20, 157]。

為了學習可轉移的無地圖導航策略，近期在 [143, 72, 53] 中引入了類似於傳統視覺教導與重複 (Visual Teach and Repeat, VT&R) [37] 的解決方案。這些方法僅需要影像序列 (無論是否與動作配對) 作為引導，與地圖相比，這類資訊更容易獲取。然而，一個限制是相鄰引導片段之間的移動必須是增量式的，否則導航系統很容易在引導資訊中迷失。這導致了對高密度引導資訊的需求，進而影響通訊與記憶體開銷，特別是在多代理人 (Multi-agent) 的情境下。此外，由於系統高度依賴密集引導，其自主性以及對環境變動或動態環境的魯棒性 (Robustness) 也受到限制。有趣的是，如 [53] 中提到的，使用全景攝影機可以增強對環境微小變化的魯棒性，因為更寬廣的視野能帶來更豐富的視覺資訊，並使機器人能夠執行返航 (Homing) 任務 [3]。

一種更為稀疏且高效的引導方式，即自然語言，也曾被嘗試用於學習視覺無地圖導航 [159, 51]，但僅限於模擬環境。由於自然語言極度稀疏、具備歧義性，且機器人難以感知多模態觀測 (視覺與語言)，使得自然語言難以成為即時機器人控制問題中準確且可靠的引導方式。

為了建立一個視覺無地圖導航系統，使其導航經驗能輕鬆且高效地泛化至未知場景、部署於實體機器人並在多個代理器 (agent) 間共享，我們在第 5 章提出了一種新型導航系統。其導航策略僅需對環境進行極簡描述 (即幾張環境快照搭配方向指令)，即可轉移至新場景。這種引導資訊可以輕易地從一個代理器的經驗中提取並分享給其他代理器。由於採用了雙層階層式架構，每一層都利用來自不同領域的數據學習強健的子策略，因此該系統也能直接部署於真實機器人。

第 3 章

視覺避障：從模擬到現實世界

本章旨在解決將模擬環境橋接至真實環境時，縮小現實差距 (reality gap) 的挑戰，並專注於單眼視覺避障 (monocular visual obstacle avoidance) 這一特定任務。避障是機器人導航中的一項基本能力，對於 DRL 策略而言相對容易學習。這裡處理的核心問題在於，當僅提供來自模擬器的單眼視覺觀測值作為訓練數據時，機器人是否能學會繞過障礙物導航。我們的目標是檢驗所提出的方法是否能在不使用真實世界數據訓練的情況下，將學習到的策略感知準確地從模擬環境遷移到真實世界。廣泛的實驗驗證了遷移後避障策略的有效性，及其對各種真實世界場景的魯棒性。

3.1 引言

當移動機器人在真實世界中運作並面臨不斷變化的條件時，其所需具備的基本能力之一就是避障。雖然這可以透過全域規劃器 (global planner) 根據地圖提供無碰撞路徑來部分解決，但由於環境中存在動態物體以及場景變化，基於機器人載體感測器 (on-board sensors) 來實現此功能更為重要。

作為機器人領域中一個長期存在的問題，避障通常透過基於測距感測器 [35] (例如雷射掃描儀和聲納感測器) 的方法來處理。這些感測器可以直接測量機器人與障礙物之間的相對位置，從而大幅簡化問題。然而，測距感測器僅能捕捉有限的資訊，且其中一些感測器價格昂貴，或者對於特定平台 (如無人機, UAV) 而言過重且耗電。相比之下，單眼相機則能提供關於機器人作業環境的豐富資訊，且具有低成本、輕量化以及適用於多種平台等優點。遺憾的是，當透過單眼視覺 (即 RGB 影像) 獲取距離感知時，避障問題會變得異常困難。這是因為三維世界被扁平化為二維影像平面，消除了像素與距離之間的直接對應關係。

深度強化學習 (Deep Reinforcement Learning, DRL) 近期在基於高維度視覺觀測的複雜任務中，已達到或超越了人類水準的表現，例如玩 Atari 遊戲 [104]。它能將原始的外觀輸入直接映射到動作輸出，使系統無需手動設計影像特徵提取，而手動特徵在許多情況下 (例如光照條件變化或面對無紋理表面時) 可能不穩定。此外，DRL 透過與環境的互動進行學習，並根據所獲得的獎勵優化其策略，使其最終有可能超越啟發式方法 [138, 154, 13]。儘管具備上述優勢，基於 DRL 方法的一個重大限制是對於互動式學習樣本的需求，這使得在現實世界中進行訓練幾乎不可能，對於行動機器人而言尤其如此。這不僅耗費時間與人力，還可能引發安全問題。因此，DRL 目前僅在訓練成本可負擔的虛擬環境中具有卓越表現。將學習到的策略從模擬環境遷移到現實世界，仍然是一個開放性問題，也是將 DRL 應用於實際機器人問題的主要限制。

在本章中，我們提出了一個基於深度神經網路的避障系統，利用機載單眼相機進行環境感知。訓練完全在模擬環境中進行，並最終能泛化到各種現實世界的場景。

3.2 相關工作

解決避障問題的標準框架包含兩個步驟：首先利用視覺資訊推論可通行空間與障礙物，接著應用傳統的路徑規劃策略。恢復視覺幾何是偵測障礙物的常用方法，例如透過光流法 [138, 90]、消失點偵測 [13]，甚至是視覺 SLAM [109]。根據視覺外觀對地板等可通行區域進行分割 [154] 也是一種流行的方法。一旦理解了周圍環境，便可以採用各種傳統的路徑規劃器來驅動機器人沿著可通行路線行駛 [36]。雖然上述方法能夠將規劃與視覺資訊解耦，並從中受益於傳統的路徑規劃器，但它們通常需要大量手動調整的參數，才能規劃出可行的路徑。此外，讓這些方法自動適應新的作業區域也具有挑戰性。

基於監督式深度學習的路徑規劃（學習如何避免碰撞）正變得越來越受歡迎。特別是隨著深度學習近期的進展，已有數種端到端（end-to-end）監督式學習方法被提出，用於直接從原始影像預測控制策略 [61, 42, 144]，而無需遵循先前的兩步驟框架。因此，它們可以避免傳統路徑規劃器複雜的建模與參數調整。例如，卷積神經網路（CNNs）被訓練用於讓飛行機器人在複雜的森林環境中導航 [42]。然而，由於其監督性質，這些方法需要手動標記，而獲取這些標記既耗時又耗費人力。

自監督學習（Self-supervised learning）可用於在額外回饋的幫助下，自動為路徑規劃器生成標籤。例如，在 [172] 中，設計了一種演算法，利用 3D 點雲為基於 CNN 的模型標記軌跡類別。[38] 則提議透過預測無人機何時發生碰撞來訓練其控制器。雖然自監督學習是在無需人工監督的情況下利用大數據集的可行方法，但所學到的策略本質上仍受限於標籤生成策略。

增強學習（Reinforcement learning）透過嘗試錯誤來探索策略，並已在 [94] 中應用於基於視覺的障礙物規避。然而，該研究將原始圖像編碼為多層深度資訊，用以預測合適的控制策略。深度增強學習（Deep reinforcement learning, DRL）近期已被證明能透過充分探索原始圖像，在遊戲中達到超越人類的表現 [104]。由於 DRL 使用的學習訊號通常比監督式學習弱得多，因此需要更龐大的訓練資料集。這使得在現實中直接將 DRL 應用於機器人領域變得十分困難。因此，通常會使用具備失敗與恢復機制的模擬環境進行訓練，而非在現實世界中進行探索 [100]。訓練完成的網路隨後可遷移至實體機器人。雖然這種方法在使用雷射掃描儀 [146] 和深度圖像 [145] 時已取得成功，但對於基於視覺的模型來說難度顯著更高 [38]。近期，[123] 的作者提出完全在 3D CAD 模型模擬器中訓練一個網路作為碰撞預測器，並對渲染設定進行高度隨機化，將現實世界近似視為訓練資料的一個子集。雖然他們的模型可以擴展到現實世界，但產生龐大資料集並進行訓練需要耗費大量的運算資源，且缺乏系統性的方法來調整隨機化參數。

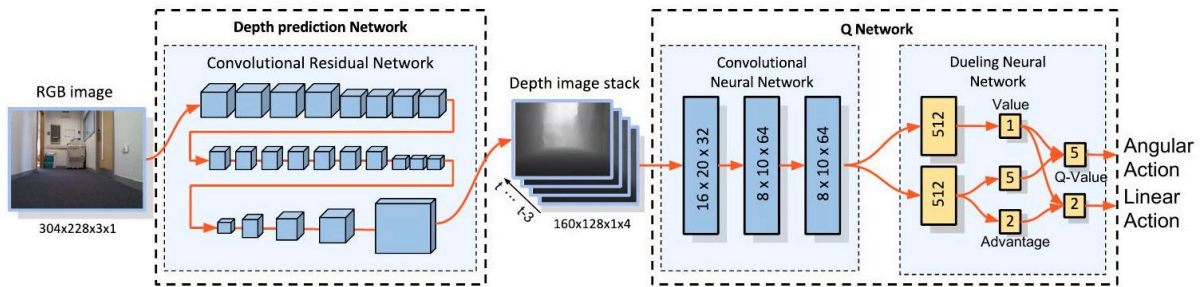


圖 9：圖 3.1：基於單眼圖像並透過深度增強學習實現障礙物規避的網路架構。首先構建一個全卷積神經網路（fully convolutional neural network），用以從原始 RGB 圖像預測深度。隨後接續一個深度 Q 網路（deep Q network），該網路由卷積網路和競爭網路（dueling network）組成，用以並行預測角速度動作與線速度動作的 Q 值。

3.3 問題定義

我們將基於單眼視覺的障礙物規避問題定義為一個決策過程（MDP），這在第 2 章中已有簡要介紹。在此，我們將時間 $t \in [0, T]$ 的 RGB 相機圖像 x_t 建模為代理人（agent）的觀測值，並將期望的控制指令 $a_t = (a_t^v, a_t^\omega) \in \mathcal{A}$ 建模為其動作，其中 a_t^v 和 a_t^ω 分別為線速度與角速度。接著，每當代理人根據觀測值 x_t 選擇一個動作 a_t 時，它會獲得一個獎勵 r_t 。同樣地，MDP 的目標是最大化累積未來獎勵 $R_t = \sum_{\tau=t}^T \gamma^{\tau-t} r_\tau$ ，其中 γ 為折扣因子。

3.4 模型與訓練細節

此系統由一個深層神經網路建模而成，如圖 3.1 所示。它包含兩個模組：深度預測網路（depth prediction network）以及 Q 值估計網路（Q-value estimation network）。

深度估計：深度預測網路是採用 [75] 中的全卷積殘差網路（FCRN）實作，用於從單張 RGB 影像預測深度資訊。它首先從 RGB 影像中提取深度資訊並將其轉換為特徵向量，接著據此重建深度影像。

FCRN 使用 <https://github.com/iro-cp/FCRN-DepthPrediction> 提供的預訓練模型進行初始化，該模型已基於 NYU 資料集 [110] 進行優化。隨後，使用透過 Microsoft Kinect¹ 相機收集的 1 萬張影像進行重新訓練。學習率設定為 10^{-5} 。

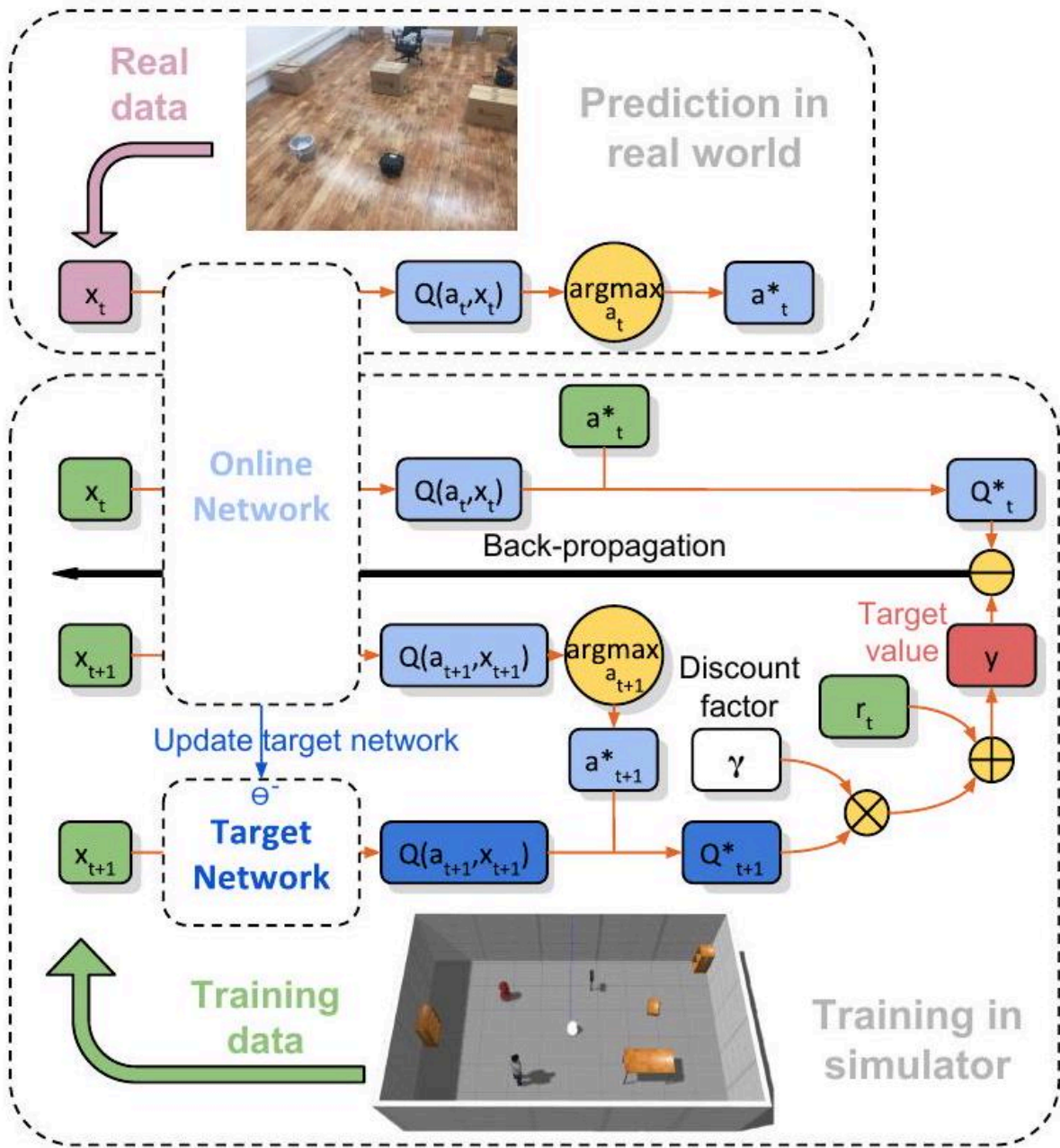


圖 10：圖 3.2：給定一批訓練數據，包括當前狀態 x_t 、動作 a 、獎勵 r 和結果狀態 x_{t+1} ，圖中顯示了 D3QN 的訓練程序。 $\oplus$, $\ominus$ 和 $\otimes$ 是加法、減法和乘法的逐元素運算（element-wise operations）。

D3QN 的 Q 網路是結合了第 2.1.6.1 章介紹的雙網路（double network）與競爭架構（dueling architecture）的 DQN。如圖 3.1 所示，它由三個全卷積層和全連接層組成。前者用於從深度圖像中提取有助於避障的特徵。值得注意的是，輸入為連續四張深度圖像，以緩解部分可觀測性（partial observability）的問題。根據競爭網路架構，所有密集層（dense layers）被分為兩個分支，每個分支分別估計狀態價值（value）和優勢（advantage）。透過這種結構，由於預測 Q 值的分解，學習效率和最終性能得到了大幅提升。狀態價值代表所有可能 Q 值的最大值。因此，透過從 Q 值中減去狀態價值計算出的優勢值會縮放在 0 附近。這對於深度神經網路透過反向傳播進行學習更為有效，同時保持所有 Q 值之間的相對關係不變。因此，這促進了策略的學習，因為網路只需要學習準確的優勢值即可知道該選擇哪個動作。

此網路根據圖 3.2 所示的雙網路機制進行優化。目標網路（target network）與線上網路（online network）具有完全相同的架構。然而，與線上網路在每個訓練步驟都透過反向傳播更新權重不同，目標網路的權重在短時間內保持固定，然後從線上網路複製。基於這種雙網路框架，[156] 聲稱應使用線上網路來選擇動作，而目標網路則應被用於解決價值估計過於樂觀的問題 [48]。更具體地說，結果狀態 x_{t+1} 被線上網路和目標網路同時採用，以計算時間 $t+1$ 的最佳價值 Q^* 。接著，利用折扣因子 γ 和當前獎勵 r_t ，得到 t 時的目標值 y 。最後，透過從線上網路在給定當前狀態 x 下預測的最佳價值 Q^* 中減去目標值來計算誤差，然後進行反向傳播以更新權重。

獎勵塑形（Reward Shaping）獎勵函數的設計在學習有效策略中扮演著至關重要的角色。更具指導性的獎勵可以大幅促進訓練，但可能會引入人為偏見，並導致系統偏離最佳策略。因此，我們提出了以下獎勵函數：

$$r_t = \begin{cases} R_{\text{crash}}, & \text{if robot crashes} \\ v * \cos(\omega_t) \delta t, & \text{otherwise} \end{cases}$$

其中 v 和 ω 分別是局部線速度與角速度，而 δt 是每個訓練迴圈的時間，設定為 0.2 秒。如果偵測到碰撞，該回合 (episode) 會立即終止，並額外處以 $R_{\text{crash}} = -10$ 的懲罰。否則，該回合將持續到達到最大步數（在我們的實驗中為 500 步）後終止，且不給予懲罰。

此獎勵函數與廣泛使用的稀疏獎勵函數 (sparse reward function) 不同，後者代理人 (agent) 僅在回合結束時才會收到回饋。它對機器人的行為提供了詳細的偏好引導，鼓勵機器人盡可能快地運行，並對原地旋轉進行懲罰，這有效地減少了代理人收斂所需的學習樣本量，並提升了學習效率。雖然獎勵函數中的這種人為偏好可能會使機器人偏離從稀疏獎勵函數中學到的最佳策略，但透過我們的實驗證明，它在避障任務中能產生可接受的策略表現。

D3QN 模型在兩個模擬環境（簡單與複雜環境）中進行訓練，兩者皆建構於 Gazebo 模擬器中，如圖 3.3 所示。D3QN 模型首先在簡單環境中進行訓練，然後再於複雜場景中進一步訓練。在模擬器中訓練完成的模型會直接在數個不同的真實世界場景中進行測試。線速度設定為 0.4 或 0.2 m/s，而

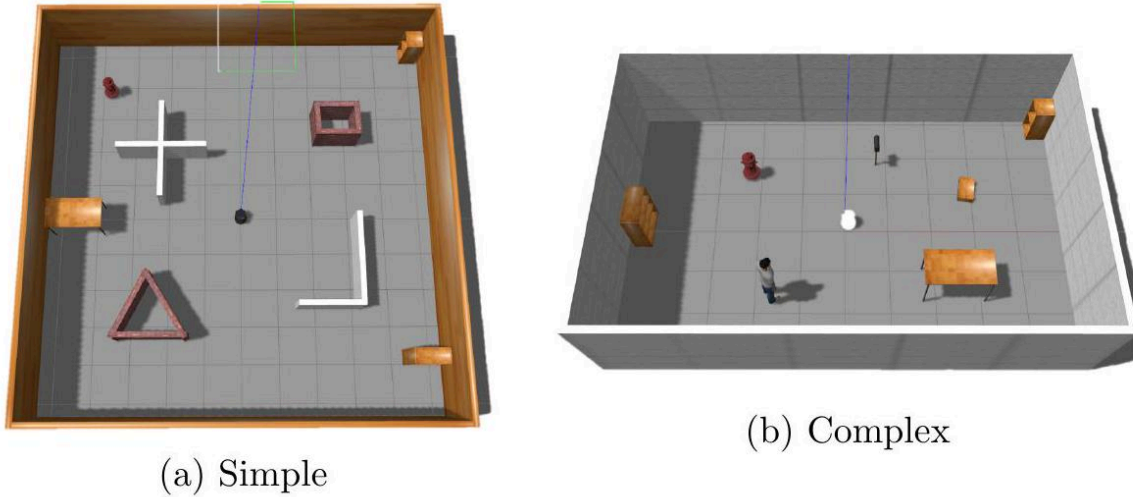


圖 11: 圖 3.3: Gazebo 中用於訓練的兩個模擬世界。

角速度為 $\frac{\pi}{6}, \frac{\pi}{12}, 0, -\frac{\pi}{12}$ 或 $-\frac{\pi}{6} \text{ rad/s}$ ，產生十種不同的行為。在整個實驗過程中，使用 NVIDIA TitanX GPU 進行訓練，並使用配備 NVIDIA GTX 860 GPU 的筆記型電腦進行即時推論與實境測試。Adam 優化器 [64] 的學習率設定為 10^{-5} 。

3.5 實驗

我們進行了廣泛的模擬與真實世界實驗，以評估所提出的避障系統。在每個訓練回合 (episode) 中，機器人會從模擬環境的中心生成，並朝向隨機方向。在我們的實驗中，我們將控制頻率設定為 5 Hz，最大訓練步數設定為 10k，在不使用模擬加速的情況下大約耗時 5.5 小時。我們使用一台搭載 Kinect 感測器的 Turtlebot 2² 機器人來測試即時控制策略。然而，在真實世界測試中僅使用了 RGB 攝影機。

3.5.1 不同模型的訓練效率

為了分析所提出的 D3QN 模型之訓練效率與性能，以及引入 Dueling 與 Double 技術在避障任務中的優勢，本研究對比了深度雙重 Q 網路 (DDQN) 與 DQN 兩個競爭模型。如圖 3.4 所示，D3QN 模型在訓練速度和獎勵表現上均優於其他兩者。與平均獎勵僅達到 10 的 DQN 不同，具有雙重網路結構的網路能學習到具有更高獎勵的策略。這可能是因為 Q 值過度估計的問題阻礙了 DQN 學習

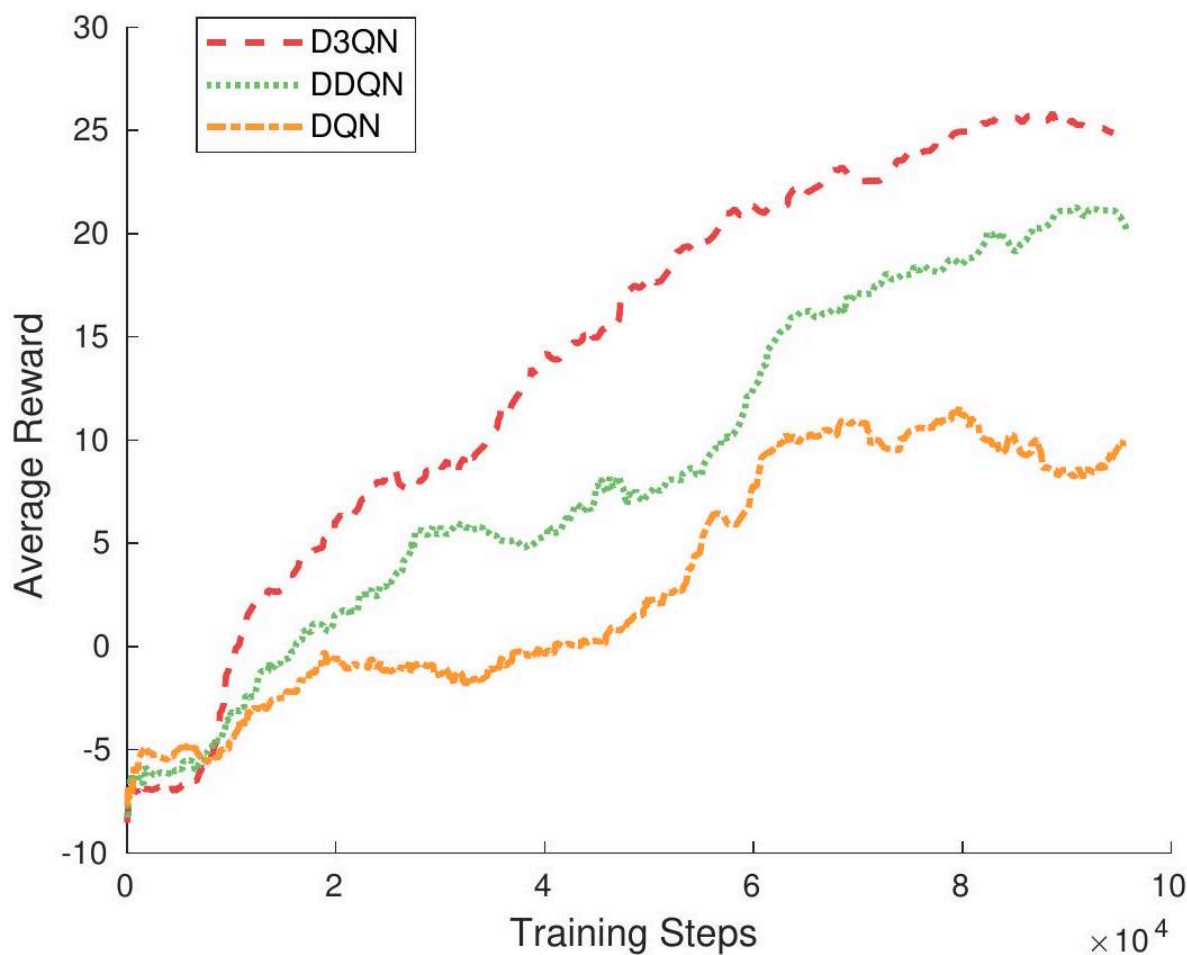


圖 12：圖 3.4：三種模型的平滑學習曲線，顯示機器人獲得的平均獎勵。

避障的穩健策略。由於 Dueling 網路能學習各個動作在不同情境下的重要性，D3QN 模型相較於 DDQN 具有進一步的優勢。它不僅在避障方面展現了卓越的性能，也為將 D3QN 應用於其他機器人領域指引了一個有趣的方向。

3.5.2 真實世界測試

我們進行了數項實驗，以直接在真實世界中測試訓練好的模型。

3.5.2.1 靜態影像的動作預測

首先，我們檢驗網路是否能針對任意且複雜的場景，預測出合理的避障動作。如圖 3.5 所示，我們使用手持相機在圖書館、博物館、閣樓和臥室等多種環境中拍攝的 RGB 影像來進行動作預測。圖中的長條圖表示每個動作的 Q value：前兩個長條代表線速度 (linear velocity) 0.2 m/s 與 0.4 m/s ，其餘則代表五種角速度 (angular velocity) 設定： $\frac{\pi}{6} \text{ rad/s}$ 、 $\frac{\pi}{12} \text{ rad/s}$ 、 0 rad/s 、 $-\frac{\pi}{12} \text{ rad/s}$ 與 $-\frac{\pi}{6} \text{ rad/s}$ 。請注意，這些場景比訓練時使用的模擬環境更為複雜，且模型從未「見過」其中任何一個場景。

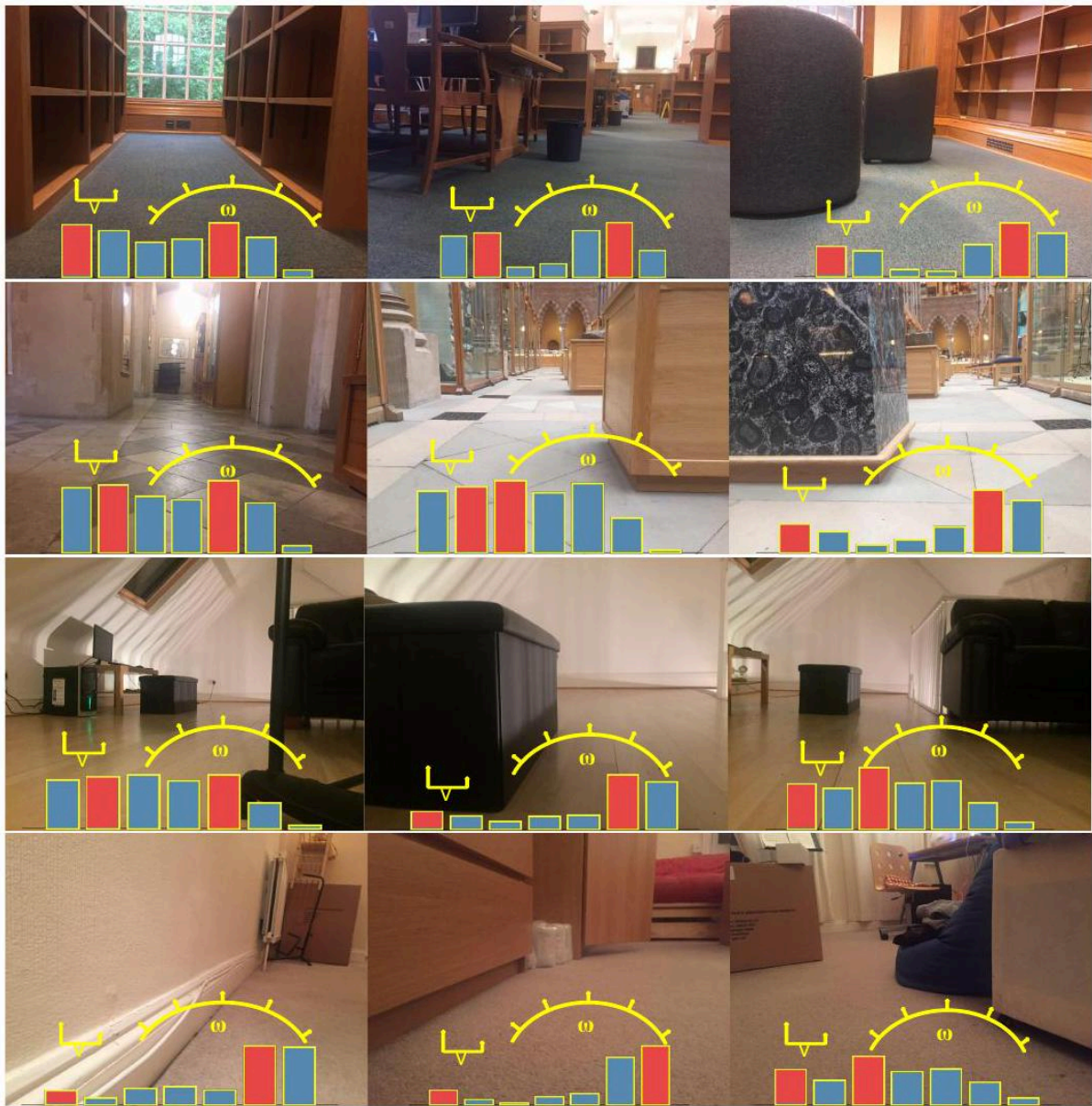
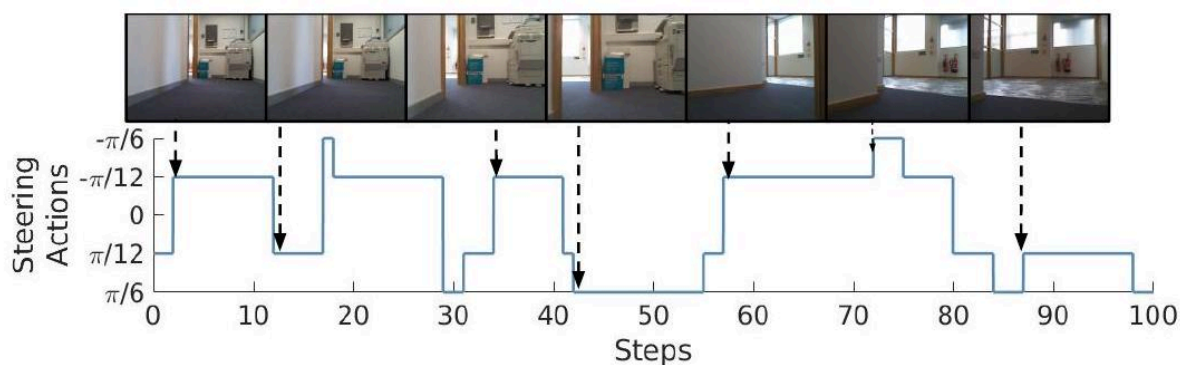
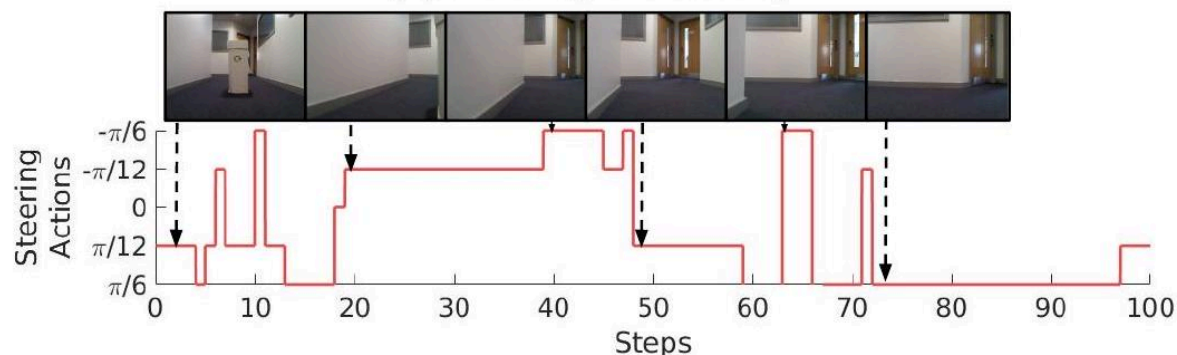


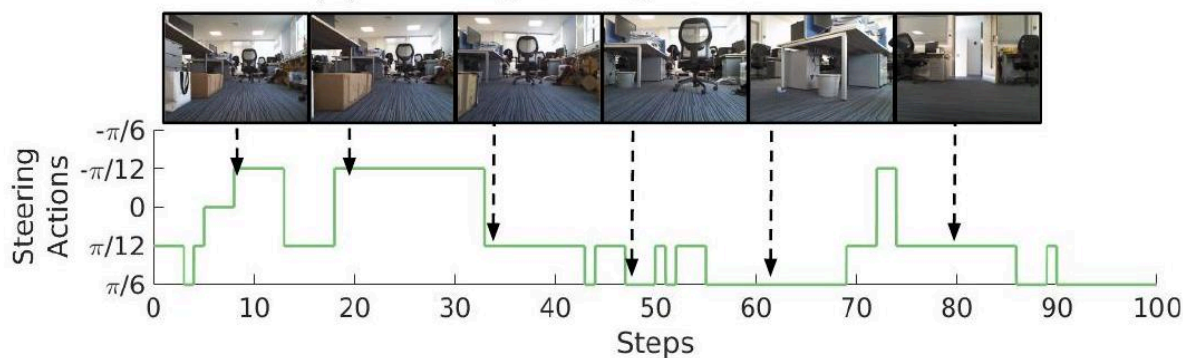
圖 13：圖 3.5：在不同室內環境（例如圖書館、博物館、閣樓和臥室，由上至下）進行的實驗。下方的長條圖展示了網路為每個線速度和角速度動作預測的 Q 值，其中紅色長條表示網路以貪婪策略（greedily）選擇的動作。請注意，前兩個長條代表線速度動作，其餘則代表轉向動作。



(a) Passing a doorway.



(b) Driving through a corridor.



(c) In an office room.

圖 14: 圖 3.6: 在三種不同場景下的真實世界測試。影像串流下方的曲線顯示了機器人在每一步所選擇的轉向動作。

之前。儘管如此，訓練好的 D3QN 模型仍能夠根據估計的 Q 值產生合理的動作來驅動機器人。

3.5.2.2 三種不同場景下的測試

經訓練的 D3QN 模型在三種不同場景（包括門口、走廊和辦公室）中進行了短期（20 秒）導航測試。圖 3.6 展示了這三次測試的轉向動作及部分樣本影像。具體而言，圖 3.6a 顯示了機器人通過門口的過程。儘管機器人在接近未見過的障礙物（印表機）時轉向動作略顯不穩定，但仍能成功通過門口。在走廊案例中，走廊中間放置了一個障礙物。如圖 3.6b 所示，機器人能夠平穩地穿過障礙物與牆壁之間的狹窄空間。同樣地，機器人也能在辦公室內安全地受控移動，辦公室是一個更為複雜的環境，包含許多模擬器中未曾出現過的物體。實驗證實了

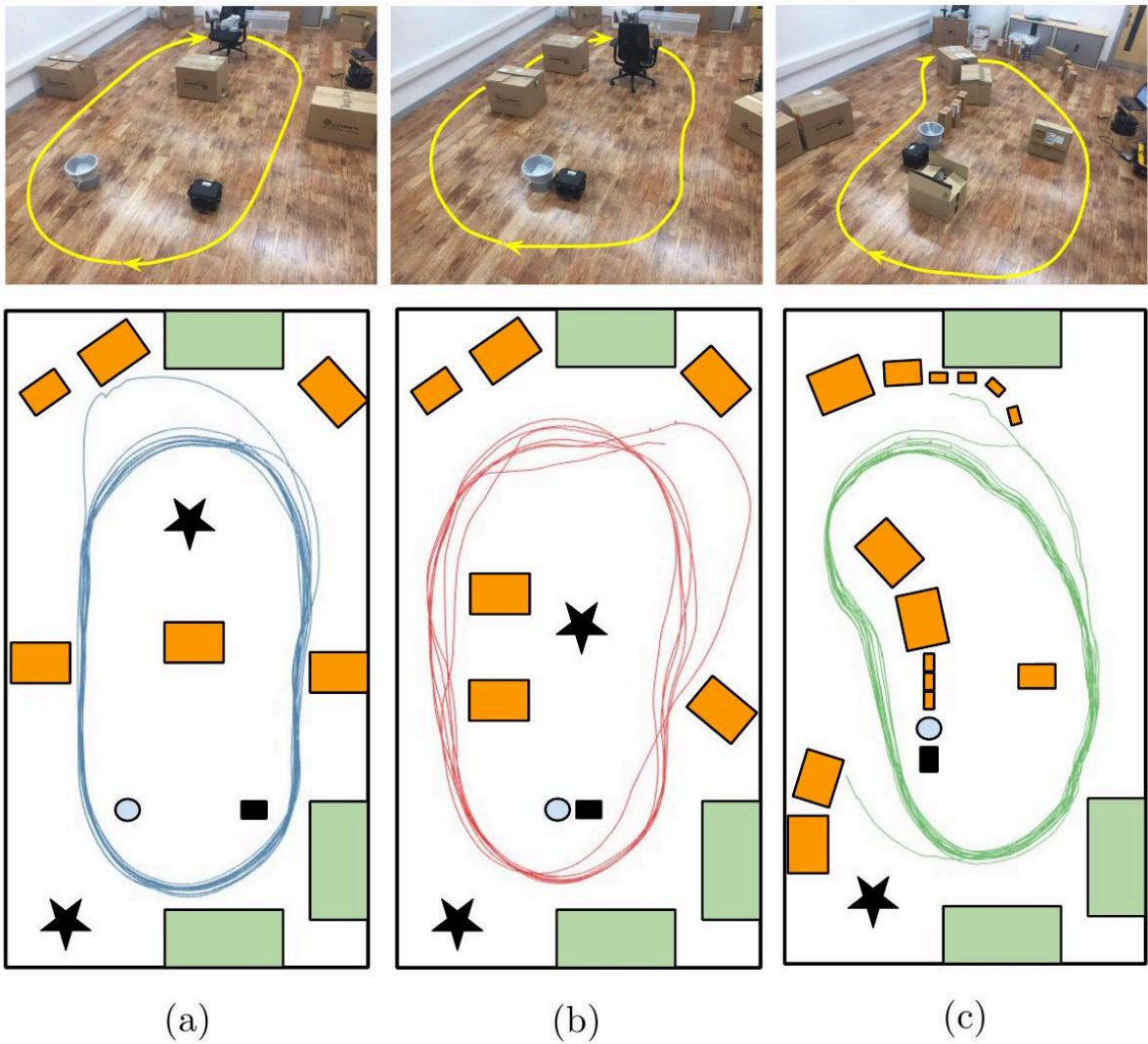


圖 15：圖 3.7：在具有不同障礙物數量與擺放位置的房間內進行實體環境測試。矩形代表箱子，星號和圓圈分別代表椅子和垃圾桶。

經訓練的 D3QN 模型使機器人僅使用單眼相機即可在不同的真實環境中避開障礙物，這完全得益於在虛擬環境中所學習到的知識。

3.5.2.3 凌亂環境中的測試

在一個雜亂的房間內進行了數次長期實驗，以進一步測試在動態佈局和物體環境下的效能。實驗中使用 Vicon 系統來記錄機器人的地面真值 (ground truth) 位姿。

圖 3.7 記錄了機器人在眾多障礙物周圍運行時的軌跡。綠色矩形代表固定家具，而橘色矩形則是可移動的箱子。其他障礙物包括兩張椅子（星號）、一個垃圾桶（圓形）和一個小行李箱（黑色矩形）。從結果中我們可以看到，機器人通常會選擇沿著相似的路徑行駛。這是因為在網路預測了每個狀態與動作組合的 Q 值後，動作是透過貪婪策略 (greedy policy) 選取的，從而導致所有狀態都有固定的策略。由於在訓練階段定義的獎勵函數 (reward function)

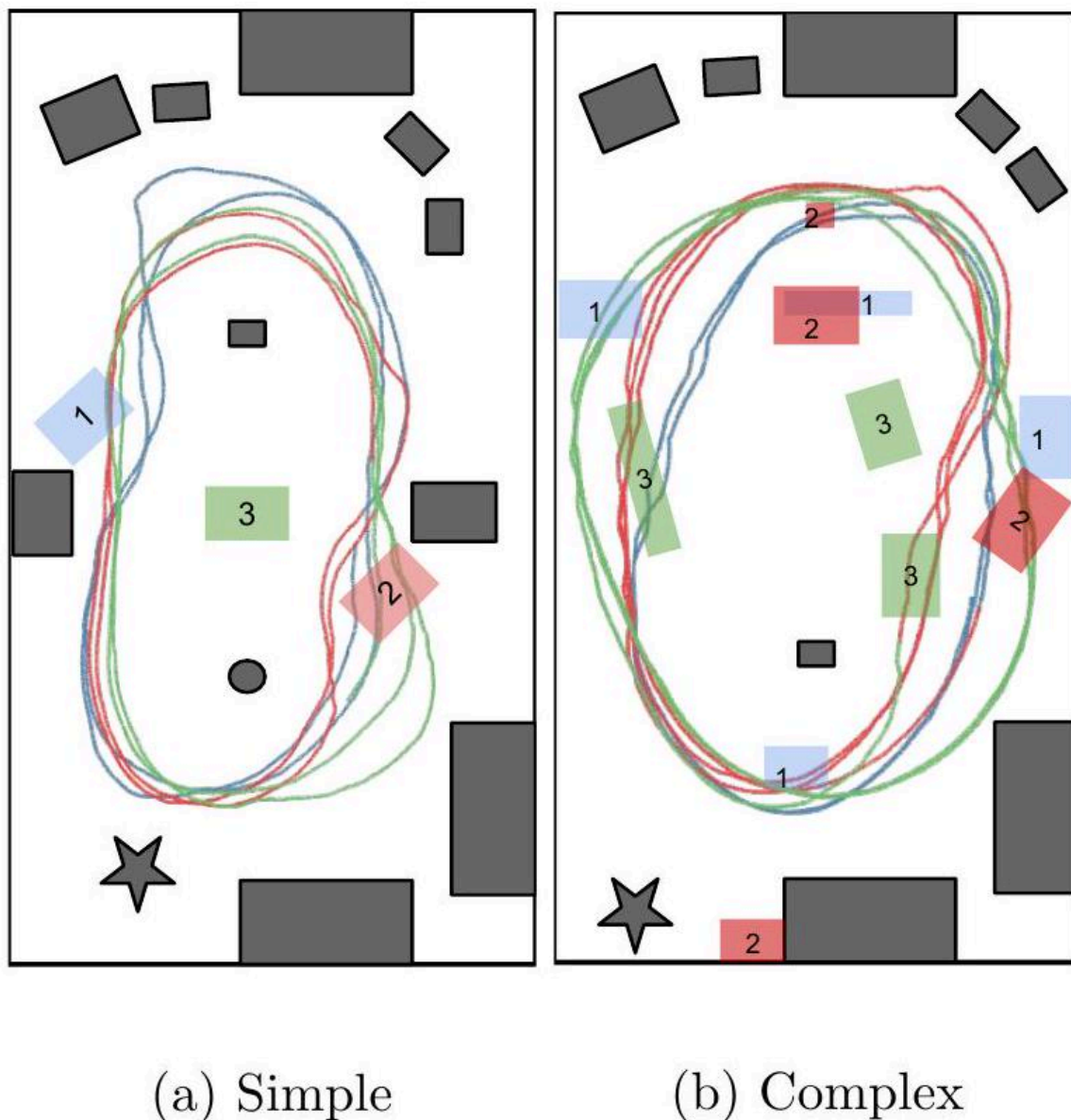


圖 16: 圖 3.8: 動態環境中的真實世界測試。灰色障礙物為固定物，彩色障礙物則表示可移動障礙物在各個時間點的運動。障礙物上的數字表示變動序列。

偏好直線行駛而非轉彎，因此機器人會沿著曲率最小的迴圈進行導航，以維持最大的線速度。

圖 3.8 展示了機器人在兩個動態環境中測試的結果。在此實驗中，我們動態移動環境中的物體，並驗證了機器人能夠透過單眼相機避開這些物體，這進一步證實了所提方法的有效性。另一個測試的影片可於 <https://youtu.be/qNIVgG4RUDM> 觀看。

3.5.2.4 與基準方法的比較

在此實驗中，將所提方法 (Ours) 與基於光流 (optic-flow) 的基準方法 [138] (OF) 進行比較。請注意，基準方法是根據 git 儲存庫³ 實作的。測試在兩個不同的環境中進行：如圖 3.7 所示的雜亂環境，以及如圖 3.6b 所示的走廊環境。在每個環境中進行 5 次獨立運行，每次運行限時一分鐘。同時，選擇碰撞次數 (NC) 和第一次碰撞前的距離 (Dsit) 作為性能指標。

表 3.1 顯示我們的方法顯著優於 OF，特別是在走廊環境中。如圖 3.9 所示，這是因為無紋理物體

圖 17: 表 3.1: 在兩種不同的環境中分別比較基於光流的基準方法 (OF) 與本研究方法。每項測試包含 5 次獨立執行，時限為一分鐘。使用的評估指標為碰撞次數 (NC) 以及發生第一次碰撞前或逾時前的行進距離 (Dist)。

	Cluttered		Corridor	
	NC	Dist(m)	NC	Dist(m)
OF	2.6	2.67	5.4	1.32
Ours	0.6	6.28	0.2	10.35

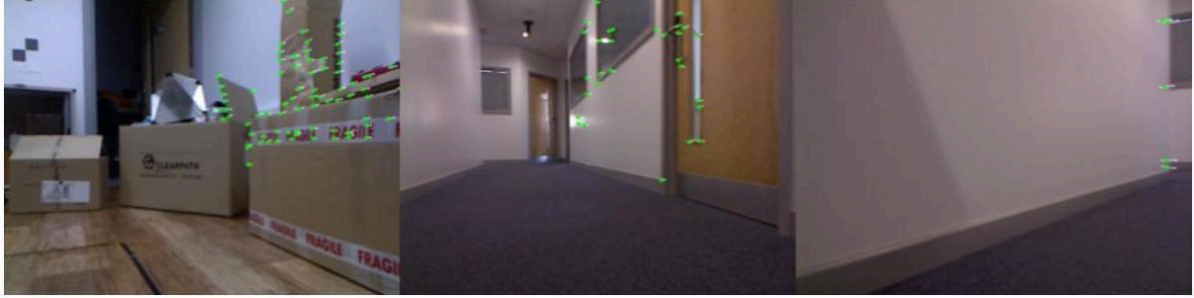


圖 3.9: 左圖為雜亂環境中的特徵檢測結果，其餘兩張圖則為走廊環境。環境中的（例如牆壁）會阻礙特徵檢測，而特徵檢測是基於特徵估計兩幀之間光流（optic-flow）的基礎。這是傳統基於視覺幾何方法的共同弱點。

3.6 結論

在本章中，我們提出了一種創新且實用的解決方案，旨在將學習到的 DRL 策略成功地從模擬環境遷移到真實世界，以執行單眼視覺障礙物規避任務。我們用來解決現實差距（reality gap）的創新見解，是將場景的視覺外觀領域（appearance domain）投影到合成深度領域（synthesised depth domain）。此方法易於實作，並能促進學習規避可見障礙物的能力。

廣泛的實驗表明，該網路可以完全在模擬器中進行訓練，然後僅透過單眼視覺直接部署在真實機器人上。此外，這是首次將 D3QN 架構應用於即時行動機器人控制任務，證明了其能提升學習效率，並對雜訊深度觀測具有強健性。

第 4 章

局部導航：透過引導加速訓練

在探討了如何彌合障礙物規避任務中的現實差距（reality gap）後，我們現在可以開始研究第二個挑戰。其目標是加速模擬環境中的訓練，以學習更複雜的任務，即區域導航（local navigation）。區域導航要求機器人：1) 到達區域座標系中的指定目的地，以及 2) 在途中規避障礙物。由於上述任一目標都可以透過簡單的傳統控制器部分解決，因此本章提出了一個新穎的框架，利用這些簡單的控制器來引導並提升離策（off-policy）DRL 方法（即 DDPG）的訓練效率。所提出的框架分別透過兩種演算法實現：用於確定性引導的 AsDDPG，以及用於隨機變體的 SGuidance。實驗結果顯示，這兩種演算法都能有效提升 DDPG 的訓練速度，尤其是在使用稀疏獎勵函數（sparse reward function）時。我們進一步注意到，SGuidance 的表現優於確定性版本，達到了接近最優（near-oracle）的性能。

4.1 引言

區域導航（目的地定義在機器人的區域座標系中）與障礙物規避相比，是機器人導航中更為複雜的任務，因為它需要更細緻的控制策略，以同時靠近目的地並繞過所有障礙物。

Deep Deterministic Policy Gradient (DDPG) [79] 是一種 Actor-Critic 演算法，原則上適用於此類連續控制任務，但在實務上，在複雜導航環境中進行探索的成本可能高得令人望而卻步。由於代理人（agent）在每個訓練回合（episode）中必須隨機探索一長串的狀態序列，高變異數（high variance）成為阻礙 DDPG 學習有效 DRL 模型的主要瓶頸。為了緩解這個問題，傳統架構通常需要大量的學習樣本，進而導致高昂的運算與環境成本。

在機器學習領域，研究人員大多專注於加速 DRL 訓練的演算法技術，例如平行訓練 [101] 和資料效率 [127]。然而，這些演算法並未考慮特定任務的上下文資訊，而這些資訊對於訓練可能非常有價值。以本研究的局部導航任務而言，確實存在現有的解決方案可使 DRL 訓練受益。雖然這些解決方案可能並非最優，但在大多數情況下，其表現仍優於隨機探索策略。如何充分利用並受益於先前的研究方法，並將其與 DRL 緊密結合以加速訓練，是一個重要且尚未解決的課題。

在本章中，我們提出了一個新的框架，允許代理人在高變異數控制器（例如 DDPG）與低變異數控制器（例如簡單的確定性控制器）之間切換，從而有效地讓 DDPG 組件快速啟動引導（bootstrapped），而非從完全隨機的動作開始。

此外，我們提出了兩種不同的切換策略學習方法。一種是輔助式 DDPG（Assisted DDPG, AsDDPG），它使用 DQN 來確定性地選用具有最高 Q 值的控制器。另一種是隨機引導（Stochastic Guidance, SGuidance），它透過 REINFORCE 演算法學習機率分佈，據此從不同的控制器中對動作進行取樣。

4.2 相關工作

許多研究已將 DRL 應用於機器人問題，例如導航 [146, 181, 124, 168, 153, 174, 173] 與操作 [45, 171]。由於大多數機器人問題涉及連續控制，因此 Policy-based 方法（如 Policy Gradient [142]）或 Actor-Critic 方法 [44]（如 DDPG [80]）被廣泛使用。然而，由於環境的高複雜性，這些方法通常需要大量的學習樣本來應對高變異度（High Variance）問題。引入歸納偏置（Inductive Bias）[52] 是緩解此問題的常用方法。[127] 對模型信心較低的數據分配較高的權重，以提高樣本使用效率。[55] 在探索新策略時利用了資訊增益（Information Gain）的概念。還有其他在不增加額外偏置的情況下加速訓練的方法，例如平行訓練 [101]，透過同時收集多個多樣化的學習回合（Episodes）來抑制變異度。在 [162] 中，提出了一種先進的網路架構

將 Q-value 的估計分解為 Value 與 Advantages。與上述將偏置緊密融合到模型中的方法不同，我們的框架將額外知識納入作為隨機引導（Stochastic Guidance），而無需對底層方法進行任何更改。

Yang 等人 [173] 提出了一種基於階層式強化學習（Hierarchical Reinforcement Learning）的方法。它透過多個獨立的 Actor 學習基本技能，最後從具有最高 Q-value 估計值的 Actor 中取樣動作，這與確定性 AsDDPG 的實作類似。然而，我們的目標僅是輔助 DDPG 的訓練，並最終捨棄其他的控制器。

Thompson Sampling [62] 與 SGuidance 均採用了學習在不同控制器之間切換的概念。其不同之處在於，我們的框架並非像 Thompson Sampling 那樣顯式地計算後驗分佈以進行更新，而是直接利用神經網路來建構潛在分佈（latent distributions），並透過反向傳播（backpropagation）與 DDPG 組件進行聯合訓練。其優點在於切換函數（switch function）可以輕鬆建立，並以所有感測器輸入為條件，使其能根據不同的情境或條件選擇不同的控制器。此外，我們框架的目標更專注於訓練一個更好的 DDPG 組件，使其能受益於隨機切換所產生的優質樣本，進而獲得低變異數（low-variance）的梯度估計器。

在 [77] 中，Leonetti 等人介紹了另一種將外部資訊納入 DRL 方法訓練中的方式。該方法利用全域規劃器（global planner），根據環境地圖來限制 DRL 的可用動作，然而這與我們的無地圖導航（mapless navigation）任務並不相容。此外，在我們的框架中，DDPG 在整個訓練過程中可以與引導機制並行，自行探索完整的動作空間，並最終能夠獨立運作。

Xie 等人 [170] 提出利用 DRL 方法學習回饋規劃策略（feedback planning policy）。該規劃器學習在各種情況下為底層 PD 控制器分配不同的參考訊號，以控制雙足機器人。雖然這種方法在早期階段能有效加速學習過程，但無論學習到的規劃策略有多好，其整體效能都會受限於底層的啟發式控制器（heuristic controller）。在此，啟發式控制器代表了基於人類知識所設計的一些次優（sub-optimal）控制器。

所提出的框架也與模仿學習（imitation learning）[32, 115] 相關。在模仿方法中，演示者（demonstrator）必須足夠優秀才能解決問題。但我們的方法適用範圍更廣，因為它應能與簡單的啟發式控制器以及複雜的可學習控制器相容。這是因為，如何使用這些

控制器由切換策略（switching policy）決定。它學習在不同的觀測值下使用合適的控制器，而這些控制器在其他情況下可能完全無用。此外，在獲得良好的 DDPG 元件後，所有的啟發式方法（或控制器）都可以被移除，我們能夠僅透過 DDPG 元件在環境中進行測試，而不需要任何其他控制器。

4.3 問題定義與假設

我們可以將局部導航問題視為一個決策過程 [10]，其中機器人被要求避開障礙物並到達目標位置。在時間 $t \in [0, T]$ ，機器人根據觀測值 x_t 採取動作 $a_t = [v_t, \omega_t] \in \mathcal{A}$ ，該動作控制機器人的線速度 v_t 和角速度 ω_t 。執行動作後，機器人根據獎勵函數接收由環境給予的獎勵 r_t ，然後轉移到下一個觀測值 x_{t+1} 。此決策程序的目標是達到最大化的折扣累積未來獎勵 $R_t = \sum_{\tau=t}^T \gamma^{\tau-t} r_\tau$ ，其中 γ 為折扣因子。請注意，由於本章僅專注於加速模擬環境中的訓練，我們直接將幾何感測數據（例如不同模擬器中的雷射掃描或深度攝影機）作為觀測值。

在時間 t 的獎勵函數 r_t 定義如下：

$$r_t = \begin{cases} R_{\text{crash}}, & \text{if robot crashes} \\ R_{\text{reach}}, & \text{if robot reaches the goal} \\ (d_{t-1} - d_t) \cos(\omega_t) - C, & \text{otherwise} \end{cases}$$

其中 R_{crash} 是碰撞的大幅懲罰， R_{reach} 是到達目標的正向獎勵， d_{t-1} 與 d_t 表示機器人在連續兩個時間步 $t-1$ 與 t 時與目標之間的距離，而 C 則是一個常數時間懲罰，用以鼓勵機器人快速接近目標。

4.4 AsDDPG：結合確定性引導的學習

我們框架背後的核心見解是提供一個簡單的控制器來輔助網路進行策略探索，進而加速並穩定訓練過程。我們將此框架的確定性實作稱為輔助式 DDPG（Assisted DDPG），簡稱為 AsDDPG。其直覺非常簡單：對於簡單的任務（例如直線行駛），原始的控制律表現會優於隨機策略。

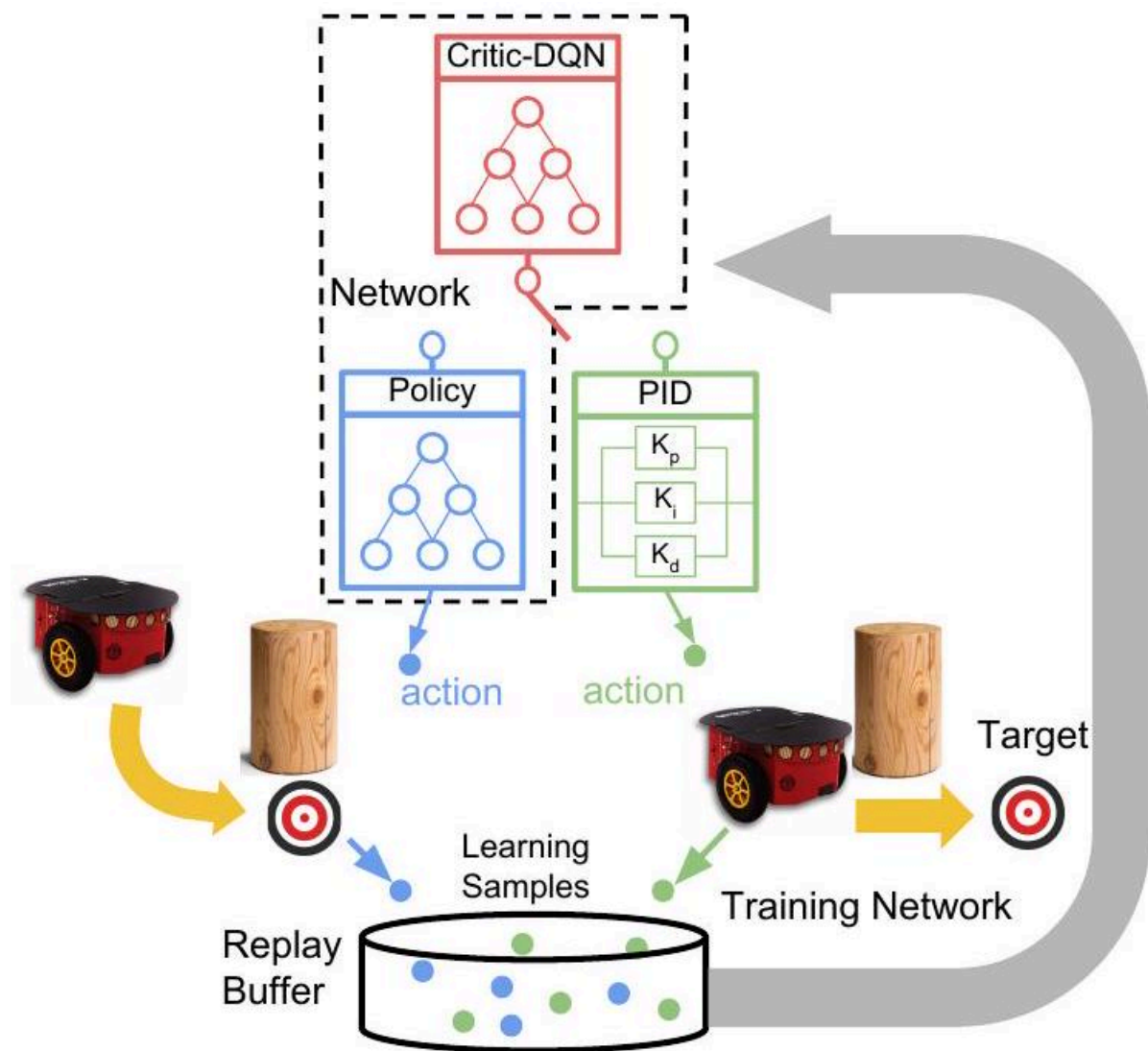


圖 18: 圖 4.1: 使用 Actor-Critic 強化學習方法訓練深度神經網路，以學習機器人導航的局部規劃。Critic-DQN 網路會同時評估外部控制器（例如 PID 控制器）與策略網路的表現，並根據情況從中選擇較佳的動作。所有產生的學習樣本都會儲存在重播緩衝區（replay buffer）中。因此，策略網路可以透過模仿外部控制器或檢視自身的策略來自我提升。

然而，我們並非僅將此控制器視為如 ϵ -greedy 般的獨立探索方法，而是將 Critic 網路與 DQN 結合，以自動判斷應使用哪種策略來最大化獎勵，如圖 4.1 所示。本質上，增強後的 Critic 網路控制著一個開關，決定機器人是遵循控制器的建議動作還是學習到的策略。這可以避免手動調整參數來決定何時以及如何使用此類外部控制器，並有潛力推導出最佳策略。

由於 AsDDPG 是一種離策（off-policy）學習方法，其網路從經驗回放池（replay buffer）中學習，因此無論目前的策略為何，Actor 網路都能從其自身記錄的策略以及輔助控制器所產生的學習樣本中獲益。起初，簡單控制器會較頻繁地被選為最佳策略。然而，隨著時間推移，學習到的策略在總獎勵方面將超越簡單控制器。一旦訓練收斂，Critic 和外部控制器皆可捨棄，機器人只需根據學習到的策略進行導航。

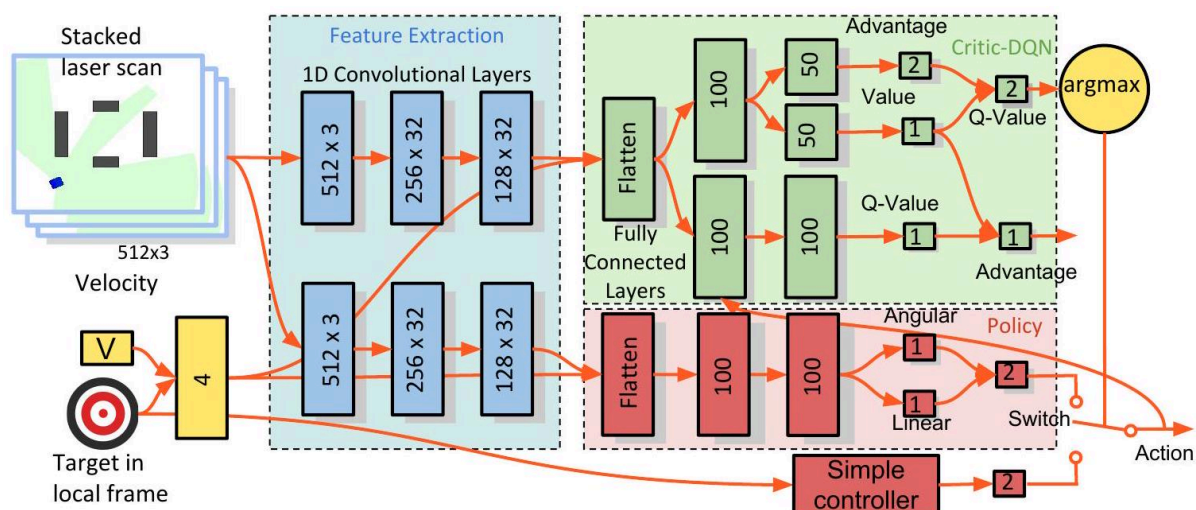


圖 19：圖 4.2：網路架構。網路層以矩形表示。橙色箭頭表示網路層與其他組件（例如輸入狀態和簡單控制器的輸出）之間的連接性。最終動作是根據 Critic-DQN 預測的 Q 值來選擇。

4.4.1 啟發式控制器

我們首先介紹作為外部引導使用的簡單控制器。這是一個比例控制器 (P controller)，為具有比例項 [5] 的比例-積分-微分 (PID) 控制器的特例，其根據機器人座標系中目標點 $[x_{\text{local}}, y_{\text{local}}]$ 的相對位置推導出動作如下：

$$a_t = K_p \cdot [x_{\text{local}}, y_{\text{local}}]^T,$$

其中 K_p 是比例項的二維係數向量。PID 控制器是應用最廣泛且最成功的控制機制之一。然而，在不考慮幾何觀測的情況下，它並不具備避障能力。採用簡單的 P 控制器而非完整的 PID 控制器，是因為後者可能會為學習過程引入過多偏差，並可能在訓練後期對達成最佳策略產生負面影響。

4.4.2 網路架構

接著，為了將先前討論的直覺付諸實踐，我們實作了如圖 4.2 所示的網路架構。它包含三個部分，分別是特徵提取（藍色）、策略網路與輔助控制器（紅色），以及增強型 Critic 網路（綠色）。

系統的第一部分是 1-D 卷積層 (convolutional layers)，用於從堆疊的密集雷射掃描 (dense laser scans) 中提取特徵。所使用的激活函數為 ReLU。我們發現這些卷積層對於策略在避障性能以及現實世界的泛化能力方面都至關重要。在 [146] 中，作者僅使用稀疏雷射掃描 (10 條掃描線)，這雖然在不同場景中具有良好的泛化能力，但機器人難以平滑地避開小型障礙物。直觀上，對高保真觀測值進行降採樣會損失資訊，並非理想做法。因此，我們傾向於保留密集雷射掃描作為輸入，並改用 1-D 卷積層來學習該任務的有效特徵。此外，將多個時間點的輸入進行堆疊也能提供更多關於環境的資訊。

第二部分是具有全連接層 (fully-connected layers) 的策略網路 (policy network)，僅根據從輸入狀態（例如：雷射掃描、當前速度和局部座標系中的目標位置）提取的特徵來估計機器人的最佳線速度與角速度。請注意，這兩個輸出的激活函數分別為 sigmoid 和 tanh。外部控制器也會根據當前位置與目標位置之間的誤差訊號產生控制訊號（策略）。

最後，由全連接層構成的 Critic-DQN 是第三部分。它包含兩個分支：一個是 critic 分支，其中動作 (action) 被串接 (concatenated) 到第二層；另一個是 DQN 分支，我們在此應用了 dueling [160] 與 double network [156] 架構，以加速訓練並避免過度估計 (overestimation)。請注意，其輸出層沒有非線性激活函數。我們將在下文詳細討論 critic。

4.4.3 Critic-DQN

critic-DQN 的兩個分支分別擔任以下角色：1) 作為評論者 (criticiser)，評估策略網路輸出的動作並生成策略梯度；2) 作為開關 (switch)，決定何時使用來自策略網路的學習策略或外部控制器。

Critic 分支與原始的 DDPG 相似。然而，它會從每個狀態-動作對的 Q-value $Q^\pi(x, a)$ 中估計優勢 (advantage) $A^\pi(x, a)$ 。這是透過 DQN 分支中的競爭網路 (dueling network) 實現的，其中每個狀態的價值 (value) $V^\pi(x)$ 也會被學習。根據優勢的定義，它可以簡單地計算為 $A^\pi(x, a) = Q^\pi(x, a) - V^\pi(x) \approx Q^\pi(x, s) - V^\pi(x)$ 。請注意，critic-DQN 分支所考慮的動作是每個時間步的切換動作 s_t 而非 a_t ，這也與 2.1.6.2 節中定義的取樣軌跡 (sampled trajectory) s 概念不同。根據 [142]：

Algorithm 1 AsDDPG

procedure Training

```

    Initialise  $(A \leftarrow (x, a \mid \theta^A), Q \leftarrow (x, s \mid \theta^Q))$ 
    Initialise target network  $(\theta^{A'}, \theta^{Q'})$  and  $(\theta^A, \theta^Q)$ 
    Initialise replay buffer  $R$  and exploration noise  $(\epsilon)$ 
    for episode  $(=1, M)$  do
        Reset the environment
        Obtain the initial observation
        for step  $(=1, T)$  do
            Infer switching  $([Q_{\text{policy}}, Q_P], Q_P) = Q \leftarrow (x_t, s_t)$ 
             $(s_t = \operatorname{argmax}([Q_{\text{policy}}, Q_P])$ 
            if  $(s_t == 1)$  then
                Sample policy action  $(a_t = \pi(x_t \mid \theta^{\pi}))$ 
            else

```

```

        Sample action  $(a_t)$  from external controller
    end if
    Execute  $(a_t)$  and obtain  $(r_t, x_{t+1})$ 
    Store transition  $((x_t, a_t, r_t, x_{t+1}, s_t))$  in R
    Sample N transitions  $((x_i, a_i, r_i, x_{i+1}, s_i))$ 
    Optimise critic-DQN by minimising Eq. 4.3
    Update the policy according to Eq. 4.4
    Update the target networks
end for
end for
end procedure

```

與 Q-value 相比，估計優勢能大幅降低變異量 (variance)，對於快速學習至關重要，特別是對於基於策略梯度的法。整個 critic 分支表示為 $A(x, a | \theta^A)$ ，而估計優勢前的部分則表示為 $Q^A(x, a | \theta^A)$ 。

DQN 分支會根據狀態輸入，估計並比較「使用動作網路策略」或「應用外部控制器動作」的 Q-value。它會貪婪地 (greedily) 切換至具有較高 Q-value 估計值的選項。DQN 分支表示為 $Q(x, s | \theta^Q)$ 。

4.4.4 訓練

學習樣本可以定義為元組 $(x_t, a_t, r_t, x_{t+1}, s_t)$ ，其中 s_t 是一個二進位變數，表示切換動作。在這些變數中， a_t 僅會影響 Critic 分支的更新，而 s_t 則僅用於 DQN 分支。

在 Critic-DQN 中，兩個分支都透過引導法 (bootstrapping) 來優化其網路參數。這意味著它們根據公式 2.3，從 Q 值估計的時序差分 (TD) 誤差中進行學習。更具體地說，權重是根據以下損失函數 L 進行優化的：

$$\begin{aligned}
 y_i^A &= r_i + \gamma Q^{A'}(x_{i+1}, \pi'(x_{i+1} | \theta^{\pi'}) | \theta^{A'}) \\
 y_i^Q &= r_i + \gamma Q'(x_{i+1}, \underset{s}{\operatorname{argmax}} Q(x_{t+1}, s_{t+1} | \theta^Q) | \theta^{Q'}) \\
 L &= \frac{1}{N} \left(\sum_i \left(y^A - Q^A(x_i, a_i | \theta^A) \right)^2 + \left(y^Q - Q(x_i, s_i | \theta^Q) \right)^2 \right)
 \end{aligned}$$

其中 $Q^{A'}, Q'$ 是兩個分支的目標網路 (target networks)，而 i 是批次 (batch) 中樣本的索引。請注意，Critic 分支是透過其 Q 值估計進行更新，而非使用於產生策略梯度 (policy gradient) 的最終輸出優勢值 (advantage)。

對於 Actor 網路，其權重會根據方程式 2.8 中定義的策略梯度 (policy gradients) 進行調整。這需要 Critic 分支先計算其優勢輸出 (advantage output) $A^\pi(x, a)$ 對動作 a 的梯度。接著，此梯度會傳遞給 Actor 網路，以計算對網路參數 θ^π 的梯度。

$$\begin{aligned}
 \nabla_{\theta^\pi} J(\theta^\pi) &\approx \mathbb{E} \left[\nabla_{\theta^\pi} A(x, a | \theta^A) \Big|_{x=x_t, a=\pi(x_t | \theta^\pi)} \right] = \\
 &\mathbb{E} \left[\nabla_a A(x, a | \theta^A) \Big|_{x=x_t, a=\pi(x_t)} \nabla_{\theta^\pi} \pi(x | \theta^\pi) \Big|_{x=x_t} \right]
 \end{aligned}$$

演算法 1 概述了 AsDDPG 的完整訓練過程。

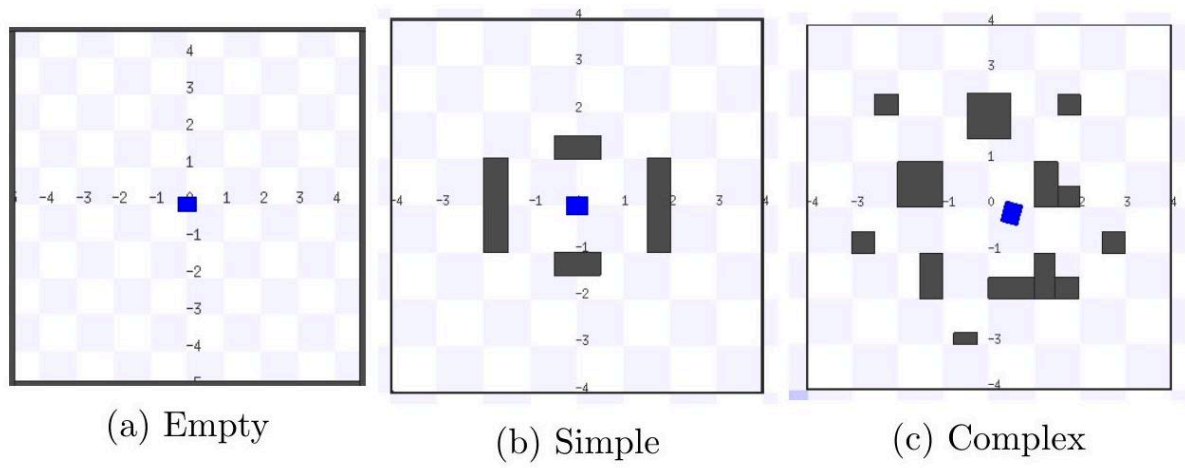


圖 20: 圖 4.3: 用於訓練的三個 Stage 模擬世界。灰色矩形是障礙物，藍色矩形是機器人。目標位置是針對每個回合 (episode) 隨機生成的。

4.5 AsDDPG 實驗

我們進行了多項實驗，以評估所提出的 AsDDPG 演算法與原始 DDPG 在機器人導航問題上的性能表現。我們在 Stage 和 Gazebo 模擬器中訓練網路，並在真實世界場景中測試學習到的策略 (policy)。

在 Stage 模擬器的訓練中，我們使用了如圖 4.3 所示的三種環境。DDPG 和 AsDDPG 均使用相同的獎勵函數 (reward function) 進行訓練。具體而言，該函數包含稀疏部分 R_{reach} 與 R_{crash} ，即機器人到達目標時獲得巨大的正向獎勵，而碰撞障礙物時則獲得巨大的負向獎勵；以及如下所示的密集部分：

$$r_t = \begin{cases} R_{\text{crash}}, & \text{if crashes} \\ R_{\text{reach}}, & \text{if reaches the goal} \\ \gamma_p ((d_{t-1} - d_t) \Delta t - C), & \text{otherwise} \end{cases}$$

其中 d_{t-1} 與 d_t 表示機器人與目標在兩個連續時間戳記之間的距離， Δt 代表每一步的時間， C 是作為時間懲罰 (time penalty) 的常數，而 γ_p 則是折扣因子 (discount factor)。

請注意，我們引入了另一個獎勵折扣因子 γ_p 。當使用策略網路時，其設定為 1；但當密集獎勵為正值時，為了懲罰對外部控制器的依賴，會將其設定為較小的值 (例如 0.5)。這有兩個目的：1) 更頻繁地嘗試策略網路，以及 2) 更快地學習如何獨立於外部控制器運作。從理論上講，由於始終存在比利用外部控制器更好的策略，網路可以學習逐漸減少對外部控制器的使用。

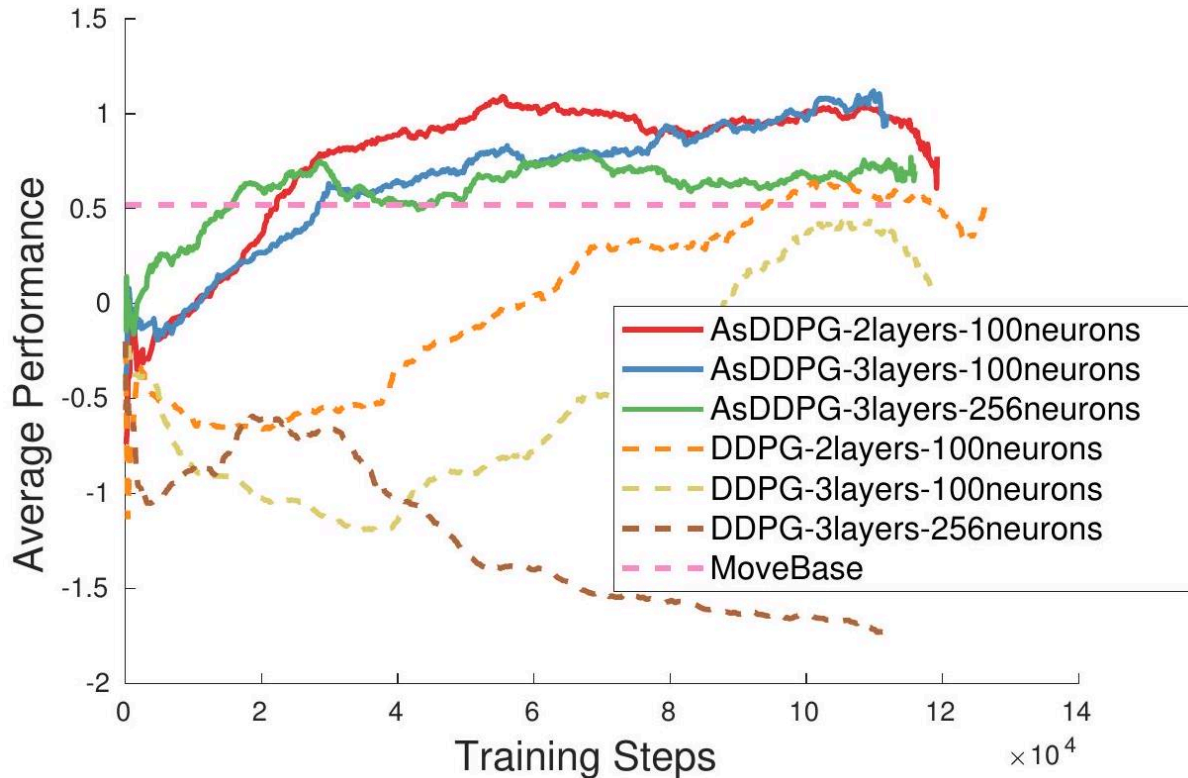


圖 21：圖 4.4：不同網路超參數下的平滑學習曲線。此圖展示了網路在每個訓練步驟中的平均表現。請注意，MoveBase 被選為基準方法（baseline），其平均表現得分約為 0.5。

上述獎勵函數並不一定能提供客觀的性能指標，因為其設計初衷是為了減輕訓練難度。因此，在本實驗中，我們使用基於到達目標所需時間以及機器人是否到達目標的導航任務指標。具體而言，每個步驟會給予 -0.01 的時間懲罰，而到達目標則給予 2 的正向獎勵。

4.5.1 利用各種超參數加速訓練過程

DDPG 的一個已知問題是對網路超參數（hyper-parameters）的高度敏感性。手動調整超參數以使 DDPG 收斂非常耗時，且在理想情況下應盡量避免。因此，在本實驗中，我們檢測了三種不同設置的網路，包括具有兩個或三個全連接層（fully connected layers）且每層 100 個神經元，以及具有三個層且每層 256 個神經元的配置。

不同超參數在超過 10 萬次訓練步數後的學習曲線如圖 4.4 所示。結果顯示 AsDDPG 在訓練效率方面優於 DDPG，且在不同網路超參數下比 DDPG 更穩定。值得注意的是，具有三層且每層 256 個神經元的 DDPG

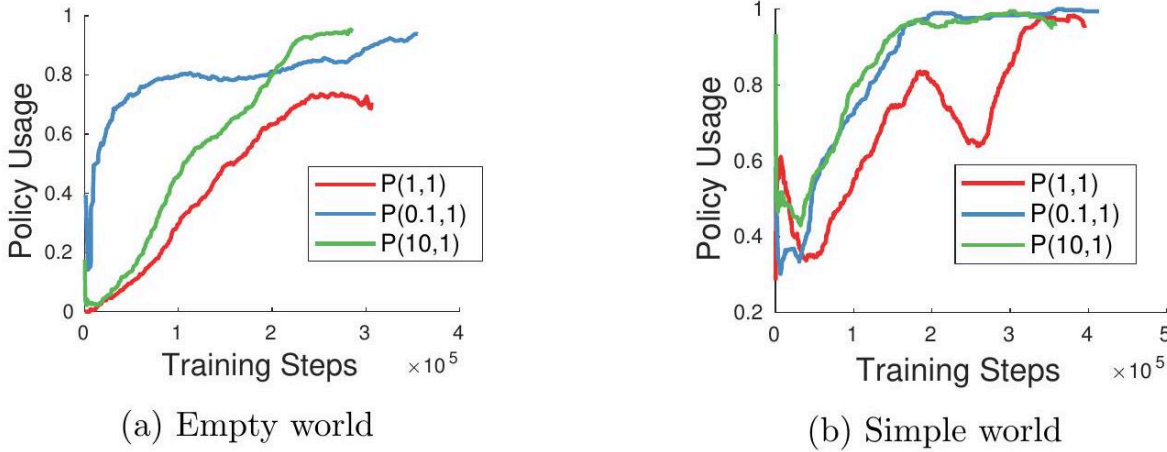


圖 22：圖 4.5：在兩個不同模擬世界中，整個訓練過程中單一回合（episode）內策略網路的使用情況。

未能學習到合理的策略。此外，我們使用 MoveBase¹ 作為基準方法，以展示所學策略與現有確定性方法的比較。MoveBase 套件² 是一個廣泛用於行動機器人導航的運動規劃器，並實作於名為 Navigation Stage 的 ROS 套件中。它由區域規劃器 [39, 36] 和全域規劃器（由 A* 演算法實作）組成。全域規劃器在環境的全域地圖上生成從起點到終點的最佳路徑，而區域規劃器則在沿著最佳路徑移動時，動態避開新偵測到的障礙物。然而，在實驗中，為了公平比較，全域規劃器僅被提供一張未記錄環境中任何障礙物的空白地圖。同時，透過增加參數「pdist_scale」，區域規劃器被賦予更高的容忍度，使其能根據觀察到的障礙物偏離預定路徑。

4.5.2 控制器參數的影響

在本實驗中，我們在兩個 Stage 環境（圖 4.3a 與圖 4.3b）中，使用不同的控制器參數訓練了數個模型，以研究 AsDDPG 對控制器參數的敏感度，即控制器參數如何影響訓練過程。我們同時也檢視 critic-DQN 選擇學習策略（learned policy）而非外部控制器的頻率，藉此研究它是否能逐漸變得獨立並學習到良好的策略。

比例控制器 $P(P_l, P_r)$ 配置了兩個參數。它分別以 $v = P_l \cdot x_{\text{local}}$ 與 $\omega = P_r \cdot y_{\text{local}}$ 來控制線速度與角速度，其中 x_{local} 與 y_{local} 是目標點在機器人局部座標系中的座標。

我們透過更改線性增益參數來研究控制器的三種設定，即 $P(0.1, 1)$, $P(1, 1)$ 與 $P(10, 1)$ 。 $P(10, 1)$ 是速度最快的控制器，但會面臨目標超調（overshooting）的問題，導致機器人需要轉身。 $P(0.1, 1)$ 是速度最慢的控制器，對機器人速度的調整非常緩慢，隨之而來的是加速度較慢。 $P(1, 1)$ 則是一個在沒有嚴重超調的情況下能提供良好性能的控制器。

圖 4.5 顯示了 critic-DQN 所選擇的策略（policy）與外部控制器（controller）之間的比例在訓練過程中隨時間演變的情況。首先，可以看出 critic-DQN 隨著時間推移，從控制器採樣的頻率逐漸降低，更多的動作來自於策略，儘管控制器的參數和環境決定了這一趨勢的速度。一個明顯的現象是，在兩個模擬世界中，網路都迅速捨棄了緩慢的控制器 $P(0.1, 1)$ ，因為它通常需要更長的時間才能到達目標，且策略網路很容易就能超越它。然而，對於 $P(1, 1)$ 和 $P(10, 1)$ ，策略與控制器之間的比例在兩個世界中有所不同。在圖 4.6a 和 4.6b 中，透過不同訓練階段的機器人軌跡更詳細地呈現了這種行為。這些圖表還顯示了隨著訓練的進行，critic 網路選擇控制器（藍色軌跡）而非學習到的策略（紅色策略）的頻率。

根據圖 4.6a 所示的結果，無論使用哪種控制器，網路首先會學習在最初的幾步中應用自己的策略，如原點周圍集中的紅色所示。這是因為總獎勵受到初始朝向的強烈影響。考慮到最快的控制器 $P(10, 1)$ ，可以看到最初機器人有時會因為速度過快而在目標周圍盤旋。策略網路透過在開始時選擇更好的朝向解決了這個問題，證明了網路可以學習何時正確使用外部控制器。

特別是在使用 $P(10, 1)$ 的第二行中，機器人在接近目標時有時會在目標附近盤旋。策略網路透過在開始時以更合適的朝向接近目標，解決了這個問題。

使用 $P(1, 1)$ 時，它會採取更準確的朝向指向目標，並直接導航至該處。在中間階段，策略網路學習到了一個更平滑但在時間上並非最優的策略。這可能是透過獎勵函數懲罰使用外部控制器所產生的副作用。然而，這種懲罰對於穩定切換策略仍然至關重要。最終，策略網路學習到的通往目標的路徑變得更加筆直且最優。由於 $P(1, 1)$ 是一個優秀的控制器，即使在經過 ≈ 8000 個訓練回合後，網路也沒有捨棄它。

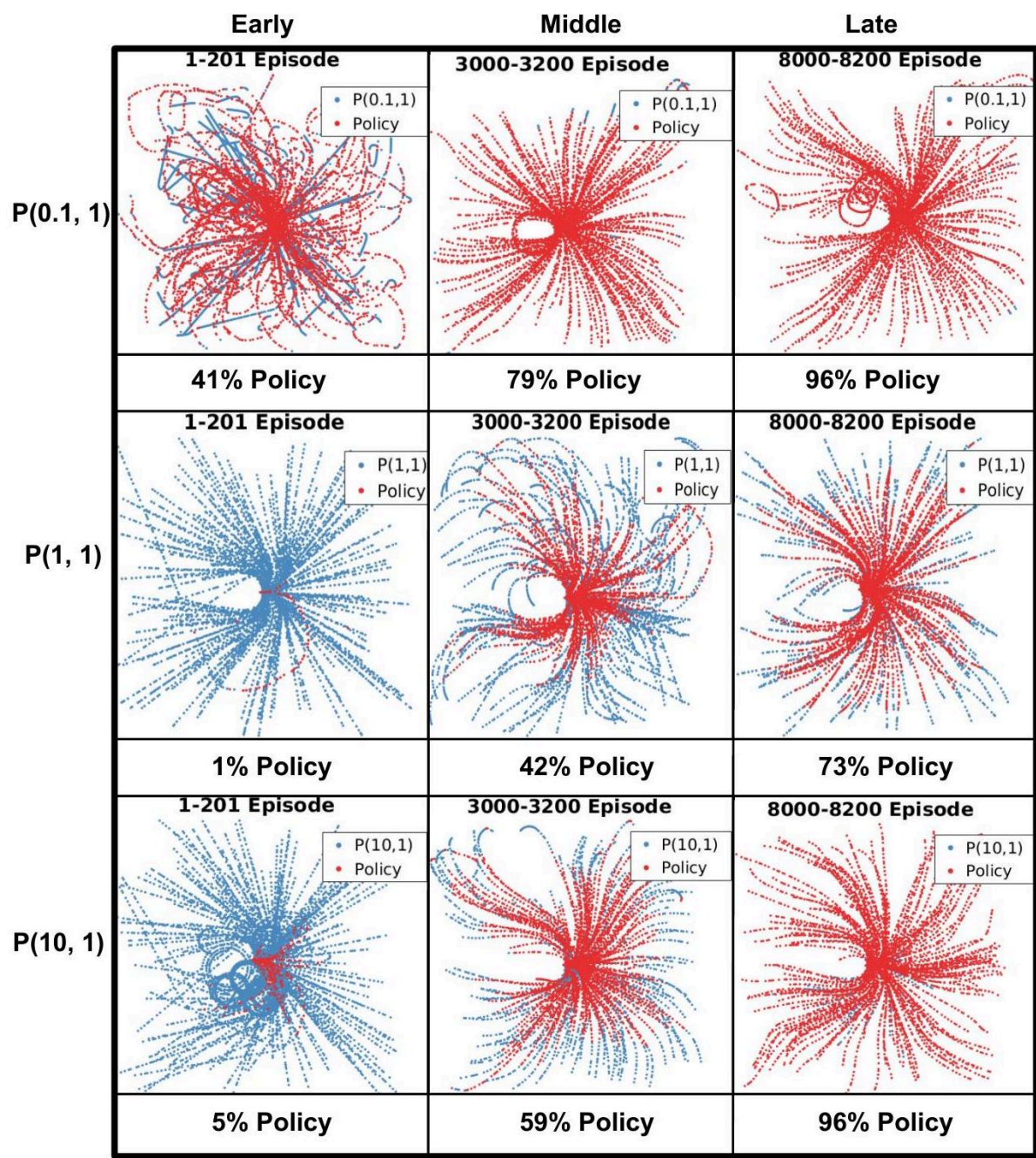


圖 23: (a) 空曠世界 (Empty world)

圖 4.6: 機器人在訓練不同階段搭配各種比例控制器 (proportional controllers) 所學習到的策略。每張圖像分別展示了在訓練程序的早期、中期和後期，跨越 200 個回合 (episodes) 的機器人軌跡，以及外部控制器與策略網路之間的切換結果。

圖 4.6b 顯示了在簡單世界 (simple world) 環境中訓練時的機器人軌跡。可以看出網路學習到了一些明顯的行為。即使在開始階段，外部控制器的取樣頻率也比在空曠世界中低。這是合理的，因為純 PID 控制器無法處理避障問題。此外，不同控制器的學習速度也各不相同。例如，

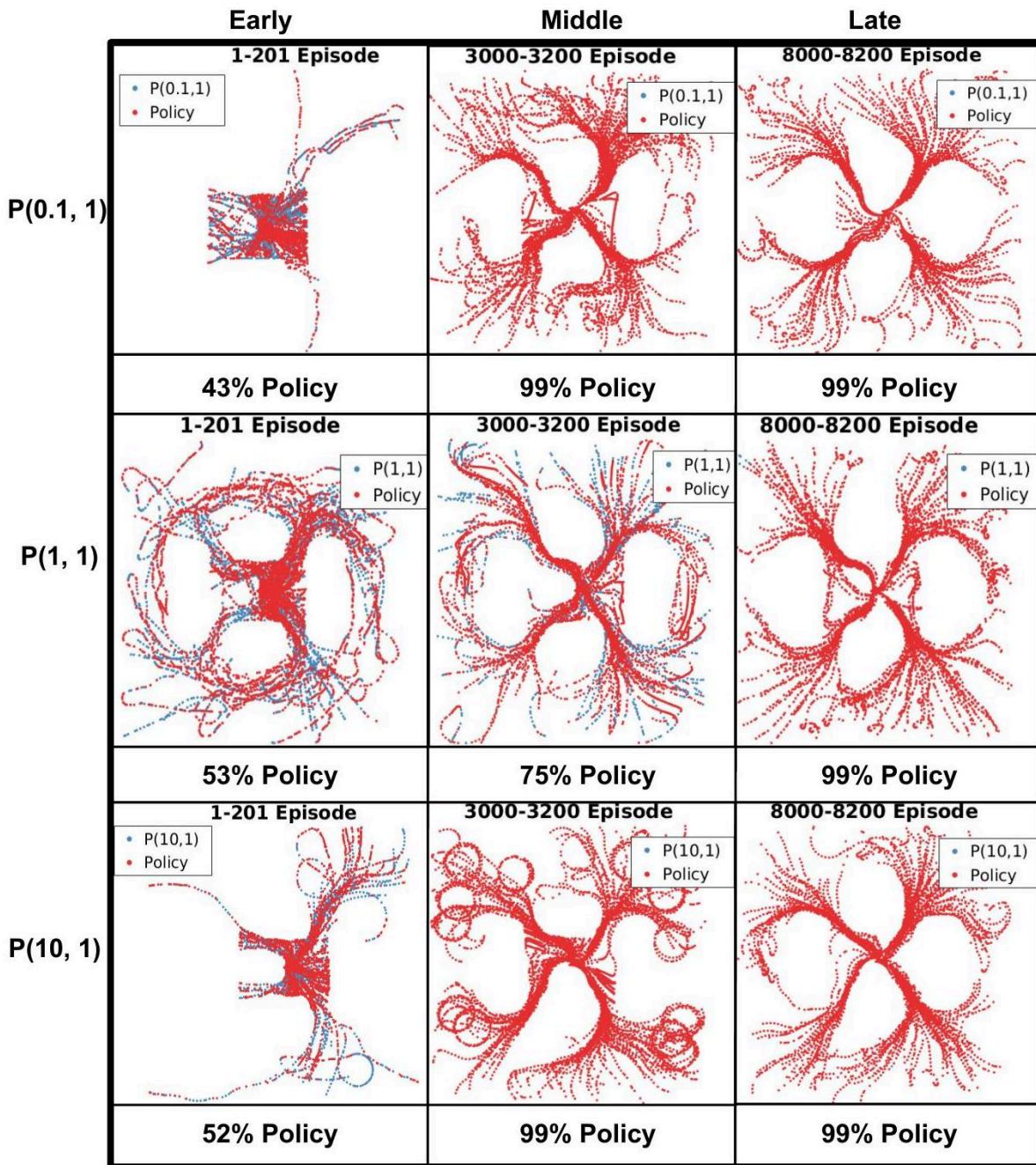


圖 24: (b) 簡單世界 (Simple world)

圖 4.6: 機器人在訓練不同階段中，配合各種比例控制器 (proportional controllers) 所學習到的策略。每張圖片分別展示了訓練初期、中期與後期，在 200 個回合 (episodes) 中機器人的運動軌跡，以及外部控制器與策略網路之間的切換結果。

相較於其他設定，網路在 $P(1, 1)$ 的情況下更容易學會繞過障礙物。而在 $P(10, 1)$ 的情況下，由於控制器提供的引導容易造成混淆，網路的學習速度明顯較慢。雖然網路很早就捨棄了控制器，但在訓練中期仍保留了一些不良行為，例如在目標附近盤旋。然而，可以觀察到評論家 (critic) 最終幾乎變得與 100% 無關。

在 ≈ 8000 個回合之後。

實驗結果顯示，經過足夠次數的訓練步驟後，學習到的策略可以捨棄外部控制器，並有效地導航機器人繞過障礙物。這證實了如果網路經過足夠回合的訓練，外部控制器對 AsDDPG 的最終效能幾乎沒有影響。然而，不同的 P 值確實會影響學習速度。不當的 P 值會導致代理人 (agent) 過早放棄 P 控制器，轉而純粹從隨機探索中學習；而較佳的 P 控制器則能引導學習過程更長的時間，並使學習速度更快。

4.5.3 在複雜環境與稀疏獎勵下的訓練

下一個實驗旨在驗證 AsDDPG 是否能在如圖 4.3c 所示的更複雜環境中，利用稀疏獎勵函數 (sparse reward function) 學習到良好的策略。在先前的實驗中所使用的密集獎勵函數 (dense reward function)，透過引導機器人縮短與目標的距離以獲得更高的即時獎勵，從而降低了訓練難度。但與此同時，它也會誘導網路學習到關於時間的次優策略 (sub-optimal policy)，因為除了以最快速度抵達目標外，網路還受到其他因素的驅動。在本實驗中，我們使用的獎勵函數其密集部分僅為常數的時間懲罰 (time penalty)。

$$r_t^{\text{sparse}} = \begin{cases} R_{\text{crash}}, & \text{if robot crashes} \\ R_{\text{reach}}, & \text{if robot reaches the goal} \\ -C, & \text{otherwise} \end{cases}$$

對於隨機探索 (random exploration) 而言，這是一個極具挑戰性的獎勵函數，因為機器人只有在真正抵達目標時才會獲得正向獎勵。

DDPG 與 AsDDPG 的效能如圖 4.7 所示。可以看出，DDPG 很少能學習到正確抵達目標的策略，且總是與障礙物發生碰撞。更具體地說，在早期階段，DDPG 經常發生超時 (run out of time)，由於累積的時間懲罰，這是最糟糕的情況。在大約 80k 個訓練步數後，它學會了透過碰撞來避免時間懲罰，而不是去達成目標。相比之下，雖然 AsDDPG 在開始時也會超時，但它在短短 3k 步內就轉向將碰撞視為更好的策略，並最終學會抵達目標以獲得最大的總獎勵。這進一步證實了所提出的 AsDDPG 在外部控制器的協助下，即使在具有稀疏獎勵的複雜環境中，也能有效加速訓練並取得更好的效能。

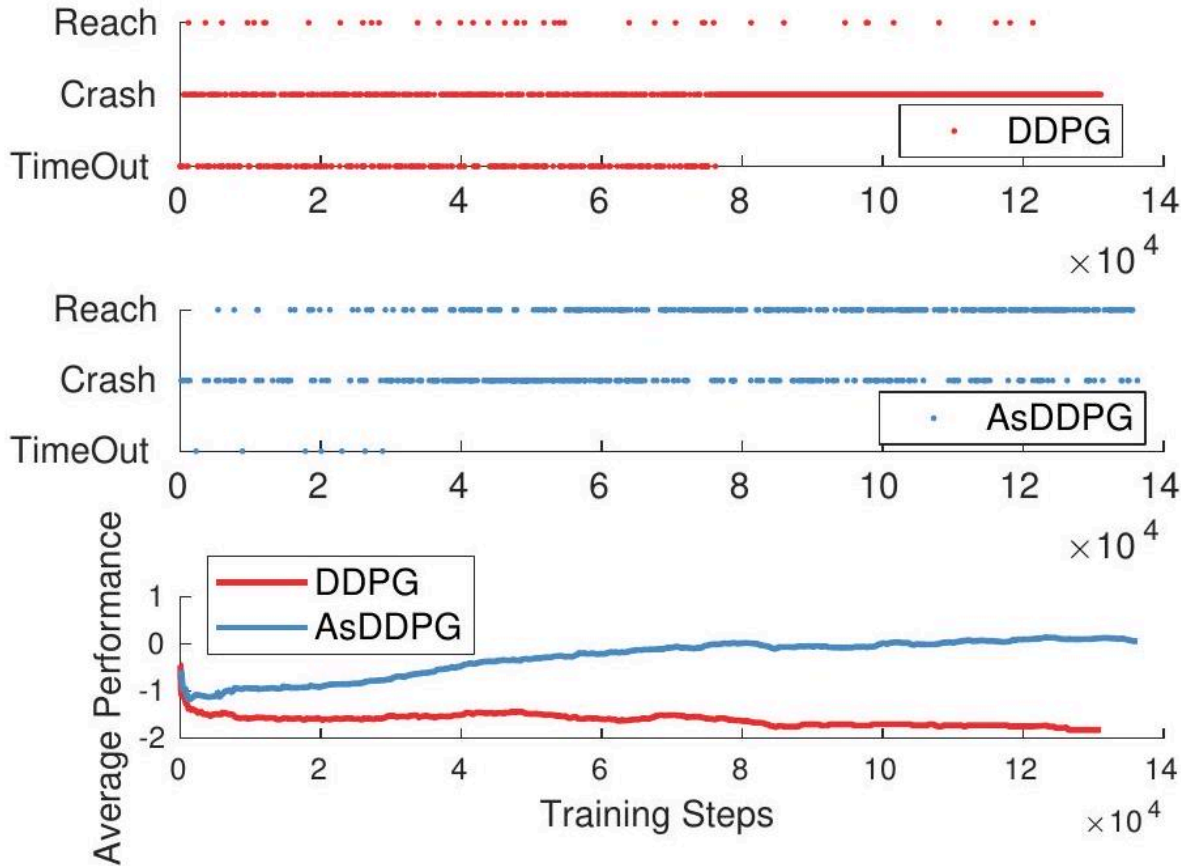


圖 25：圖 4.7：每個回合 (Episode) 的最終結果（到達目標、碰撞與逾時）以及平滑後的學習曲線。

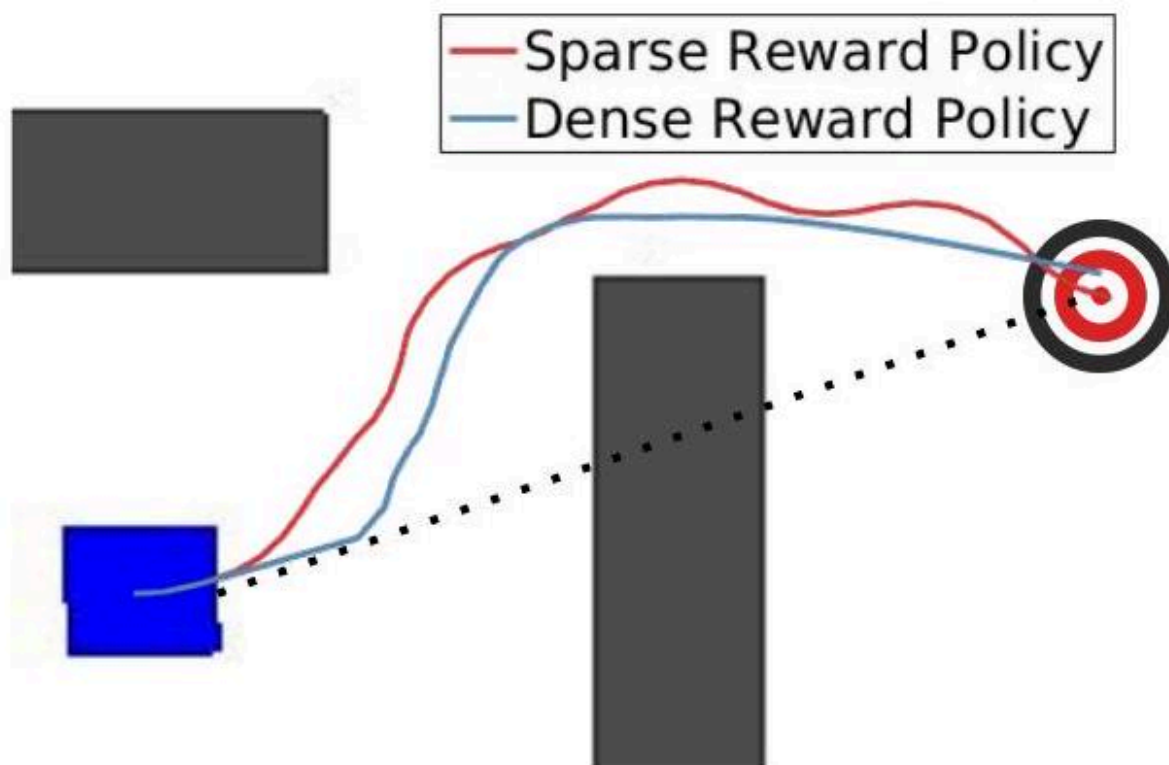


圖 26：圖 4.8：使用密集（dense）與稀疏（sparse）獎勵函數所學習到的策略；其中稀疏獎勵策略花費 43 步（8.6 秒）到達目標，而密集獎勵策略則花費 53 步（10.6 秒）。

一個極度稀疏的獎勵函數，使得隨機探索引導的網路難以學習到良好的策略。

此外，透過使用稀疏獎勵函數，網路學會了在與障礙物保持合理安全距離的同時，以更快的速度駕駛機器人。如圖 4.8 所示，在密集獎勵的情況下，機器人傾向於透過減速並進行平緩的轉向來順滑地繞過障礙物。然而，在稀疏獎勵的情況下，網路學會了更早進行規劃以避開障礙物，並與其保持更大的間距。因此，即使前往目標的路徑不是最短的，機器人仍能以最高速度行駛。

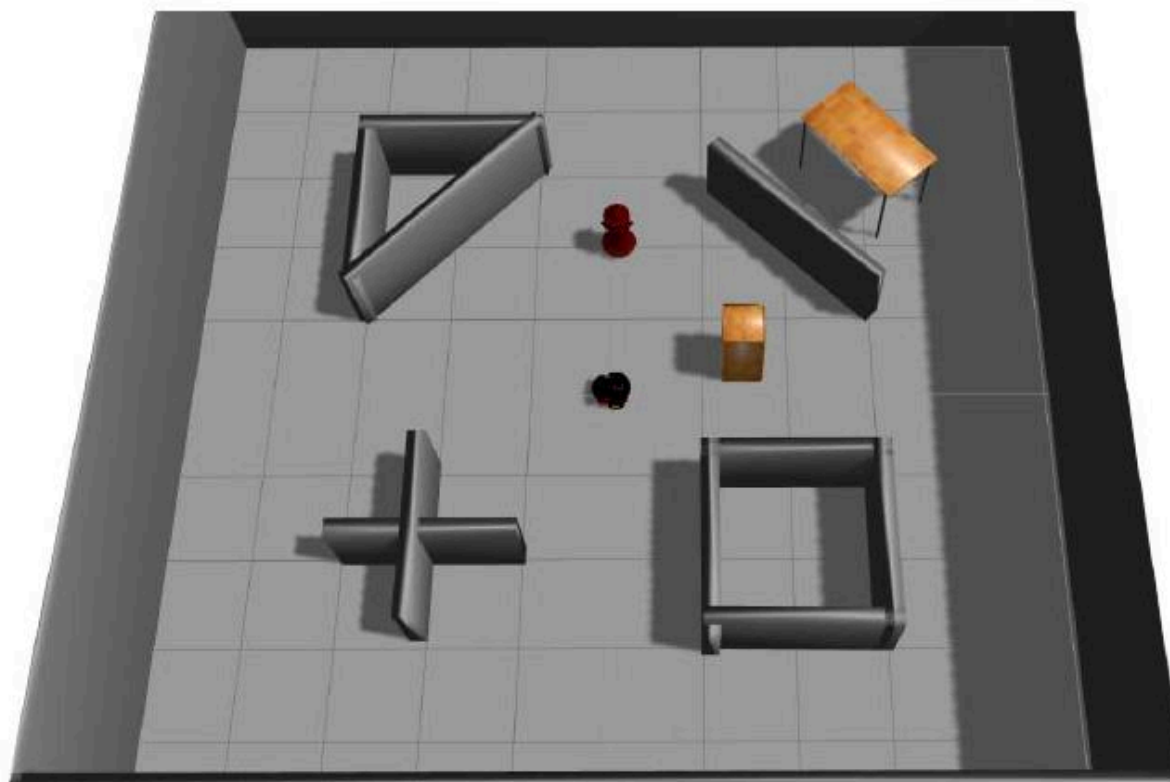


圖 27：圖 4.9：Gazebo

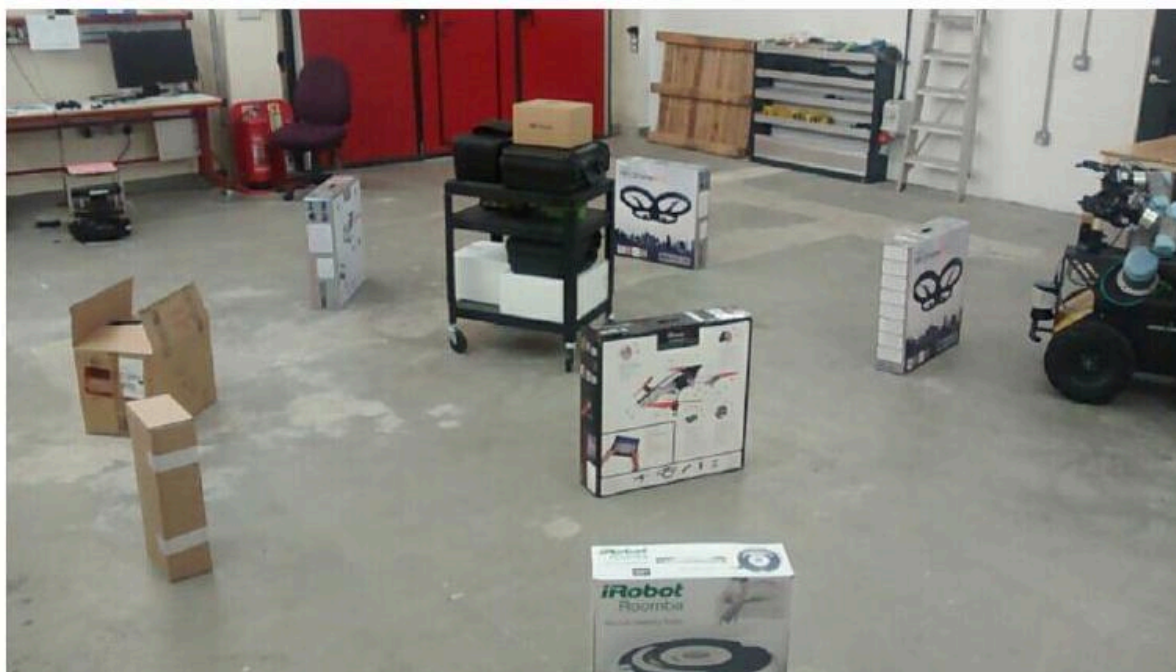


圖 28：圖 4.10：真實世界

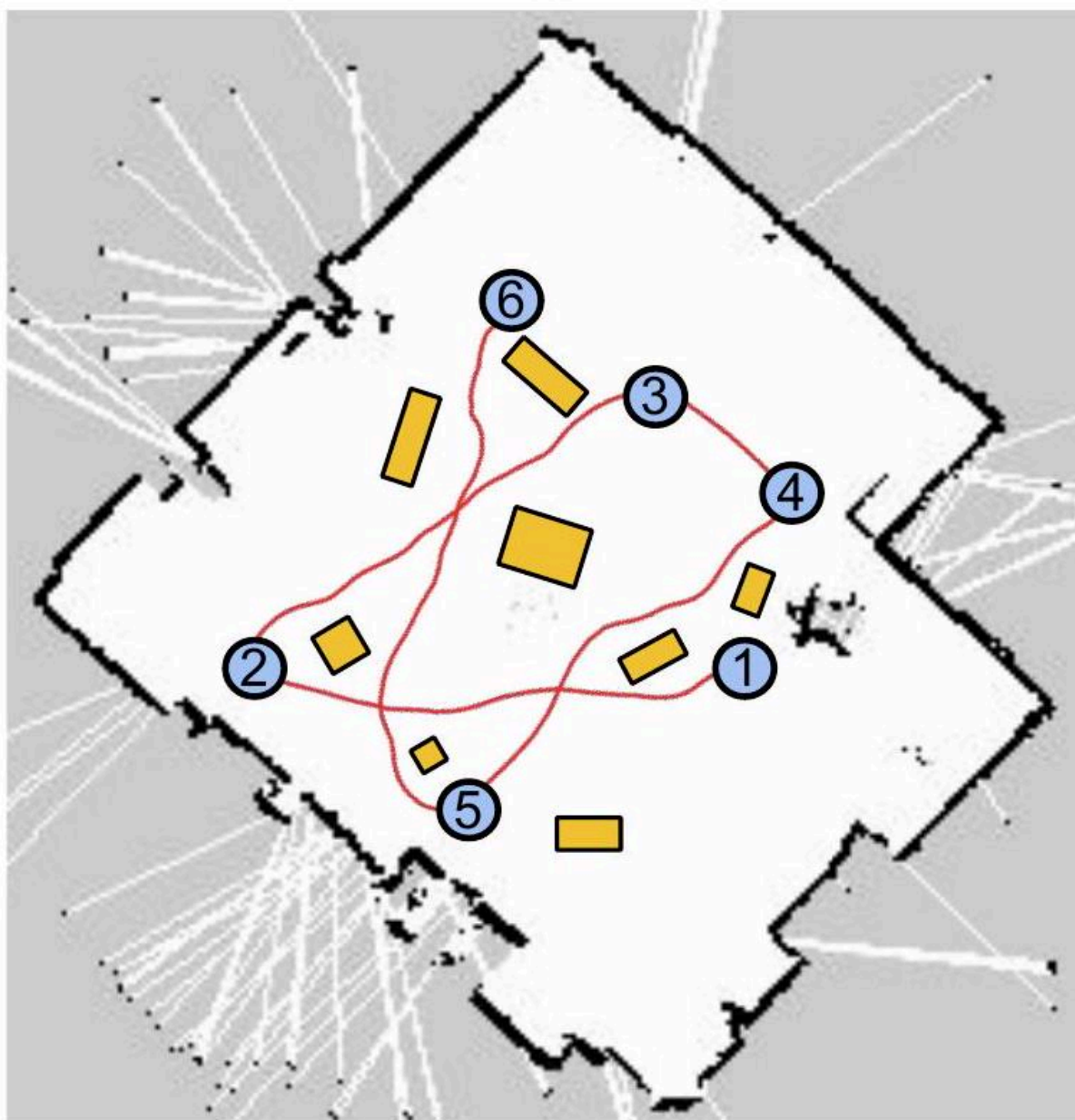


圖 29：圖 4.11：機器人在真實世界實驗中的軌跡。黃色矩形為障礙物，藍色圓圈表示序列目標點。

4.5.4 真實世界測試

在真實世界的實驗中，我們使用了一台配備 Hokuyo 雷射掃描儀的 Pioneer 機器人。為了根據現有地圖對機器人進行定位，我們應用了 AMCL ROS 套件。由於我們的網路僅作為局部規劃器 (local planner)，負責在無碰撞的情況下抵達近處目標，因此它必須與全域規劃器 (global planner) 結合，以構建完整的導航系統。具體而言，在接收到目的地後，全域規劃器會生成一條通往目標的路徑，路徑上的每個點都會連同雷射掃描數據和機器人速度，一起轉換到機器人的局部座標系中作為網路輸入。

圖 4.9 所示的 Gazebo 模擬環境被用於訓練網路。接著，該網路直接在真實世界場景中，透過具有多個障礙物的真實機器人進行測試 (圖 4.10)。

障礙物 (圖 4.10)。在此實驗中，我們建立了一張室內沒有任何障礙物的地圖，並由學習到的策略驅動機器人，在避開障礙物的同時接連抵達多個目標點。圖 4.11 展示了疊加在地圖上的機器人軌跡與障礙物。機器人的軌跡以紅色曲線繪製。實驗結果顯示，機器人能夠平滑地避開所有障礙物，並成功抵達每個目標。

4.6 SGuidance: 利用隨機引導進行學習

在研究了學習確定性切換策略 (deterministic switching policy) 之後，我們現在轉向隨機性 (stochastic) 的對應方案。其核心見解在於，當學習確定性切換策略時，必須手動調整並引入雜訊以探索更好的切換策略，但這很難在探索 (exploration) 與利用 (exploitation) 之間達到最佳平衡。相比之下，學習隨機切換策略可以從根本上解決這個問題，並讓 DDPG 最終收斂到更好的策略。

具體而言，與 AsDDPG 不同，SGuidance 中的切換機制被建構為一個隨機函數，並透過 REINFORCE 學習訊號 [164] 進行更新，以實現總獎勵最大化。同時，DDPG 組件是透過採用隨機切換器所選定的動作來進行學習，而非直接使用其策略網路產生的輸出動作。因此，切換機制能幫助 DDPG 在訓練早期階段避免無意義的探索，並學習在探索與啟發式引導之間取得平衡。更有趣的是，一旦訓練完成，DDPG 組件可以脫離其他控制器獨立使用，在這種情況下，切換器會被關閉，導航完全由 DDPG 組件執行。類似於模仿學習 (imitation learning) [116, 115] 的概念，DDPG 組件能夠從引導組件 (在我們的案例中為 PID 和 OA 避障組件) 提供的示範中學習，並立即泛化到新情況 (PID 和 OA 無法處理的情況)。在這裡，引導可以視為減少梯度估計器變異數的一種歸納偏置 (inductive bias)，而模型在從中獲益後，能夠移除這種偏置。

在定量評估方面，我們首先將結合了隨機切換的模型與原始 DDPG 基準以及確定性基準進行比較，以展示透過額外基礎控制器進行引導 (bootstrapping) 所帶來的優勢。接著，我們研究了使用不同獨立控制器的影響，結果顯示該框架具有強大的泛化能力，並且

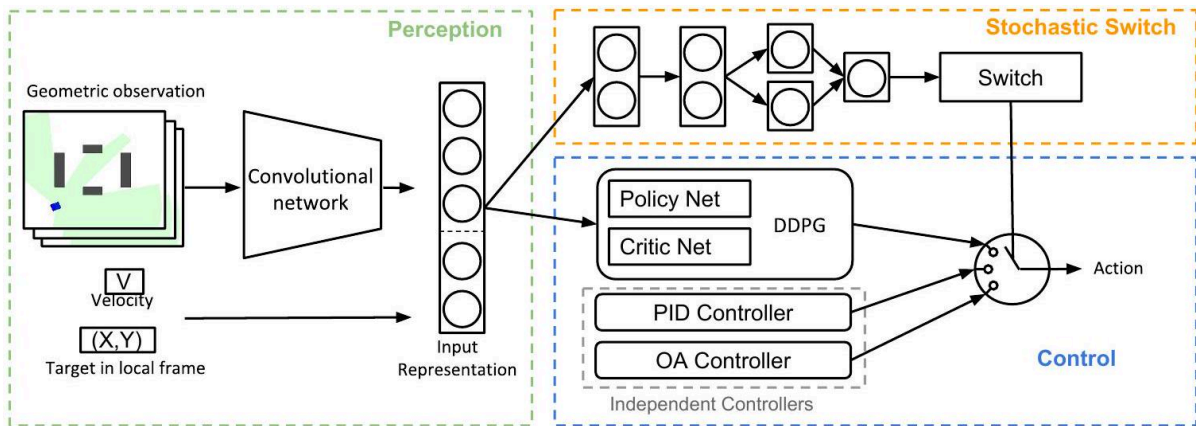


圖 30: 圖 4.12: 所提框架之架構。它由三個部分組成，分別是：感知 (perception)、控制 (control) 與隨機切換 (stochastic switch)。感知部分處理觀測值並生成相應的輸入表示。控制部分包含一個標準的 DDPG 控制器和若干獨立控制器——在此案例中為比例積分微分 (PID) 控制器與避障 (OA) 控制器。隨機切換器則決定應選擇哪個控制器的動作來進行導航。

能夠累積來自不同簡單控制器的優勢。此外，我們提出了三種切換機制變體進行比較，包括均勻隨機切換 (uniformly random switch)、argmax 切換 (類似於 AsDDPG) 以及湯普森採樣切換 (Thompson sampling switch)。最後，我們展示了當額外控制器的使用率下降到閾值以下時，模型可以捨棄這些控制器，並能僅透過 DDPG 組件繼續進行自我學習。在定性評估方面，我們在真實世界場景中測試了我們的模型：無需進一步修改，在模擬環境中訓練的模型即可直接轉移並執行導航任務。

4.7 模型

所提出的模型由三個部分組成：感知 (perception)、控制 (control) 和隨機切換 (stochastic switch)，如圖 4.12 所示。在每個時間步，感知部分處理觀測值並生成相應的輸入表示。接著，不同的控制器可以根據該輸入表示提出候選動作。最後，隨機切換機制決定執行哪一個動作。由於感知部分和 DDPG 組件與 AsDDPG 中的相同，我們僅介紹 SGuidance 中的啟發式控制器 (heuristic controllers) 和隨機切換機制。

4.7.1 啟發式控制器

與 AsDDPG 不同，在本節中，我們利用兩個獨立的控制器來輔助 DDPG 學習合理的策略。一個是 4.4.1 節中介紹的 P 控制器。另一個則是簡單的避障 (OA) 演算法，它可以驅動機器人而不發生碰撞。它利用幾何觀測資訊，透過控制機器人的朝向（旋轉速度）來偵測並避開附近的障礙物：

$$|a^\omega| = \begin{cases} a_{\max}^\omega \cdot \frac{|d_o - \beta|}{\beta}, & d_o < \beta \\ 0, & \text{otherwise} . \end{cases}$$

其中 d_o 是到最近障礙物的距離， $a_{\max}^\omega$ 代表最大旋轉速度，而 β 表示預定義的最小安全距離。在機器人與物體之間的距離小於安全距離（即 $d_o < \beta$ ）的情況下，機器人將會旋轉以避免碰撞。這兩個控制器互為補充，為隨機切換 (stochastic switch) 提供候選動作。請注意，OA 僅產生 a^ω ，而選定的 a^v 則由 DDPG 控制器提供。值得注意的是，這只是兩個簡單的範例控制器，未來有可能加入更多數量或更複雜的控制器。

4.7.2 隨機切換

PID 控制器、OA 演算法和 DDPG 是三個獨立的來源，為切換網路提供候選動作以進行（最佳化）選擇。切換網路是一個隨機深層神經網路，由參數化網路和多項式分佈 (multinomial distribution) 組成。切換機制會根據預期獎勵並以輸入表示 (input representation) 為條件，有效地學習該選擇哪一個控制器。我們將其視為另一個 MDP，其中學習代理人 (learning agent) 所採取的動作 s_t 是在每個時間步長從三個控制器中進行選擇。此切換策略與其他深層學習組件（例如 DDPG 和選用的輸入表示網路）進行端到端 (end-to-end) 訓練。

4.7.2.1 折棍法 (Stick-breaking)

傳統上，可以使用 softmax 層來提供多項分佈 (multinomial distribution) 的參數 θ 。在此，我們改為採用折棍法結構 (stick-breaking construction) [134, 59, 93]，這是 softmax 的另一種替代方案。

其直覺在於引入一種偏好 (bias)，以鼓勵更多地使用深度強化學習演算法（在我們的案例中為 DDPG）。由於我們的框架旨在訓練一個能從隨機引導中獲益的強健 DDPG 元件，我們希望它在該框架中比其他元件更頻繁地被使用，以便在經過一段時間的訓練後，能夠捨棄簡單的獨立控制器。它基本上將多項機率參數的建模，轉換為二項機率參數之對數機率 (logits) 的建模。

我們首先將 $\alpha = f_s(x_t | \theta^s)$ 定義為切換網路 (switching network) $f_s(\cdot | \theta^s)$ 在給定輸入表示 x_t 下，由全連接層輸出的未縮放 Logits，並將 θ^s 定義為切換網路的參數。接著，二項式 Logits η 可透過 $\eta = \text{sigmoid}(\alpha)$ 獲得。請注意，由於在我們的案例中有 3 個控制器，因此 $\eta = [\eta_1, \eta_2]^T$ 與 $\alpha = [\alpha_1, \alpha_2]^T$ 皆為二維向量。

因此，多項機率參數 $\xi = [\xi_1, \xi_2, \xi_3]$ 可透過碎棍函數 (stick-breaking function) $f_{\text{SB}}(\eta)$ 經由兩次切割產生：

$$\begin{aligned} \xi_1 &= \eta_1 \\ \xi_2 &= \eta_2 (1 - \eta_1) \\ \xi_3 &= (1 - \eta_2) (1 - \eta_1) \end{aligned}$$

其中，若存在 K 個控制器， f_{SB} 可被泛化至更多次的切割 $\xi_k = \eta_k \prod_{i=1}^{k-1} (1 - \eta_i) (1 < k < K)$ 。

在以當前觀測值 x_t 為條件的情況下，我們能夠建構隨機切換策略 (stochastic switch policy) $s_t \sim \pi^s(s_t | x_t; \theta^s)$ 如下：

$$\begin{aligned} \xi &= f_{\text{SB}}(\text{sigmoid}(f_s(x_t | \theta^s))) \\ s_t &\sim \text{multinomial}(\xi) \end{aligned}$$

在每個時間步 t ，隨機開關會取樣一個決策 s_t ，而 ξ_1, ξ_2, ξ_3 分別對應 DDPG、PID 與 OA。接著，根據決策 s_t ，DDPG 的 Critic 網路將最終動作 $a_{s_t} \in \{a_{\text{DDPG}}, a_{\text{PID}}, a_{\text{OA}}\}$ 作為輸入，並據此更新網路。同時，隨機開關透過 REINFORCE 學習訊號進行更新，使開關網路能夠透過觀察環境，動態選擇透過探索 (DDPG) 來學習，或是選擇使用啟發式控制器 (PID 或 OA) 的輸出來作為引導。

4.7.2.2 帶有基準值 (Baselines) 的 REINFORCE 演算法

由於梯度無法直接透過離散樣本進行反向傳播，我們採用 REINFORCE 演算法 [164] 為開關網路建構梯度估計器。其理論已在 2.1.6.2 節中完整說明，在此我們僅註明梯度是根據最近一次的訓練回合 (Episode)，透過以下方程式計算得出：

$$\sum_{t=0}^T \nabla_{\theta^\pi} \log \pi(s_t | x_t, \theta^\pi) (R(x_t) - b^c - b(x_t))$$

其中 $R(x_t)$ 是基於深度神經網路的總未來獎勵估計器，而 x_t, b^c 與 $b(x_t)$ 則是學習到的輸入無關 (input independent) 與輸入相關 (input dependent) 基準值 (baselines)，兩者皆使用全連接層進行建模。

隨機切換 (stochastic switch) 的引入可被視為一種歸納偏置 (inductive bias)，旨在學習如何透過更好的動作樣本進行導航。經由 REINFORCE 更新後，隨機切換能夠感知環境、避免無意義的探索，並為學習 DDPG 策略選擇更佳的动作。此外，由於獨立控制器是透過隨機切換整合的，啟發式方法所引入偏置的負面影響也因此受到限制。一旦 DDPG 訓練充分，即可明確禁用獨立控制器，儘管這種情況在很大程度上會自然發生。

4.7.3 演算法

演算法 2 展示了結合隨機引導 (SGuidance) 訓練 DDPG 的簡要演算法。由於 DDPG 是一種離線策略 (off-policy) 方法，因此應用經驗回放暫存區 (replay buffer) R 來儲存所有轉移過程 (x_t, a_t, r_t, x_{t+1}) 。在每個訓練步驟中會採樣一批轉移資料來更新 DDPG。相比之下，使用 REINFORCE 學習切換策略是一個在線策略 (on-policy) 過程，因此我們僅儲存目前的切換軌跡 (x_t, s_t, r_t) (其中 $t = 1, 2 \dots T$)，以便在每個回合 (episode) 結束時計算梯度，進而更新包含控制變量 (control variate) 在內的切換網路。

4.8 SGuidance 實驗

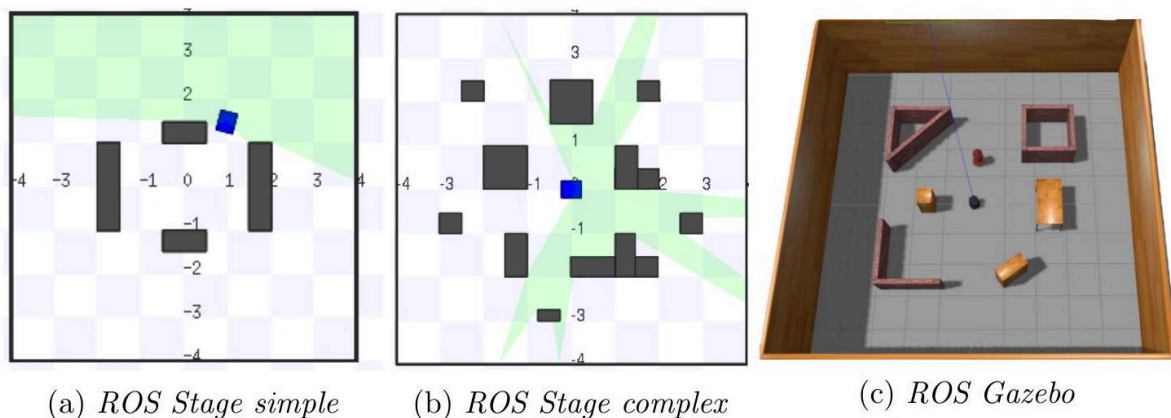
我們進行了模擬與真實世界的實驗，以驗證所提出的架構。在本節中，我們在模擬和真實環境中對模型進行實驗。我們在模擬器中定量評估訓練效率與性能，並與基準模型 (baseline models) 進行比較。接著，我們在真實世界的機器人導航任務中對模型進行定性測試。

Algorithm 2 SGuidance



procedure Training

```
Initialise switching network  $(f_s \leftarrow (x_t \mid \theta^s))$  and  $b^c$ 
Initialise DDPG network  $(Q \leftarrow (x, a \mid \theta^Q))$  and  $(\pi \leftarrow (x \mid \theta^\pi))$ 
Initialise the target network of DDPG.
Initialise replay buffer  $R$  and exploration noise  $(\epsilon)$ .
for episode  $(=1, \dots, M)$  do
    Reset the environment.
    Initialise replay buffer  $(R_s)$ .
    Obtain the initial observation  $(x_t)$ .
    for step  $(=1, \dots, T)$  do
        Sample switch  $(s_t = f_s(x_t \mid \theta^s) \in \{0, 1\})$ .
        Sample  $(a_t^0 = \pi(x_t \mid \theta^\pi) + \epsilon)$ .
        Get  $(a_t^1)$  from PID controller.
        Get  $(a_t^2)$  from OA controller.
        Execute  $(a_t = a_t^{s_t})$  and obtain  $(r_t, x_{t+1})$ .
        Store transition  $(x_t, a_t, r_t, x_{t+1})$  in  $(R)$ .
        Sample a batch of transitions from  $(R)$ .
        Update the actor and critic network of DDPG.
        Update the target network of DDPG.
        Store switching trajectory  $(x_t, s_t, r_t)$  in  $(R_s)$ .
    end for
    Update switching network with trajectory in  $(R_s)$ .
end for
end procedure
```



(a) *ROS Stage simple*

(b) *ROS Stage complex*

(c) *ROS Gazebo*

圖 31: 圖 4.13: (a) 四個灰色矩形為障礙物，藍色正方形代表機器人。機器人上安裝了稀疏雷射 (sparse laser)，其偵測區域以綠色區域表示。(b) 顯示由 ROS Stage 模擬的更複雜環境。(c) ROS Gazebo 也用於訓練可遷移至真實世界環境的模型。Turtlebot 2 (地面機器人平台) 被用作配備深度攝影機的移動平台。

4.8.1 訓練環境與設定

所提出的框架在兩個不同的模擬器中進行訓練。第一個是輕量級模擬器 *ROS Stage*³ (圖 4.13a 與圖 4.13b)，在此進行了大量重複實驗，用以展示學習曲線、證明隨機引導 (stochastic guidance) 帶來的改進，並與其他基準模型進行比較。在此模擬器中，我們在行動機器人上安裝了雷射掃描儀以提供周圍環境的幾何資訊。因此，在這種情況下不使用卷積神經網路 (圖 4.12)，而是將雷射掃描結果直接與其他觀測值串接，作為輸入表示。透過加速模擬時間，我們在 *ROS Stage* 中經由大量重複實驗獲得了定量評估。請注意，圖 4.13a 所示的簡單環境是我們的預設訓練環境，而更複雜環境中的實驗則在第 4.8.2.6 節中進行。

另一個模擬器 *ROS Gazebo*⁴ (圖 4.13c) 包含物理引擎，可以精確模擬行動機器人的動力學。因此，在 *ROS Gazebo* 中訓練的模型可直接應用於現實世界場景，以定性評估導航性能，但與 *ROS Stage* 相比，其運算開銷較大。在此，我們利用深度圖像來觀察周圍環境，因此構建了一個 3 層的

卷積網路 (濾波器分別為 [4,4,3,8]、[4,4,8,16] 與 [4,4,16,32])，以提供基於深度圖像的輸入表示。

在每個訓練回合 (episode) 中，機器人從原點 (中心點) 出發，具有隨機的朝向，而目的地則隨機設定在障礙物以外的區域。當機器人撞到障礙物或到達目的地時，當前回合即終止。動作控制頻率為 5 Hz (每兩個連續時間步之間有 0.2 秒延遲)，切換頻率為 1 Hz。對於在 *ROS Stage* 中進行的所有實驗，訓練過程持續 10 萬步，並重複 5 次。圖中展示了平均學習曲線以及變異數⁵，用以說明效能表現。

關於超參數 (hyper-parameters)，批評者 (critic) 與行動者 (actor) 網路的隱藏層各包含 100 個 ReLU 單元，而行動者網路的輸出層則分別針對角速度與線速度應用 tanh 與 sigmoid 函數。在更新 DDPG 參數時，會從基於排名的優先經驗回放 (rank based prioritised experience replay) [127] 中隨機抽取 32 個學習樣本作為訓練批次 (batch)，行動者網路、批評者網路以及隨機切換器 (stochastic switch) 的學習率分別為 10^{-4} 、 10^{-3} 與 10^{-3} ，其餘設定則遵循 [80]。

4.8.2 模擬環境中的導航

在此，我們討論在 *ROS Stage* 中進行定量評估的實驗。應用隨機切換器與 DDPG 的預設設定，是使用 REINFORCE 演算法與控制變量 (control variates, CV)，並同時接受來自 PID 與 OA 控制器的引導來進行學習。

4.8.2.1 結合隨機引導的強化學習

圖 4.14 比較了所提出的模型與競爭模型，並展示了透過隨機切換 (stochastic switch) 進行學習所帶來的優勢。SGuidance 是我們提出的模型，具備隨機切換機制，能從 DDPG、PID 和 OA 控制器所提出的候選動作中動態選擇動作。如圖所示，SGuidance 的效能顯著優於 DDPG 基準模型。同時，根據學習曲線周圍寬廣的透明區域顯示，DDPG 存在高變異數 (high variance) 的問題，而 SGuidance 則穩定得多。這是由於環境的高複雜度導致 DDPG 提供的學習樣本具有高度變異性，進而可能導致無效的探索。此外，隨機梯度

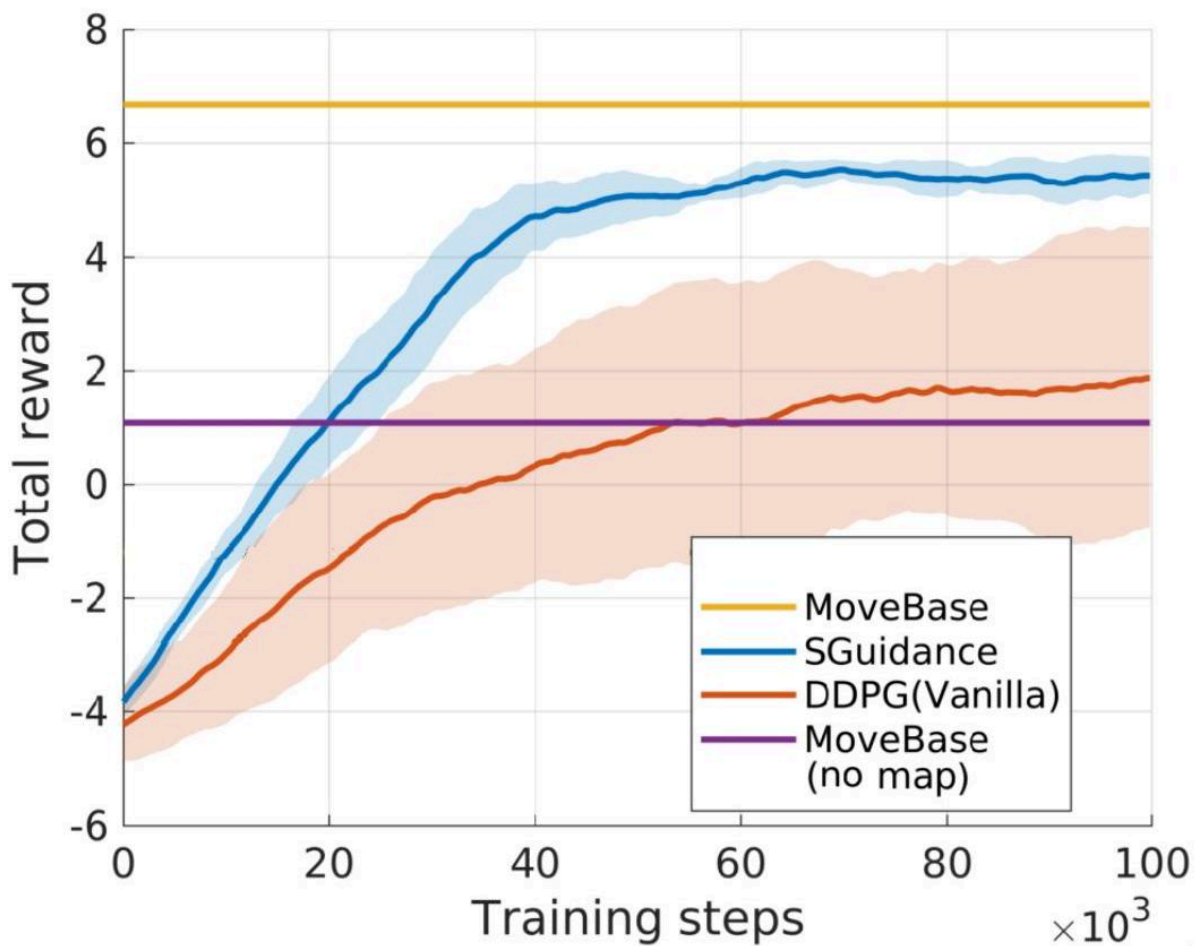


圖 32：圖 4.14：我們的模型與其他基準模型在比較下的總獎勵。曲線代表 5 次重複訓練過程的平均值，透明區域則表示結果的變異數。

DDPG 的估計器採用了有偏近似 (biased approximation)，這使得難以保證收斂性與穩定性。相比之下，SGuidance 能夠在訓練程序的早期階段受益於啟發式的簡單控制器，而不是從完全隨機的動作開始。

在此實驗中，我們也繪製了 MoveBase (no map) 與完整 MoveBase 的獎勵以進行比較。請注意，MoveBase 套件已在第 4.5.1 節中介紹，它能夠在擁有環境全域地圖的情況下，生成前往目的地的最佳動作。因此，我們在此實驗中將完整的 MoveBase 視為一個先知控制器 (oracle controller)，設定了效能的上限，而 MoveBase (no map) 則作為一個公平比較的基準模型。關於 MoveBase (no map) 的具體設定已在第 4.5.1 節中描述。如圖 4.14 所示，DDPG 能夠獲得與 MoveBase 相當的效能。然而，SGuidance 顯著超越了確定性的 MoveBase 模型。即使在無法存取全域地圖的情況下，SGuidance 僅透過使用幾何資訊，就展現出強大的環境導航能力。基本上，簡單的確定性控制器無法獨立應用於執行導航任務（其累積獎勵均低於 0）。然而，當透過隨機切換機制與 DDPG 結合時，它們對於緩解原生 DDPG 所面臨的高變異數問題做出了重大貢獻。

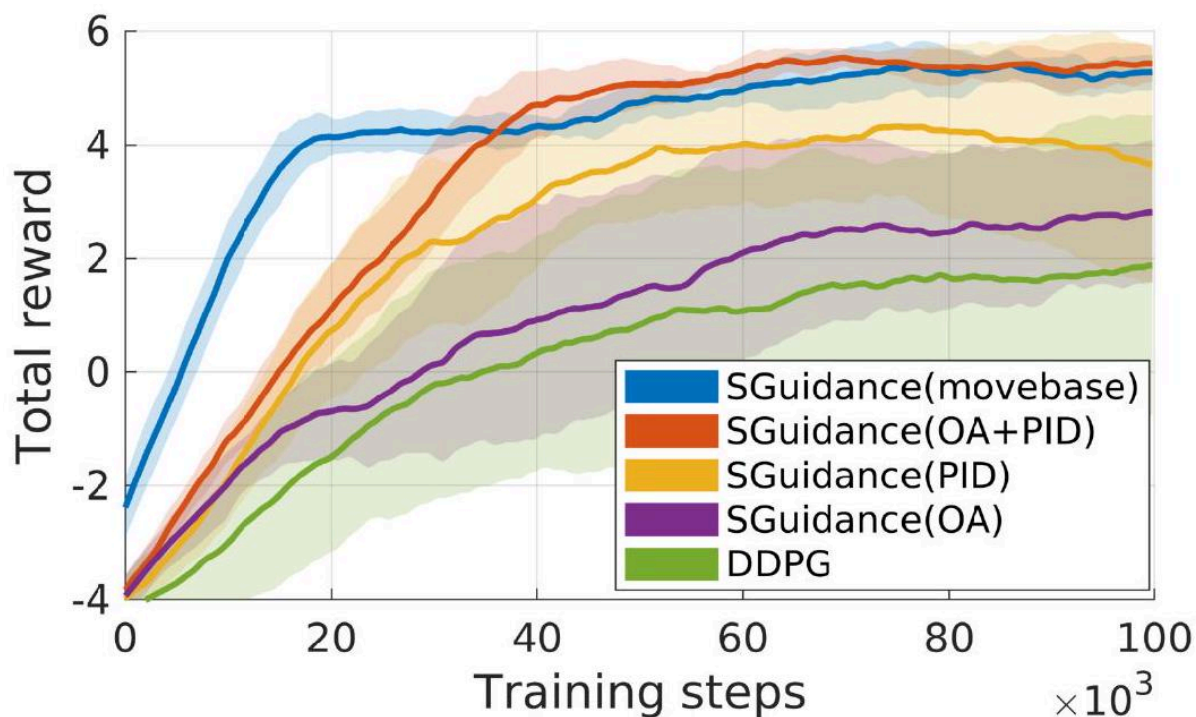


圖 33: 圖 4.15: 將不同的啟發式控制器與 DDPG 結合後獲得的平滑總獎勵。SGuidance (PID+OA) 同時利用了 PID 與 OA 控制器，而 SGuidance (OA) 和 SGuidance (PID) 則分別僅採用其中之一。SGuidance (movebase) 由 movebase 引導，而 DDPG (Vanilla) 則是沒有啟發式引導的基準 DDPG。

4.8.2.2 使用不同的獨立控制器

本實驗展示了不同獨立控制器對訓練效能的影響。

如圖 4.15 所示，與僅使用 PID 或 OA 的 DDPG 以及不含任何獨立控制器的 DDPG 相比，SGuidance (PID + OA) 取得了最佳效能。這證明了透過加入更多控制器，隨機切換 (stochastic switch) 的貢獻得到了極大增強，從而產生更穩定的學習曲線和更好的導航效能。有趣的是，在此情境下，PID 控制器比 OA 控制器帶來更多益處，且兩者的優點可以藉由隨機切換進行累積。此外，當 MoveBase 作為獨立控制器與 DDPG 一起使用時，學習速度有明顯提升，特別是在訓練早期階段。這些結果顯示，所提出的框架使我們能夠整合各種不同的控制器，從初階到更複雜的控制器皆可。相較於需要優質示範才能學習的模仿學習 (imitation learning)，這是一個巨大的優勢。

4.8.2.3 使用不同的切換機制

圖 4.16 將隨機切換 (stochastic switch) 與其他切換變體進行了比較，包括 argmax 切換、均勻隨機切換 (uniformly random switch) 以及基於 Thompson Sampling [121] 的切換。

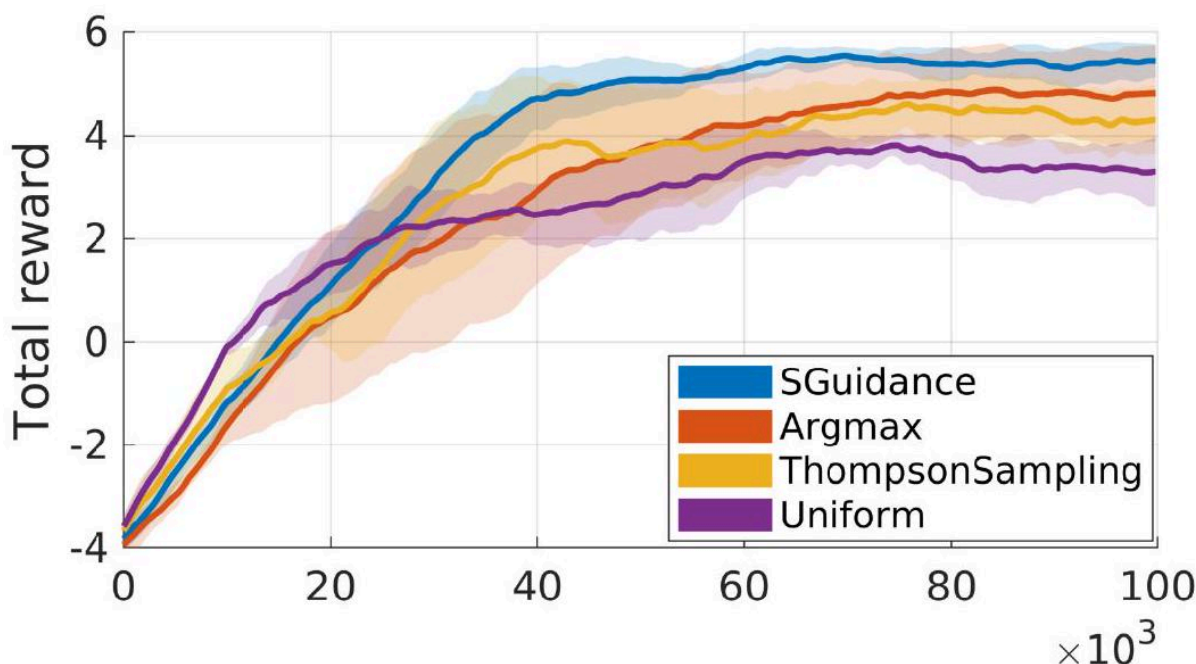


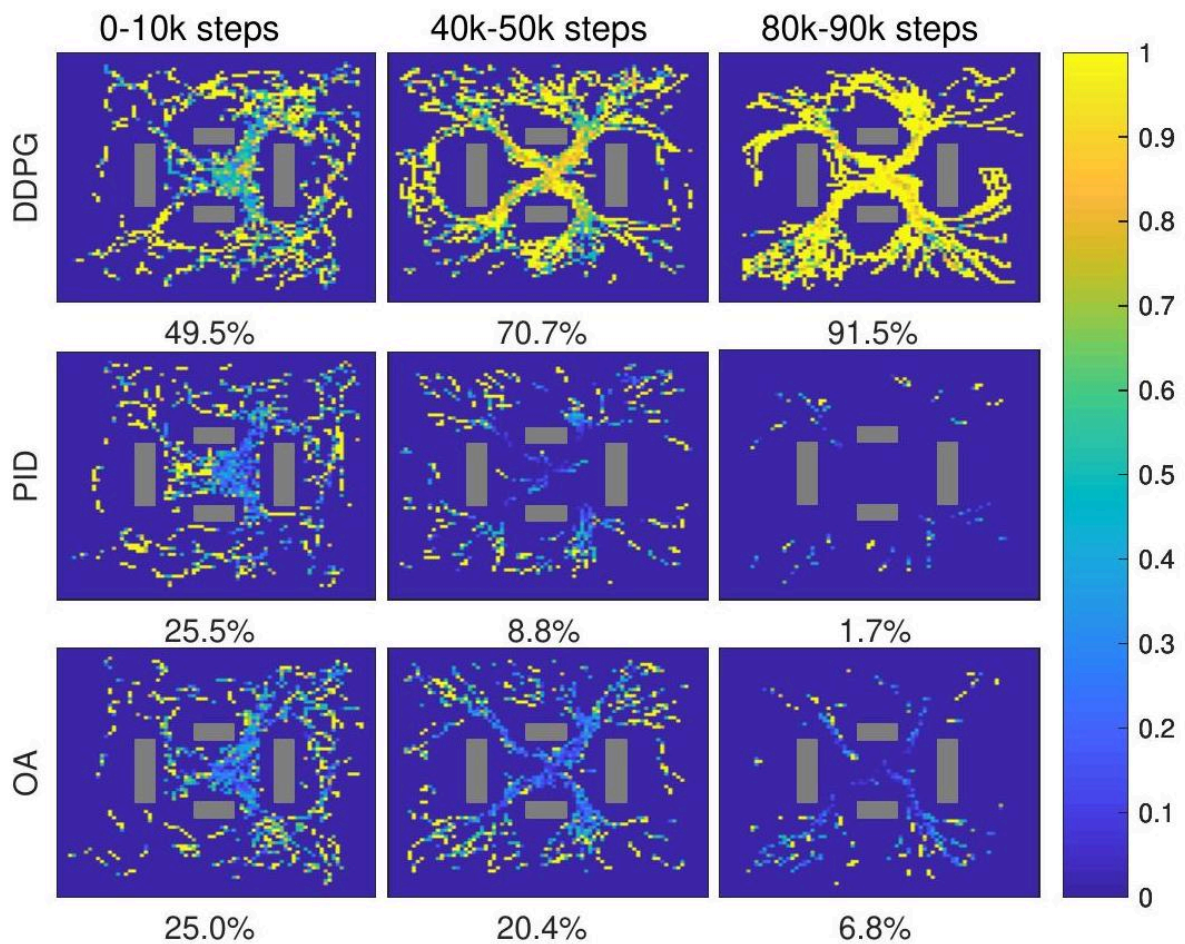
圖 34: 圖 4.16: 使用不同切換機制進行學習所獲得的累積獎勵。Stochastic 代表我們預設的隨機切換設定。Argmax 和 Uniform 分別是所提出的 argmax 切換和均勻分佈切換。ThompsonSampling 是以高斯先驗 (Gaussian prior) 實作的

Thompson Sampling。

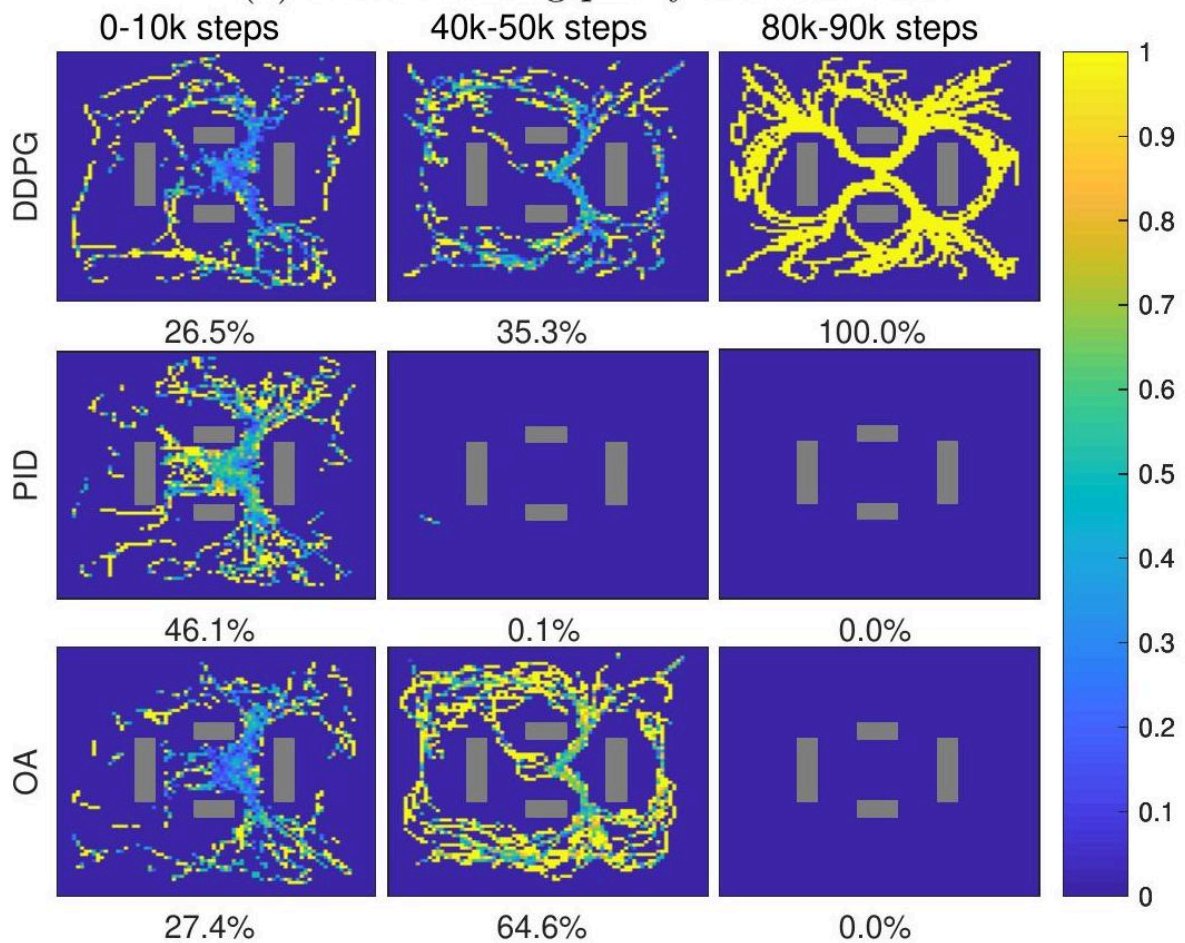
Uniform 切換為 DDPG、PID 和 OA 控制器分配固定的均勻機率，而 argmax 切換則應用偏向的 argmax 輸出，而非從隨機切換網路中隨機抽取樣本。Thompson sampling [121] 是以高斯先驗分佈實作的，並在部署每個控制器時估計總獎勵的後驗機率。如圖 4.16 所示，SGuidance 的效能最佳，而 Uniform 最差。然而請注意，Uniform 實際上在最初的 20k 步中學習速度最快。ThompsonSampling 的效能與 Argmax 相當。前者學習速度稍快，主要是由於其隨機性，並持續尋求潛在更好的切換策略。隨後 Argmax 的表現優於 ThompsonSampling 和 Uniform，因為它是根據當前的觀測值來選擇控制器，而 ThompsonSampling 和 Uniform 則不然。與 SGuidance 相比，Argmax 在總獎勵上的變異數較大。這是因為 Argmax 是一個有偏採樣器（biased sampler），而引入的偏差由於減少了探索，進而損害了其最終效能。

4.8.2.4 與 Thompson Sampling 的進一步比較

為了理解學習情境感知切換策略（situation-aware switching policy）所獲得的優勢，我們比較了訓練期間 SGuidance 與 ThompsonSampling 在不同控制器使用上的情況。圖 4.17a 與圖 4.17b 展示了控制器使用的空間密度，其中每一列代表單一控制器的使用情形，每一欄則表示訓練的初期（10k-20k 步）、中期（50k-60k 步）或後期。



(a) The switching policy of SGuidance



(b) The switching policy of Thompson Sampling

圖 35: 圖 4.17: 在三個不同學習階段中, 各個控制器 (DDPG、PID 或 OA) 使用的空間密度 (上方為 SGuidance, 下方為 ThompsonSampling)。每一列代表單一控制器的使用情形, 每一欄則表示訓練的初期 (0-10k 步)、中期 (40k-50k

步) 或後期 (80k-90k 步) 階段。請注意，所有灰色區域皆為障礙物。

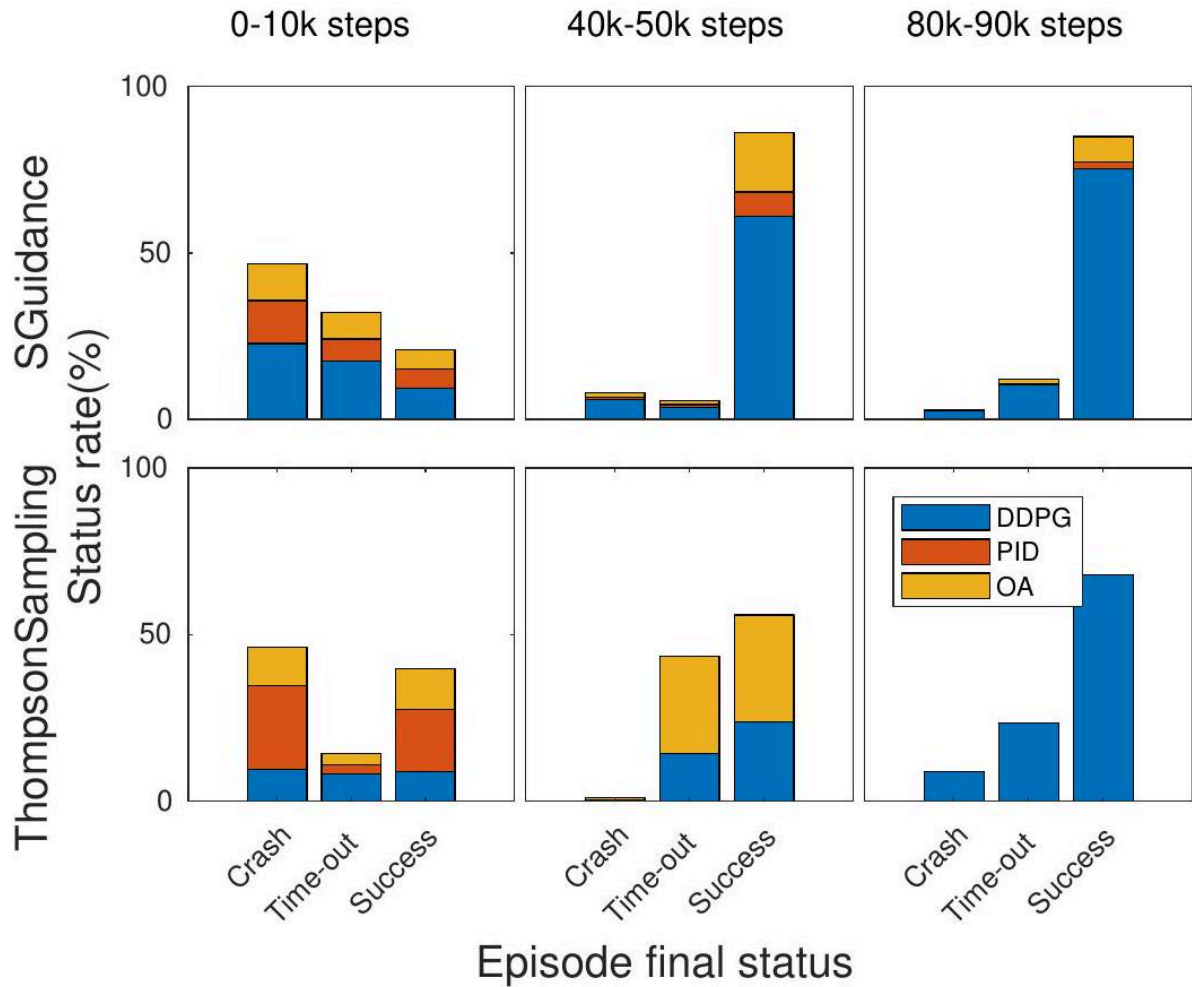


圖 36: 圖 4.18: 分別應用 SGuidance 與 ThompsonSampling 時，在一個回合 (episode) 中達到不同最終狀態之機率的示意圖。每一欄代表一個訓練階段，而每個長條圖中的不同區段則表示該階段內不同控制器的使用比例。

(90k-100k 步) 的訓練週期。請注意，訓練是基於圖 4.13a 中的 ROS Stage 簡單環境，且在每個回合 (episode) 中，機器人皆從中心點初始化，而目的地則是隨機產生的。

圖 4.17a 與 4.17b 顯示了 SGuidance 與 ThompsonSampling 在切換策略 (switching policies) 上的主要差異。SGuidance 會探索各個控制器在不同情境下的優缺點，而 ThompsonSampling 則不然。請注意圖 4.17a 的第二行，當機器人仍被障礙物包圍時，SGuidance 會逐漸捨棄來自 PID 控制器的候選動作，僅在接近目標時才使用它。OA 控制器 (第三行) 則偶爾被採用以避開障礙物。然而，在圖 4.17b 中顯示，各控制器的使用頻率並不取決於其在環境地圖中的位置。事實上，ThompsonSampling 最終完全捨棄了 PID 與 OA 控制器。由於它只能評估控制器在單一回合中的整體表現，在經過長時間訓練後，一旦對 DDPG 的優越性有足夠信心，便會輕易收斂至 DDPG，而未考慮具體情境 (context)。

圖 4.18 繪製了應用 SGuidance 與 ThompsonSampling 時，達到不同最終狀態的機率，以及在單一回合中各控制器的平均使用率。

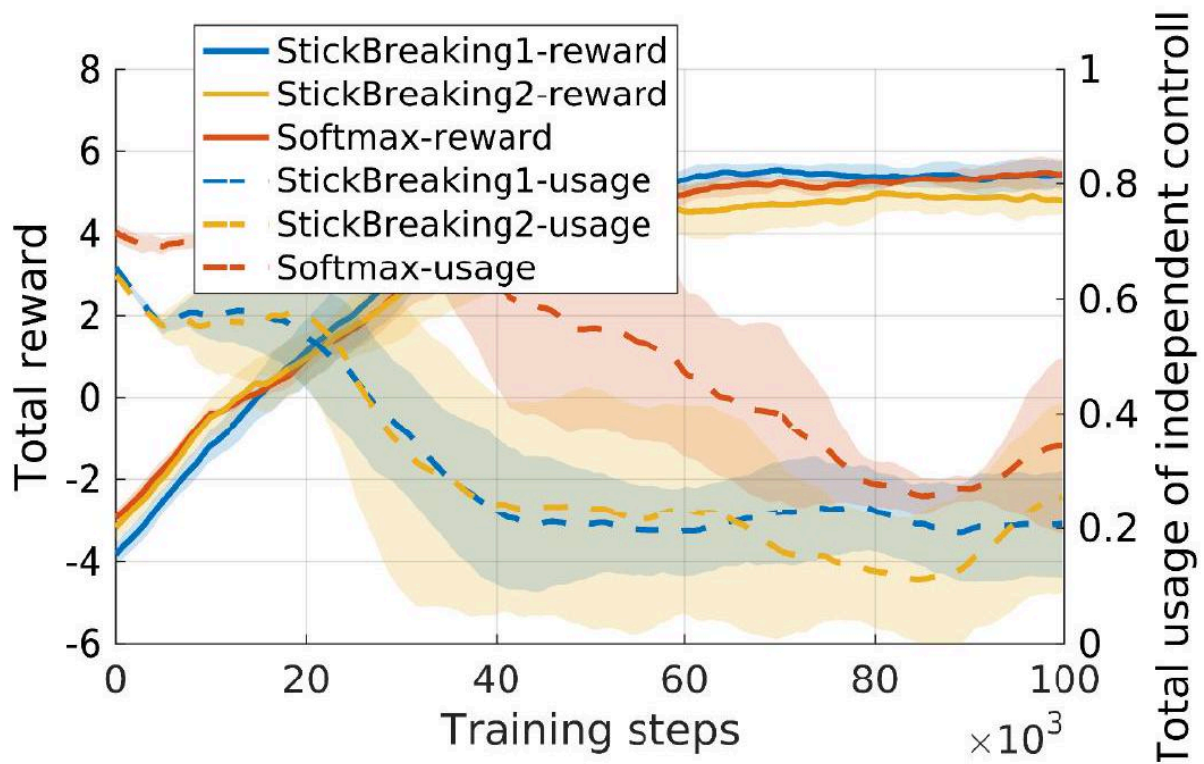


圖 37: 圖 4.19: 使用不同隨機切換函數 (stochastic switch functions) 之影響示意圖。左側 y 軸顯示所有方法的總獎勵 (total reward), 右側 y 軸則顯示各獨立控制器的總使用率。

分別進行採樣。在第一行中，我們可以看到在應用 SGuidance 時，各個控制器在不同類別的訓練回合 (episodes) 中所佔的比例有所不同。它在大多數情況下避免了使用錯誤的控制器，因此幾乎不會在使用 PID 或 OA 控制器時發生碰撞。然而請注意，在使用 ThompsonSampling 時，每個控制器的採樣頻率在不同的最終狀態中是均勻分佈的，因為它僅控制每個控制器的整體採樣率，而不考慮具體情況。

4.8.2.5 隨機切換函數 (Stochastic Switch Function) 的建構

由於我們引入了偏好 (bias)，這可以被視為在使用折棍法 (stick-breaking) 選擇不同控制器時的一種偏好，本部分將研究在折棍法中為 PID 和 OA 控制器設置不同順序的影響。在圖 4.19 中，StickBreaking1 (DDPG, PID, OA) 代表我們在第 4.7.2.1 節中介紹的順序，而 StickBreaking2 (DDPG, OA, PID) 則代表獨立控制器的另一種順序。更具體地說，根據公式 4.8，StickBreaking1 和 StickBreaking2 都將 ξ_1 分配給 DDPG 控制器，並對 PID 和 OA 控制器給予不同的順序，其中 ξ_2 在 StickBreaking1 中分配給 PID 控制器，但在 StickBreaking2 中分配給 OA 控制器。如圖所示，與 StickBreaking1 相比，Softmax 能夠獲得幾乎相同的總獎勵。然而，根據獨立控制器的總使用量，DDPG 組件在 Softmax 中的使用頻率低於 StickBreaking1 和 StickBreaking2。因此，Softmax 函數是建構隨機切換函數的一個安全選擇，它

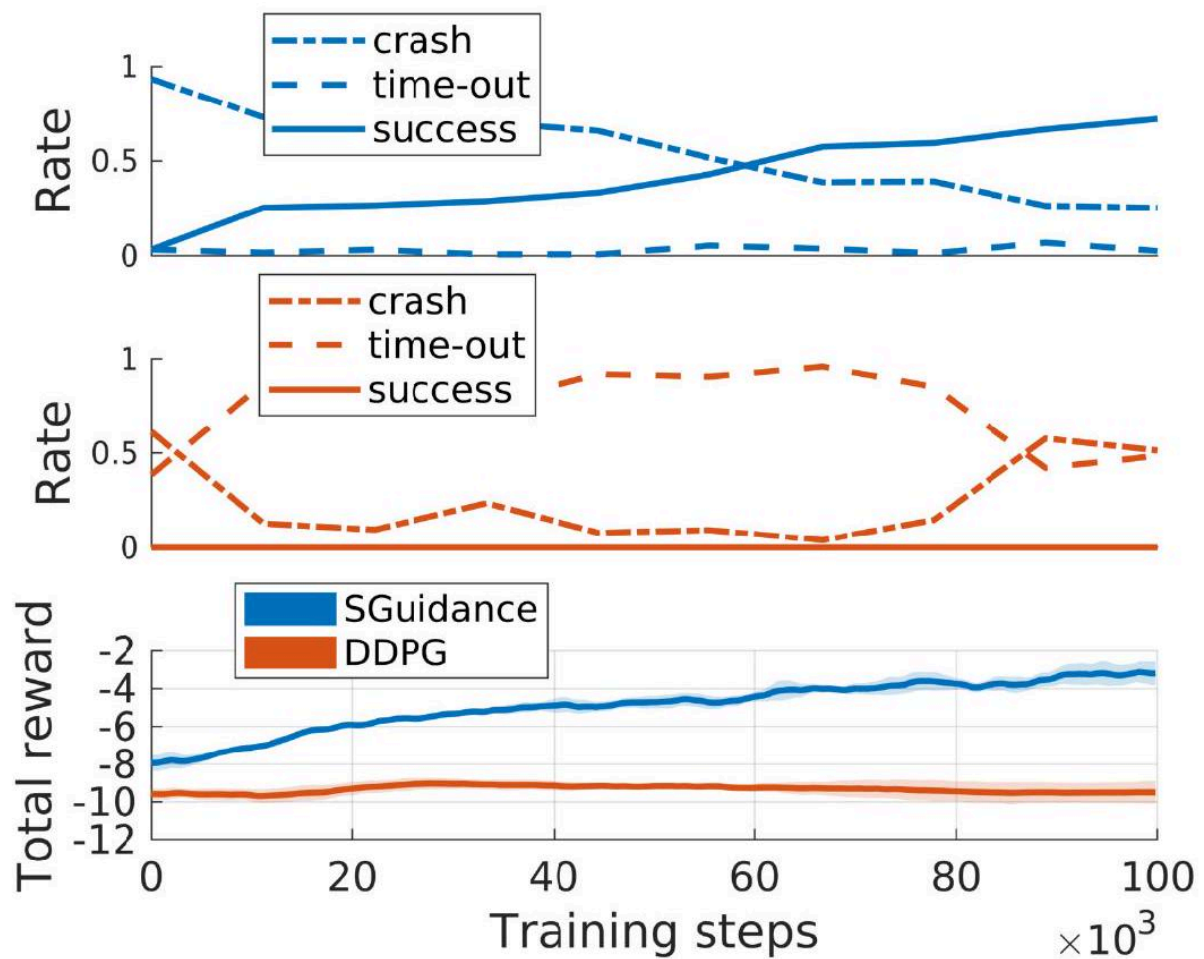


圖 38: 圖 4.20: 在圖 4.13b 所示的具有稀疏獎勵 (sparse rewards) 的複雜環境中的實驗結果。機器人僅在碰撞或到達目的地時獲得獎勵，此外每一步都會受到時間懲罰。前兩行代表在部署引導 (guidance) 與否的情況下，每個導航回合獲得不同結束狀態 (碰撞、超時或成功) 的機率。最後一行也顯示了每個回合的總獎勵。

對所有控制器與 DDPG 策略分配相同的權重。然而，透過 Stick-breaking 過程將更多偏好分配給 DDPG 策略，可以減少對簡單控制器的依賴，並從自我探索中學習更多。如果我們對於簡單控制器的效能有明確的先驗知識 (例如當它們過於簡單而無法處理任務時)，這將會更有幫助。另一方面，儘管兩種 Stick-breaking 函數對獨立控制器的總使用量相似，但 StickBreaking2 的表現略遜於 StickBreaking1，這顯示獨立控制器的順序對效能有微小的影響。

4.8.2.6 複雜環境與簡單獎勵

在本實驗中，我們在如圖 4.13b 所示更為擁擠的環境中測試 SGuidance，機器人必須繞過更多障礙物才能到達隨機分配的目的地。此外，獎勵函數再次被簡化，如公式 4.5 所示，除了每一步的時間懲罰外，機器人僅在碰撞或到達目的地時才會收到獎勵。

圖 4.20 的前兩行分別提供了在有/無隨機引導 (stochastic guidance) 的情況下，訓練期間每個回合 (episode) 獲得不同結束狀態的趨勢。由於隨機引導結合了容易與障礙物碰撞的 PID 控制器，因此幾乎不會以超時 (time-out) 狀態結束回合。隨著 DDPG 控制器效能的增長，SGuidance 到達目的地的頻率也隨之增加。然而，沒有任何引導的原始 DDPG 從未探索出如何安全到達目的地，因此它僅學會了避開障礙物。有趣的是，它隨後在行駛一段短時間後開始發生碰撞，這導致其總獎勵與以超時狀態結束回合的獎勵相似。最後一個子圖展示了有與無隨機引導時相對應的學習曲線。

4.8.2.7 無引導推論

本實驗探討經過隨機引導 (stochastic guidance) 訓練後的 DDPG 策略，在關閉所有啟發式控制器 (heuristic controllers) 的情況下，是否能夠獨立執行導航任務。

我們首先使用 SGuidance 訓練 DDPG，並比較在推論階段開啟或關閉隨機引導的效能表現。該實驗在 ROS Stage 的簡單環境中進行，其獎勵函數定義如等式 4.1 所示。

如表 4.1 所示，在測試階段關閉引導後，整體效能僅有輕微下降。

有趣的是，當我們在 ROS stage 複雜環境中進行相同的測試時，關閉引導甚至提高了成功率 (SR) 並降低了碰撞率 (CR)。這是因為此時與 ThompsonSampling 類似的 SGuidance 會持續探索潛在的更佳選項。然而，這種探索偶爾也會導致錯誤的動作，特別是在訓練後期或測試階段，反而會產生負面效果。這在複雜環境中更為明顯，因為單次錯誤動作就可能導致碰撞。

4.8.2.8 使用不同 γ 的學習情況

在本實驗中，我們研究了不同的獎勵折扣因子 γ 數值如何影響 SGuidance 所學習到的切換策略。如圖 4.21 所示，當 γ 較小時，SGuidance 取樣 DDPG 的頻率較低，因為它變成了一個更趨於貪婪 (greedy) 的控制器，且長期表現較差。因此，當 DDPG 中的策略無法到達目的地時，SGuidance 會偏好選擇 OA 控制器，因為它至少能避免碰撞，即避免產生巨大的瞬時負獎勵。

引導	Poli-U	PID-U	OA-U	SR	TR	CR	AR
在具有密集獎勵的簡單環境中進行測試							Reward
保持	88.1%	3.5%	8.4%	92%	7%	2%	5.01
關閉	100%	0%	0%	90%	4%	6%	4.91
在具有稀疏獎勵的複雜世界中進行測試							
保留	81.7%	2.5%	15.8%	81%	14%	5%	-2.29
關閉	100%	0%	0%	83%	16%	1%	-2.34

表 1：表 4.1：比較訓練後保留或關閉引導 (guidance) 的效能表現，包含三種控制器的使用率 (Poli-U、PID-U、OA-U)、每個回合 (episode) 最終狀態的比例 (SR：成功率、TR：超時率、CR：碰撞率) 以及每個回合的平均總獎勵 (AR)。

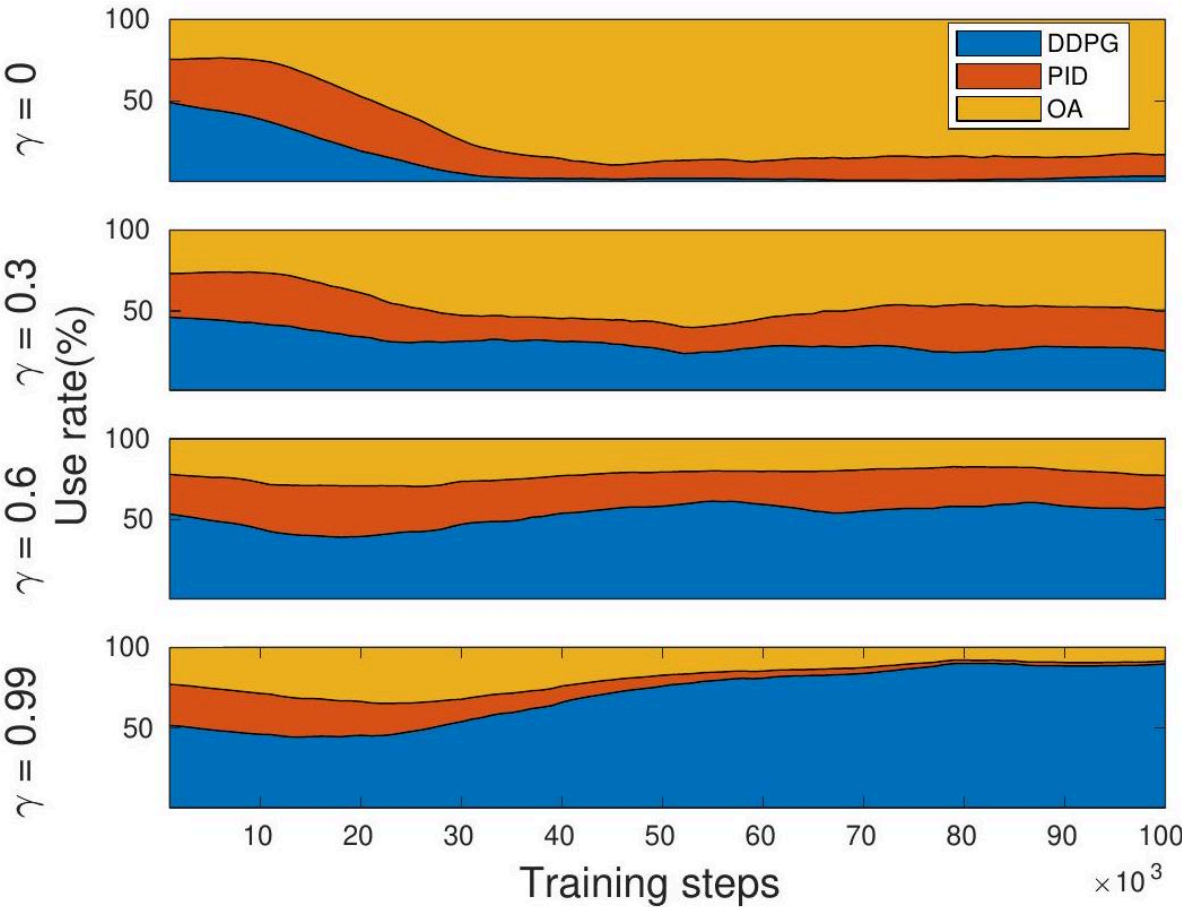
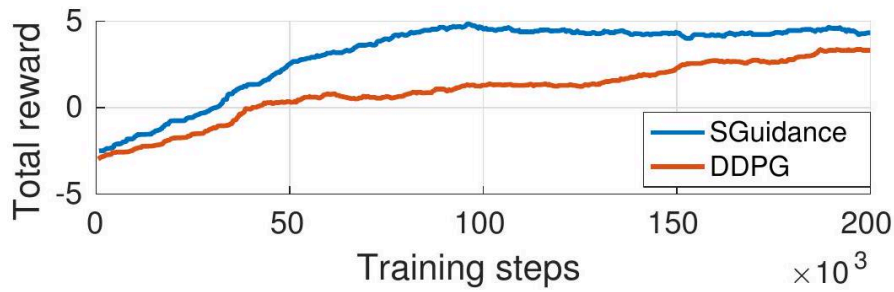


圖 39：圖 4.21：在不同折扣因子 (discount factor) γ 下，SGuidance 各控制器的使用率。每張圖分別繪製了在不同 γ 下三種控制器的使用率。

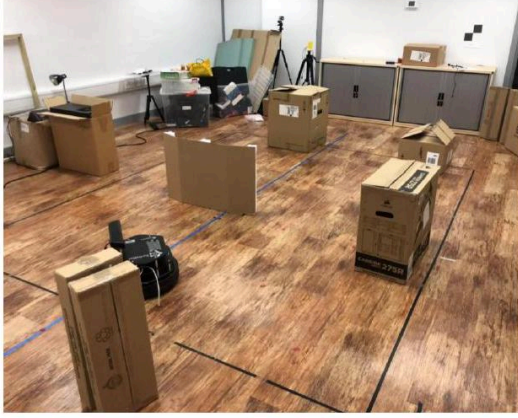
4.8.3 真實世界環境中的導航

在本實驗中，我們定性分析了模型應用於真實世界環境的效能。該模型在由 ROS Gazebo 建立的模擬世界 (圖 4.13c) 中進行訓練，並採用等式 4.1 的密集獎勵函數 (dense reward function)。隨後，模型在不經過任何微調 (fine tuning) 的情況下直接遷移至真實世界場景，以驗證模型的有效性與強大的泛化能力。如圖 4.22a 所示的學習曲線表明，SGuidance 的表現優於原始的 DDPG。

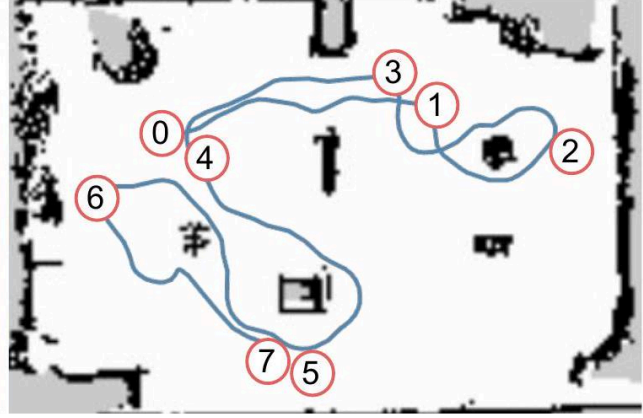
我們使用搭載 Kinetic 深度攝影機的 Turtlebot 2 機器人作為移動平台。因此，為了將訓練好的模型部署到真實機器人上，我們應用了



(a) Learning curves in Gazebo Simulator



(b) Real world environment



(c) Destinations and navigation trajectory

圖 40: 圖 4.22: (a) SGuidance 與 DDPG 模型在 ROS Gazebo 模擬器中達成的總獎勵比較。請注意，由於 Gazebo 模擬了物理特性，與 ROS stage 相比，控制策略更難學習且訓練時間更長。(b) 真實世界場景。使用 Turtlebot 作為移動平台，並在房間內放置數個紙箱作為障礙物。(c) 房間佈局與障礙物為黑色區域。藍色曲線代表機器人的軌跡，目標點以紅圈標示，其中的數字表示順序。

深度影像作為模型輸入。此外，由於深度影像的維度大幅增加，因此採用了 3 層卷積神經網路（如圖 4.12 所示）來提供幾何特徵表示。其他輸入（即速度與目標位置）則與幾何特徵表示串接，形成稠密輸入表示（dense input representation）。

由於在真實世界環境中無法取得機器人位置的地面真值（ground truth），我們應用現成的 AMCL ROS 套件⁶來提供機器人位置的估計，並計算局部座標系中的目標位置。為了提高定位精度，我們使用 Gmapping ROS 套件⁷記錄環境地圖。值得一提的是，這項

在訓練或測試期間，模型的導航組件皆不使用全域地圖。障礙物在房間內的佈置如圖 4.22b 所示。本實驗的目的是運用學習到的策略，控制機器人連續到達多個目的地且不發生任何碰撞。圖 4.22c 顯示了機器人的軌跡（藍色曲線）。請注意，僅透過在模擬環境中使用所提出的隨機引導模型進行學習，機器人就能平滑地避開所有障礙物並成功到達每個目標。

4.9 結論

在本章中，提出了一個有效整合啟發式控制器的新框架，以克服 DDPG 應用於局部導航任務時的高變異性問題。我們的核心創新見解在於，除了學習 DDPG 的策略外，我們還學習了一種與情境相關的切換策略，以決定是否從啟發式控制器中採樣動作，作為 DDPG 組件的訓練數據。因此，與現有方法不同，這種「軟性」引導：1) 可以是簡單的控制器或專家控制器；2) 僅在有利於 DDPG 訓練時才被利用；3) 當 DDPG 的表現超越它們時會自動捨棄。

廣泛的實驗證明，此框架的確定性與隨機性變體都能顯著引導學習過程，並且能超越最先進的基準模型，尤其是在使用極其稀疏的獎勵函數時。在部署時，訓練好的 DDPG 策略可以在不需要引導控制器輸入的情況下直接使用。因此，本章展示了如何高效地學習實用的機器人導航。

第 5 章

全域導航：具備稀疏方向指引與視覺參考的無地圖導航

在研究了將學習到的策略部署於實體機器人，以及加速模擬環境中訓練程序的解決方案後，我們現在將重點轉向學習無地圖導航中全域規劃（global planning）的終極挑戰。在這種情況下，由於缺乏某種形式的空間地圖，全域規劃變得非常具有挑戰性，因此需要額外的環境資訊作為指引。使用何種類型的指引，以及如何有效地將其與即時機器人控制相結合，是需要解決的關鍵問題。在本章中，我們介紹了一個實用的無地圖機器人導航系統：SnapNav。這是一個基於學習的導航系統，僅需要少量的序列方向指令和視覺參考圖像，即可在未知環境中進行導航。我們為此任務引入了一個兩層級的「指揮與控制」（command-and-control）架構。對於高層級的指揮器（commander），我們提出了一種新穎的自我監督訓練方法，以

在有限的真實世界數據下增強其效能。我們也評估了在低層級控制器（controller）中使用前饋（feed-forward）或遞迴（recurrent）策略網路的效用。實驗結果證明，SnapNav 可以直接部署在實體機器人上，並在未見過的真實世界環境中穩健地進行導航。

5.1 引言

「請問接待處在哪裡？」
「直走，看到出口後左轉。」

上述對話是人類在詢問前往未知目的地路徑時，非常常見且高效的互動範例。這類指令

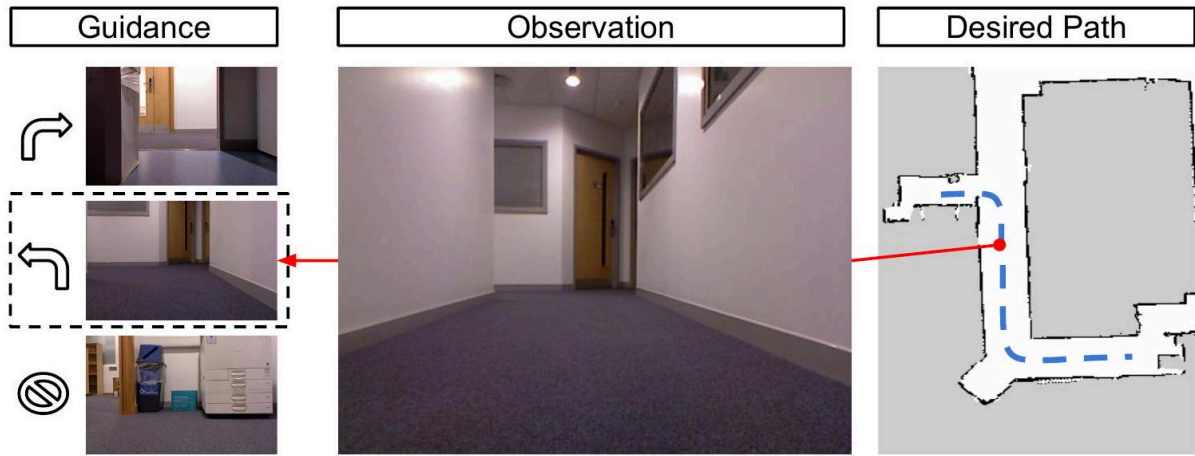


圖 41：圖 5.1：具有稀疏方向引導的視覺導航任務範例。機器人會根據其目前的觀測結果，從提供的引導中自動進行選擇。

由連續動作配對視覺參考所組成的指令，不僅廣泛應用於導航，也用於各種活動中，例如閱讀新產品的使用者指南。這類導航指令具有兩個顯著特徵：第一是動作與視覺觀察空間之間的直接結合，這為「做什麼」以及「在哪裡做」提供了具體的指引；第二個顯著特徵是其固有的稀疏性，因為複雜的指令可以被簡化為在視覺重要路標或提示處的幾個關鍵動作，並依賴人類在這些點之間導航的內在能力。因此，這種形式的指引能產生可高效傳達的準確指令。

這一觀察引發了一個有趣的問題：機器人在提供極其稀疏的指引時，是否能模仿這些人類行為在完全陌生的環境中導航？這樣的機器人系統在通訊上將非常高效，並且具備強大的泛化能力以應對陌生場景，而無需預先建立環境地圖。此外，我們可以想像，只要路標保持視覺上的辨識度，這樣的系統對於動態變化可能會更具魯棒性（robustness）。

在本章中，我們提出了一個基於深度神經網路的系統 SnapNav，作為在未知環境中進行無地圖視覺導航的實用解決方案。它具有以下兩個特徵：首先，它僅需少量的指引即可在未知環境中導航。如圖 5.1 所示，指引包含一張快照圖像，以及在路徑中每個轉向點或終點之前的目標動作（左轉、右轉、停止）。其次，該導航系統採用兩層式階層架構設計，以充分利用來自不同領域（如外觀或深度觀察、模擬或現實環境）的訓練數據，並能夠將學習到的策略直接部署在真實機器人上。

5.2 相關工作

基於學習的無地圖視覺導航在複雜環境中展現了卓越的性能。先前已有許多研究提議透過將環境資訊編碼進深度神經網路參數中，來學習最短路徑策略 [99, 181, 98]。儘管這些技術展現了強健的導航能力，但這些代理人（agent）實際上對訓練環境產生了過擬合（overfitting），無法將學習到的經驗應用於先前未見過的場景。無地圖導航方法的另一個分支是局部導航 [146, 169, 178]，其前提是必須已知相對目標位置，這大大限制了其在現實世界中的應用，僅能用於具備精確定位的場景。

最近，研究人員引入了能夠遵循示範路徑的學習代理人 [72, 53, 143, 112]，這與機器人領域中傳統的視覺教導與重複（Visual teach and repeat, VT&R）範式 [37] 相似。這類解決方案在未知環境中導航時，需要簡單的环境描述，例如原始攝影機影像序列，或額外標註的動作，而非預定義的地圖。在這些研究中，僅有 [53, 112] 能部署在真實機器人上。然而，前者是使用來自全景攝影機的大量人工標註真實世界數據進行訓練，而後者則需要對序列中的示範影像進行顯式定位，這大大降低了其導航的實用性。此外，對長視訊串流的依賴也阻礙了代理人之間的高效通訊。

自然語言指令作為一種極其稀疏的引導形式，也被引入到視覺導航中 [159, 51]。雖然它模仿了人類行為並能產生具有更高自主程度的代理人，但視覺語言接地（visual language grounding）的困難（即關聯來自兩個完全不同模態的感知），加上自然語言本身的歧義性，限制了語言引導導航的性能。[51] 中提到的代理人採用街景縮圖列表結合自然語言指令作為引導，這與我們的 SnapNav 相似，但它僅在名為 StreetNav 的模擬工具包中學習高階導航策略。它並未學習真實機器人所需的低階任務，例如避障。在某種程度上，它也過擬合於街道定義明確且有視覺標記輔助人類駕駛這一事實。例如交叉路口、紅綠燈和停車標誌。因此，他們的方法使用單一策略網路，且僅在模擬環境中進行訓練與執行；正如我們在本章中所展示的，這對於部署在真實機器人上是不夠的。

5.3 任務描述

我們在此研究的任務可歸類為具備稀疏引導的視覺路徑跟隨。特別是，我們考量了自主機器人在即時感知與控制方面的挑戰，並將機器人從純模擬環境帶入現實世界。

5.3.1 任務分解

雖然如 [72,51] 中所述，使用單一策略網路來解決整個任務在概念上非常直觀，但要將這些系統部署在真實機器人上卻非易事。一方面，由於缺乏能同時渲染逼真攝影機數據並精確捕捉機器人動力學 (dynamics) 的模擬器，無法透過單一模擬器學習到可部署於現實環境的指令與控制策略。另一方面，手動標記真實世界數據的成本極高 [53]，且訓練樣本的限制往往因缺乏探索而難以產生強健的控制策略。然而，透過將問題分解為高階指揮官 (high level commander) 與低階自主控制器 (low level autonomous controller)，我們可以利用不同類型與來源的數據來學習子策略，進而優化各個子任務。具體而言，由於目前觀測結果與導引快照 (snapshots) 之間的視覺匹配高度依賴於觀測物件的外觀，因此指揮策略需要具備高視覺保真度的數據，但很大程度上可以忽略機器人動力學。相反地，控制策略僅專注於周圍環境的幾何形狀以進行避障、機器人動力學以及偶爾的指令/高階動作。因此，此策略可以更容易地在機器人模擬器中進行訓練，因為模擬器能精細地模擬機器人動力學與深度觀測，這建立在先前章節所呈現的工作基礎之上。機器人也可以接觸到各種任意的世界設置，從而實現更強健的區域導航。每個子任務將在以下部分進行更具體的闡述。

5.3.2 指揮子任務

指揮官 C 被提供 n 對導引 $\{\mathbf{G}_i = (\mathbf{S}_i, m_i) \mid i = [1, 2, \dots, n]\}$ ，其中 $\mathbf{S}_i$ 與 m_i 分別代表快照與導引指令。每一份導引中的快照 $\mathbf{S}_i$ 記錄了機器人應改變方向或在特定点停止之區域的第一人稱視角 RGB 影像。由於機器人僅在需要改變方向或終止任務時才會收到命令，因此導引中只有三種類型的指令： $m_i \in \{\text{「右轉」}、\text{「左轉」}、\text{「停止」}\}$ ，而隱含的動作則是繼續直線前進。

接著，結合所有引導資訊與來自機載攝影機的當前影像觀測值 $\mathbf{O}_t$ ，指揮官 C 在每個時間步 $c_t \in \{\text{預測一個高階指令 } t \text{ 「右轉」、「直行」、「左轉」、「停止」}\}$ ，表示為 $c_t = C(\mathbf{O}_t, \mathbf{G}, \mathbf{h}_c)$ ，其中 $\mathbf{h}_c$ 是指揮官中 GRU 單元的隱藏狀態。

5.3.3 控制子任務

控制子任務被正式定義為一個馬可夫決策過程 (MDP)。在時間 $t \in [1, T]$ ，機器人根據觀測值 $\mathbf{X}_t$ 採取動作 $\mathbf{a}_t \in \mathcal{A}$ 。執行動作後，機器人會根據獎勵函數接收由環境給予的獎勵 r_t ，然後轉移到下一個觀測值 $\mathbf{X}_{t+1}$ 。此 MDP 的目標是達到最大化的折扣累積未來獎勵 $R_t = \sum_{\tau=t}^T \gamma^{\tau-t} r_\tau$ ，其中 γ 為折扣因子。

更具體地說，動作是機器人的控制訊號 $\mathbf{a}_t = (a_t^v, a_t^\omega) \in \mathcal{A}$ ，其中 a_t^v 與 a_t^ω 分別表示在時間 t 的預期線速度與角速度。觀測值 $\mathbf{X}_t$ 是第一人稱視角的深度影像，可以直接在模擬器 (例如 ROS Gazebo¹) 中獲取，或在現實世界中透過現成的估計器 [43] 從 RGB 影像中估計。在時間 t 的獎勵函數 r_t 定義如下：

$$r_t = \begin{cases} R_{\text{crash}}, & \text{if robot crashes} \\ R_{\text{reach}}, & \text{if robot reaches the goal} \\ d_{t-1} - d_t, & \text{otherwise} \end{cases}$$

其中 R_{crash} 是碰撞懲罰， R_{reach} 是到達目標點的正向獎勵， d_{t-1} 與 d_t 則表示機器人在連續兩個時間步 $t-1$ 與 t 時，與目標點 (下一個轉向點或最終目的地) 之間的距離。

5.4 網路架構

5.4.1 基於注意力機制的指揮器 (Attention-Based Commander)

為了完成指揮子任務，指揮器設計了卷積神經網路 (CNN) 層與線性嵌入 (EMB) 層，用以處理原始的

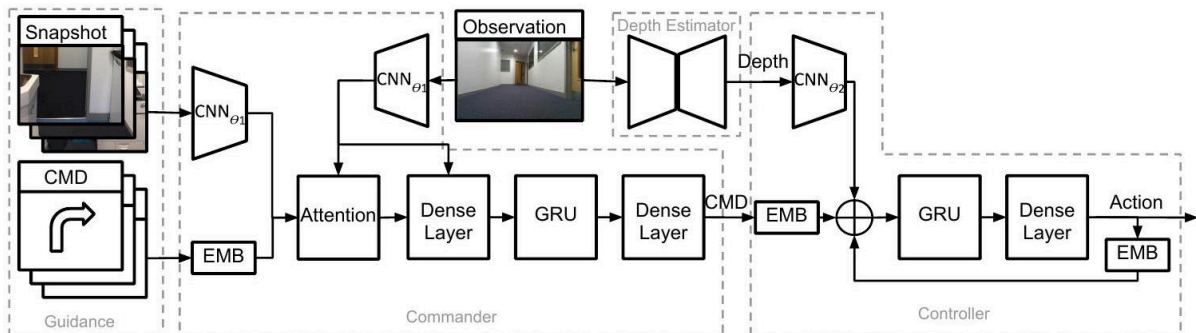


圖 42: 圖 5.2: SnapNav 的網路架構，由兩個模組組成。指揮官 (commander) 首先透過尋找與當前觀測值 (observation) 正確匹配的項目，來關注特定的引導指令。接著它會發布一個高階指令。隨後，控制器 (controller) 根據該指令、估計的深度圖像和上一次預測的動作，來預測低階機器人動作。請注意，指令以縮寫「CMD」表示，「EMB」代表

線性嵌入層 (linear embedding layer) , 「GRU」表示閘道循環單元 (Gated Recurrent Unit) [25], 而 $\oplus$ 則是串接 (concatenation) 運算。

輸入, 隨後是一個硬注意 (hard attention) 元件和一個循環策略網路。

如圖 5.2 所示, 輸入至指揮官的圖像首先由 5 個卷積層進行編碼, 而指令則輸入至線性嵌入層。更具體地說, 引導資訊 $\mathbf{G}$ 中的快照 $\mathbf{S}_i$ 與指令 $m_i (i = 1, 2, \dots, n)$, 以及當前觀測值 $\mathbf{O}_t$, 分別被轉換為向量 $\mathbf{v}_i^S = \text{CNN}_{\theta_1}(\mathbf{S}_i)$ 、 $\mathbf{v}_i^m = \text{EMB}(m_i)$, $\mathbf{v}_t^O = \text{CNN}_{\theta_1}(\mathbf{O}_t)$ 。由於快照和當前觀測值屬於同類型數據, 用於編碼它們的 CNN 具有相同的參數 θ_1 。

給定向量化輸入 $\mathbf{v}_i^S, \mathbf{v}_i^m$ 和 $\mathbf{v}_t^O$, 目標是選擇包含與當前觀測值最相關快照的引導資訊。直觀上, 由於引導資訊的稀疏性, 在每個時間步長中, 只有高度相關的快照應與指令策略進行匹配。因此, 硬注意 (hard attention) 優於僅對所有具備不同權重的引導指令進行加總的軟注意 (soft attention)。

硬注意力 (Hard attention) 通常使用離散隨機層 (discrete stochastic layer) 進行建模, 由於其不可微分, 因此必須使用傳統反向傳播 (backpropagation) 以外的梯度估計方法來進行優化 [102]。幸運的是, 如 [51, 86] 所示, 另一種替代方案是採用最大池化算子 (max-pooling operator) 的泛化形式來選擇最佳引導指令。這避開了不可微分的問題:

$$(\mathbf{v}_{i^*}^S, \mathbf{v}_{i^*}^m) = \underset{(\mathbf{v}_i^S, \mathbf{v}_i^m)}{\operatorname{argmax}} \left[\operatorname{softmax} \left(-\|\mathbf{v}_i^S - \mathbf{v}_t^O\|_2 \right) \right].$$

這產生了一個次可微 (sub-differentiable) 模型, 並能與其他基於梯度的模型結合。我們隨後的實驗證明, 注意力機制的效能可以被

透過將 REINFORCE 演算法 [164] 等其他梯度估計器與反向傳播 (back-propagation) 結合, 或是利用輔助任務 (即度量學習, metric learning) 進行訓練, 可以大幅提升效能。接著, 注意力機制處理後的快照向量 (attended snapshot vector) 會與嵌入指令 (embedded command) 一同經由全連接層處理, 並與編碼後的觀測值 (encoded observation) 進行串接。最後, 使用循環網路 (recurrent network) 在所選快照與當前觀測值足夠相似時, 預測注意力機制下的指令 (attended command)。若相似度較低, 則發出預設的「前進」(Go forward) 指令。因此, 來自指揮器 (commander) 的預測指令可表示為 $c_t = C(\mathbf{O}_t, \mathbf{G}, \mathbf{h}_c)$, 其中 $\mathbf{h}_c$ 為循環網路的隱藏狀態 (hidden state)。

5.4.2 控制器

給定由 Commander 預測的指令 $c_t \in \{ \text{「直行」, 「右轉」, 「左轉」, 「停止」} \}$, Controller 的上一個動作 $\mathbf{a}_{t-1}$, 以及由模擬器或深度預測網路提供的深度圖像 $\mathbf{X}_t = D(\mathbf{O}_t)$, Controller 會根據其訓練好的策略輸出導航環境的最佳動作。與 Commander 類似, 原始深度圖像經由 CNN 編碼為 $\mathbf{v}_t^X = \text{CNN}_{\theta_2}(\mathbf{X}_t)$, 而指令則被嵌入為 $\mathbf{v}_t^c = \text{EMB}(c_t)$ 。接著, 這些資訊被用於 Controller 預測動作 $\mathbf{a}_t = \pi(\mathbf{v}_t^X, \mathbf{v}_t^c, \mathbf{h}_\pi, \mathbf{a}_{t-1})$, 其中 $\mathbf{h}_\pi$ 是 Controller 中的循環隱藏狀態 (recurrent hidden state), 而 θ_2 則代表深度編碼 CNN 的參數。

循環網路 (recurrent network) 在 Controller 中至關重要, 因為它需要精確決定何時執行左轉或右轉指令。這是因為高階指令是與最大視覺匹配項對齊的, 而這些匹配項可能並未精確對齊預期的轉向點。這種解耦設計實現了更高程度的自主性, 因為低階 Controller 可以自行決定何時是最佳轉向時機。

5.5 訓練

在本節中, 我們將分別介紹 Controller 與 Commander 的訓練機制, 因為指令策略 (commanding strategy) 與控制策略 (control policy) 是使用不同的方法與數據分別進行學習的。

5.5.1 自監督式指揮官訓練標籤

本節提出一種新穎的自監督技術來建立偽標籤 (pseudo-labels), 而非依賴人工標註的影片。給定一段原始影片, 光流 (optical flow)

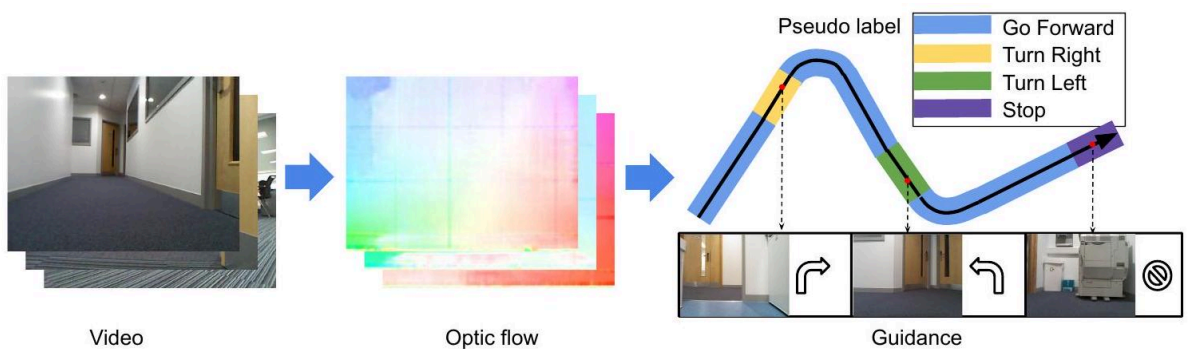


圖 43：圖 5.3：影片首先根據估計的光流進行分割。接著，在代理機器人實際轉向之前，分配方向變化指令的偽標籤（黃色和綠色區段）。重要的是，這些標籤是隨機分配的，而非根據地面真值（ground-truth）。「停止（Stop）」指令僅在影片的最後部分（紫色部分）標註，其餘影格則設定為預設指令「前進（go forward）」（藍色區域）。

首先使用 FlowNet [30] 估計相鄰影格之間的光流，並在光流較高處（即可能的轉向點）進行分割。這是假設沿著直線、連續軌跡收集的影像具有高度相似性，而轉角處的影像則顯示出高度差異。接著，在每個區段末尾附近隨機取樣一個快照，並連同其最近的 k 個影格（本章中為 $k = 20$ 個），標註一個從 {“Turn right”, “Turn left”, “Stop”} 中隨機抽取的指令。值得注意的是，每個轉向點之前的左轉或右轉指令並非根據代理機器人在數據收集期間採取的實際動作來標註，而是隨機分配的。這是防止指揮官網路（commander network）對小型真實世界數據集產生過擬合（overfitting）的關鍵。最後，其餘影格皆標註為「Go Forward」指令，作為指揮官的預設預測。

請注意，僅收集了包含 5,000 張真實序列影像的小型資料集用於微調 commander 網路。在此之前，我們使用從 ROS Gazebo 透過標準導航套件 ROS Navigation² 自動收集的 100,000 張序列影像來初始化網路。

5.5.2 訓練 Commander

給定標記數據後，可以使用幾種不同的學習訊號來優化 commander，這些訊號可以單獨或共同使用。我們在下方討論每種方法的相對優缺點。

Command Loss 基本方法是最小化預測指令的機率分佈 $p \in \{0, 1\}^M$ 與偽指令標籤 $y \in \{0, 1\}^M$ 之間的交叉熵損失（cross-entropy loss），公式如下：

$$Loss_{cmd} = -\frac{1}{T} \sum_{t=0}^T \sum_{m=0}^M y_t^m \log(p_t^m)$$

其中 T 和 M 分別代表給定數據序列的序列長度與指令類別數量。這依賴於 [51] 中提到的 $\text{argmax}(\cdot)$ 次可微（sub-differentiable）特性，並可以使用標準反向傳播演算法進行優化。

使用 REINFORCE 學習注意力策略：本章中我們的新穎見解是使用 REINFORCE [164] 演算法來估計梯度，以學習更好的注意力策略，因為該策略由於次可微的 argmax 函數而導致訓練效果不佳。REINFORCE 演算法的框架通常將策略學習過程建模為一個 MDP，其中代理（agent）是指揮器（commander）中的注意力層（它獨立於控制策略學習的 MDP）。請注意，注意力層不包含任何可訓練參數，因此注意力策略 $\pi(u_t | \mathbf{G}, \mathbf{O}_t; \theta_1)$ 完全依賴於由 θ_1 參數化的 CNN 編碼器。給定引導 $\mathbf{G}$ 和序列觀測值 $\mathbf{O}_t$ ，注意力策略會引發可能交互軌跡 $s = \mathbf{O}_1, u_1, \dots, \mathbf{O}_T, u_T$ 的分佈，其中 u_t 和 $\mathbf{O}_t$ 分別是快照的關注位置和時間步 t 的觀測值。目標是最大化沿著交互序列 s 累積的獎勵 $J(\theta_1) = \mathbb{E}_{p(s; \theta_1)} \left[\sum_{t=1}^T \gamma^{T-t} r_t \right]$ 。請注意，軌跡 s 出現的機率取決於注意力策略，且每次的獎勵 r_t 與指令預測準確度成正比。梯度使用具有 N 個學習樣本的蒙地卡羅（Monte-Carlo）方法進行近似：

$$\begin{aligned} \nabla_{\theta_1} J &= \sum_{t=1}^T \mathbb{E}_{p(s_{1:T}; \theta_1)} [\nabla_{\theta_1} \log \pi(u_t | \mathbf{G}, \mathbf{O}_t; \theta_1) R_t] \\ &\approx \frac{1}{N} \sum_{n=1}^N \sum_{t=1}^T \nabla_{\theta_1} \log \pi(u_t^n | \mathbf{G}^n, \mathbf{O}_t^n; \theta_1) R_t^n \end{aligned}$$

度量學習（Metric Learning）：作為 REINFORCE 學習訊號的另一種替代方案，度量學習 [129] 也可以應用於顯著改進影像編碼，透過強制輸出嵌入（embeddings）位於度量空間內。這可以被視為與指令預測並行的輔助任務。



圖 44: 圖 5.4: ROS Gazebo 中的隨機環境及其地圖範例。系統設計了一個 Oracle 指揮官，在機器人到達轉向點和終止點之前給予正確指令。紅圈標示出一個具挑戰性的情況，該路口未完全受障礙物限制，容易使機器人產生混淆。

與 [129] 相似，本研究採用三元組損失 (triplet loss) 進行度量學習 (metric learning)。由於在生成偽指令標籤 (pseudo command labels) 時已對每段影片進行分割，因此可以從其中一個片段中採樣一張錨點圖像 (anchor image)。接著，鄰近的圖像可被定義為正樣本 (相似) 圖像，而其他片段中的所有圖像則被標記為負樣本 (不相似) 圖像。因此，經由 CNN 編碼後，可以從每個原始圖像序列中隨機生成一個包含錨點圖像向量 $\mathbf{v}^a, k^+$ 、正樣本圖像向量 $\mathbf{v}^+$ 以及 k^- 個負樣本圖像向量 $\mathbf{v}^-$ 的三元組，其損失函數公式如下：

$$\text{Loss}_{\text{metric}} = \left[\frac{1}{k^+} \sum_{i=1}^{k^+} l_2(\mathbf{v}_i^+ - \mathbf{v}^a) + \sigma - \frac{1}{k^-} \sum_{j=1}^{k^-} l_2(\mathbf{v}_j^- - \mathbf{v}^a) \right]_+,$$

其中 $[\cdot]_+$ 是 Hinge 函數， $l_2(\cdot)$ 代表歐幾里得距離 (Euclidean distance)， σ 是設置為 1 的邊界值 (margin)，而 k^+ 與 k^- 設置在 20 以下，以確保正樣本圖像的高度相似性，並維持正負樣本之間的平衡比例。此損失函數可透過梯度反向傳播 (gradient backpropagation) 與指令損失 (command loss) 共同最小化，因此與使用 REINFORCE 演算法相比，更加方便且簡單。

5.5.3 控制策略的強化學習

控制策略純粹是在虛擬環境中透過 DRL 學習而得。為了增強控制器的泛化能力並降低代理人記憶環境的可能性，訓練環境的幾何形狀每 40 個回合 (episode) 會隨機變換一次，且如圖 5.4 所示，在每個回合中，機器人需遵循的期望路徑也是隨機生成的。接著，為了實現從模擬機器人到真實機器人的強健轉移 (robust transfer)，我們在虛擬和真實世界中皆使用精細建模的 Turtlebot 2 機器人³，並配備 Microsoft Kinect⁴ 以擷取深度與 RGB 影像。請注意，SnapNav 可以使用 Kinect 提供的地面實況 (groundtruth) 深度，或是估計的深度，使其能相容於僅配備單眼相機的系統。

我們使用幾種不同的演算法來訓練控制策略，例如 DDPG [79]、RDPG [50] 和 DRQN [49]。所有這些 DRL 演算法首先會配合「神諭指揮官」(oracle commander) 進行 1 M 個訓練步數，該指揮官會根據機器人在期望路徑中的位置始終給予代理人正確的指令；隨後再利用學習到的指令策略所產生的雜訊預測指令進行 0.5 M 個訓練步數的微調。我們在表 5.1 中展示了在不同指揮官下持續提升的整體效能。訓練是在單張 GTX970 GPU 上進行，每次執行約耗時 20 小時，其中控制頻率為 5 Hz，且模擬器的運行速度比真實世界快 4 倍。

5.6 實驗

我們進行了模型消融研究 (ablation study) 與真實世界測試，以評估 SnapNav 提案模型的最佳性，以及其對真實場景的泛化能力。

5.6.1 虛擬世界中的模型消融研究

消融研究中的每個模型都在類似於圖 5.4 的隨機環境中進行測試，共執行 1000 次獨立運行，並以到達最終目的地的成功率 (SR) 作為評估指標。

注意力策略學習：我們比較了幾種不同的模型與學習訊號來訓練指揮器 (commander)。最基礎的模型不使用注意力機制，僅簡單地加總所有編碼後的引導資訊來進行指令預測，我們將其稱為 AllSum。接著，由於 [51] 指出軟注意力 (soft attention) 的效能與 AllSum 相似，因此我們考慮了如 5.4.1 節所述的硬注意力 (hard attention) 模型，該模型純粹透過指令損失 (command loss) 學習訊號進行優化，並且是

表 2：表 5.1：使用不同指揮官（Commander）的成功率（SR%），以及純粹以 Oracle 指揮官初始化，或使用各個指揮官進一步微調後的 DRQN 成功率。

	DRQN	微調後的 DRQN
AllSum	50.2%	54.4%
HardAtt	60%	61.6%
REINFORCE	72.7%	74.2%
指標	70.2%	73.6%
組合	71.2%	74.9%
Oracle	85%	—%

稱為 HardAtt。接著，我們加入 REINFORCE (REINFORCE) 或度量學習 (Metric) 的學習訊號。我們也將三種學習訊號結合在一起 (Combined)。

從表 5.1 顯示的結果中，我們可以清楚地看到，採用注意力機制（attention mechanism）的指揮官（commander）表現明顯優於 AllSum，這證明了在指令預測任務中，關注特定的快照（snapshot）至關重要。與現有技術相比，我們引入的 REINFORCE 演算法或度量學習（metric learning）都帶來了進一步且顯著的提升。

為了更深入了解造成這種差異的原因，圖 5.5 展示了樣本影片經由指揮官編碼後的歐幾里得距離矩陣（Euclidean distance matrix）。結果顯示，與 HardAtt 和 REINFORCE 相比，Metric 和 Combined 所學習到的編碼器能更清晰地區分不同片段的影像。因此，這應該能讓導航過程中更容易關注到正確的快照。然而，根據表 5.1，與度量學習相比，REINFORCE 的學習訊號在提升指揮官整體效能方面更為顯著。這可能是因為 REINFORCE 演算法是直接根據指令預測的準確性來優化注意力策略，而度量學習則是在編碼空間中手動引入的偏差，與導航任務並不完全相關。

控制策略學習（Control policy learning）：選擇合適的 DRL 方法來訓練控制器是整個系統效能的關鍵。從圖 5.6 所示的學習曲線中，我們首先透過比較 DDPG 與 RDPG（後者為前者的循環版本），證明了在控制器模型中引入循環網路（recurrent network）的必要性。由於指令是在機器人執行動作前分配給控制器的

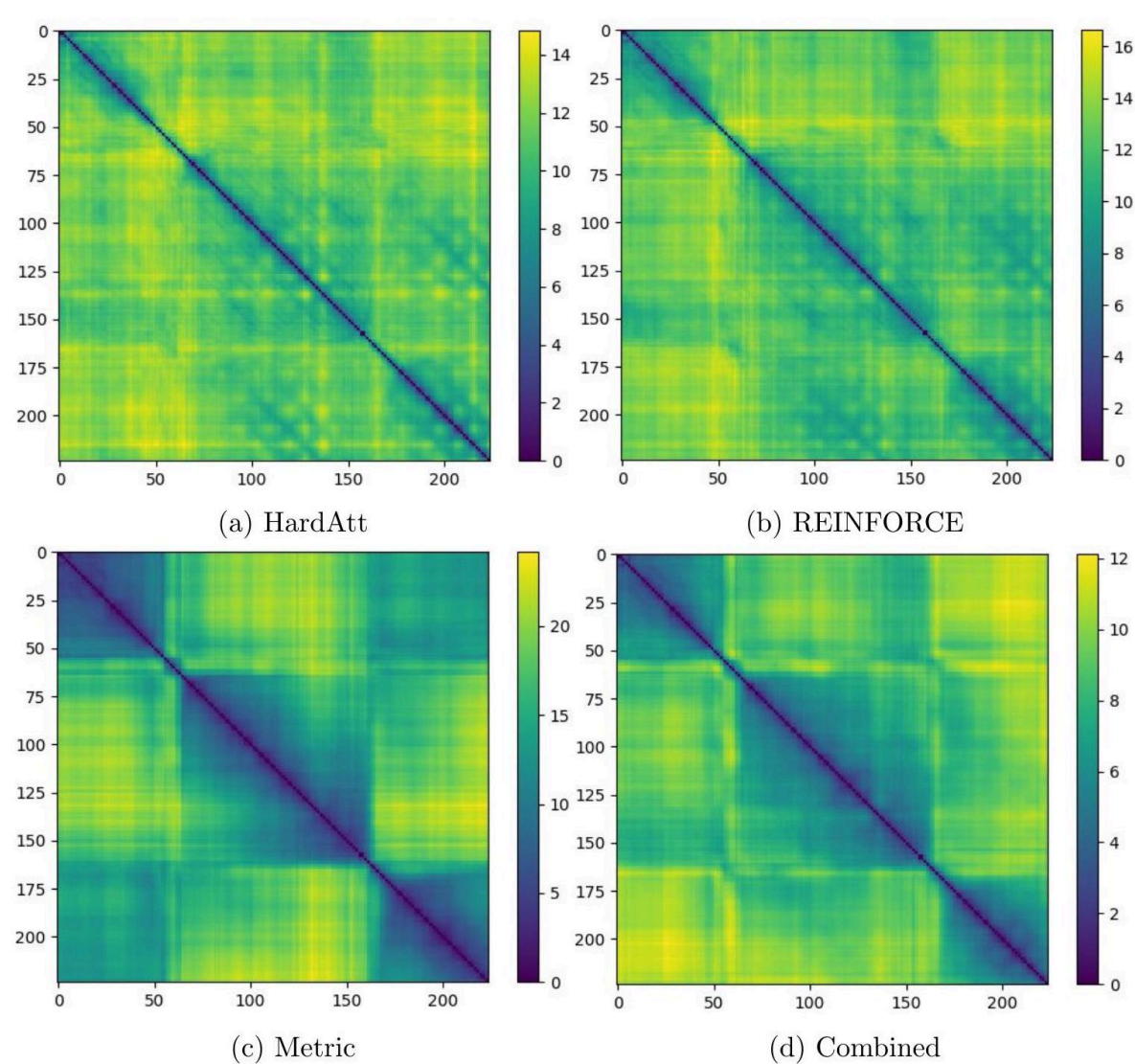


圖 45：圖 5.5：樣本影片中編碼影像的歐幾里得距離矩陣。

到達轉折點時，循環網路對於記憶近期指令變得至關重要。

接著，我們探討透過利用基於數值的 DRQN 方法，藉由限制動作與搜尋空間所帶來的改進。動作空間被離散化為三個選項：以最大線速度直行，以及以最大角速度和較慢的線速度左轉或右轉。相應地，Q-network 僅需根據當前觀測值和循環網路的隱藏狀態，估計這些選項的 Q 值。這大幅縮小了搜尋空間並提升了學習效率。

表 5.2 深入探討了所學策略的性能。DDPG 在大多數回合中以碰撞告終，因為它試圖維持預期的總

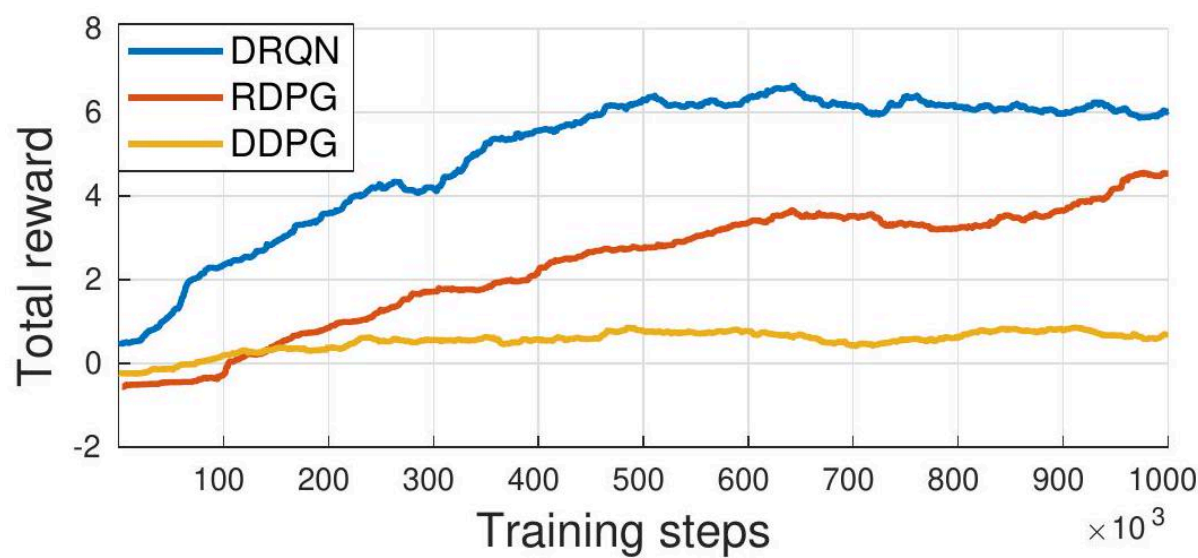


圖 46：圖 5.6：分別為 DRQN、RDPG 與 DDPG 的平滑學習曲線。每種演算法皆在 ROS Gazebo 中使用 oracle commander 進行了 100 萬步的訓練。

表 3：表 5.2：不同控制器在 1000 次測試回合中的獎勵分佈。請注意，Success 表示成功抵達最終目的地，而 TimeOut 和 Crash 分別表示機器人無法在 300 步內抵達目的地或與障礙物發生碰撞的情況。

	Success	TimeOut	Crash
DDPG	3.4%	31.8%	64.8%
RDPG	60.9%	36.6%	2.5%
DRQN	85%	10.3%	4.7%

透過最小化朝向目的地反方向移動的機率來獲得獎勵，因為這會導致即時的負獎勵。相比之下，其循環版本（RDPG）由於能夠記憶歷史指令，並在交叉路口據此轉向正確方向，因此實現了更高的成功率。然而，連續動作的大型探索空間阻礙了網路在高度隨機的路口學習穩定且強健的轉向行為，特別是在圖 5.4 中標出的具有不常見幾何形狀的交叉路口。最後，在離散動作空間中進行探索的 DRQN 展現了顯著的增益。

5.6.2 真實世界測試

除了在虛擬環境中進行的大量實驗外，我們還在真實世界場景中評估學習到的策略，以展示其在實際機器人場景中的實用性。為了讓指揮器（commander）在真實世界測試時能更好地泛化至大量的引導資訊，我們對注意力策略（attention policy）施加了一個手動調整的閾值，當目前的引導快照（snapshot）與環境相似時，便切換至下一個快照。

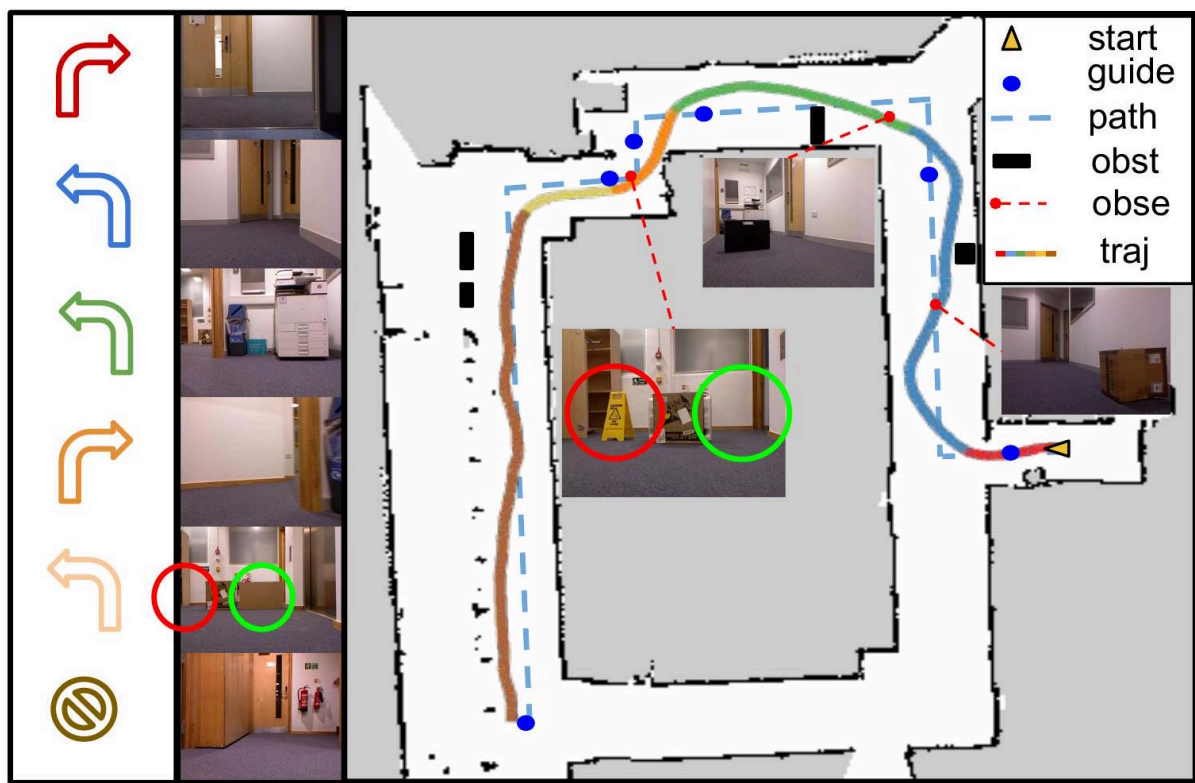


圖 47：圖 5.7：真實世界測試範例。圖例由上至下分別為：起始位置、引導記錄位置、示範階段路徑、測試階段的隨機障礙物、機器人觀測結果以及測試中的機器人軌跡。注意力位置以機器人軌跡中的不同顏色表示，對應於引導中動作的顏色。紅色和綠色圓圈強調了示範與測試之間環境視覺外觀和幾何形狀的變化。

足以應對當前的觀測。圖 5.7 展示了一個真實世界的測試範例，定性地檢驗了 SnapNav 的避障能力以及對環境變化的魯棒性。

此外，如表 5.3 所示，我們提出了兩個基準模型進行定量比較。SimOnly 實作了與 [51] 相似的模型，但增加了深度觀測，且完全在模擬器中透過 DRL 進行訓練。另一個則是基於有限真實世界數據的深層 VT&R 模型 [73] 的監督式變體。SimOnly 基準模型僅在模擬中訓練，其表現優於 SnapNav，這證明了直接轉移在單一網路中學習到的命令與控制策略並非易事。VT&R 是 SnapNav 的密集引導對應版本，直接使用真實世界數據的監督進行訓練。在我們的實驗中，它的表現最差，完全沒有學會避開任何新障礙物。這驗證了我們的假設，即有限的

表 4：表 5.3：真實世界環境中的評估。使用的指標包括：成功率 (SR)、碰撞前或到達目的地前的平均行駛距離 (Dist) 以及作為引導使用的圖像數量 (ImgNum)。每個模型均經過 5 次獨立測試。

	SR	Dist(m)	ImgNum
SnapNav	80%	24.29	6
SimOnly	40%	20.19	6
VT&R	20%	18.32	326

訓練數據無法產生穩健的導航策略，同時也證明了我們兩層式階層架構所帶來的高自主性。

5.7 結論

在本章中，我們針對全域無地圖導航 (global mapless navigation) 提出了一個實用的解決方案。其基本概念與近期的 StreetNav [51] 相似，皆是在關鍵路點提供少量快照 (snapshots) 並搭配方向指令作為導航指引。然而，由於訓練數據與學習策略之間的緊密耦合，StreetNav 無法很好地遷移到新場景，且僅在模擬環境中進行過測試。我們提出的技術透過使用兩層式階層架構，將感知與執行解耦，使該系統具備實用性。透過廣泛的實驗，我們展示了 SnapNav 相比 StreetNav 有顯著的改進，特別是：

透過我們創新的自我監督學習機制，注意力策略 (attention policy) 得到了顯著提升，該機制僅需極少量的真實世界數據即可成功訓練。

該導航系統已成功適配於即時機器人控制任務，且由於其具備兩層式階層架構 (two-level hierarchy)，可以直接部署於實體機器人上。

改良後的損失指標 (loss metric) 是讓注意力機制穩健地聚焦於正確視覺路標 (visual waypoint) 的關鍵，並展現出對場景變化的良好免疫力。

我們相信這是首項在室內環境中，利用稀疏視覺引導 (sparse visual guidance) 實現實用化全域無地圖導航 (global mapless navigation) 的研究工作，也是邁向提升機器人自主性的一步。

結論與未來展望

本論文探討了移動機器人無地圖導航 (mapless navigation) 的多項任務。針對將深度強化學習 (DRL) 方法成功應用於現實世界機器人問題時所面臨的挑戰，提出了實用的解決方案。

由於在現實世界中進行訓練並不可行，首要問題是如何將學習到的策略從模擬世界遷移到現實世界，而遷移的成功與否高度取決於模擬器的真實度。由於缺乏能同時捕捉機器人動力學並渲染高視覺保真度攝影機觀測結果的虛擬環境，直接將訓練好的策略部署到實體機器人上具有挑戰性。在第 3 章中，我們透過解決單眼視覺障礙物規避任務來探討這一挑戰。我們利用一個具備高效能物理引擎但簡化攝影機觀測的模擬器，這凸顯了感知遷移是我們的核心問題。在我們的案例中，透過將機器人的觀測從視覺外觀領域 (appearance domain) 投影到互補的深度領域 (depth domain) 來緩解現實差距 (reality gap)，這具有多項優點。首先是實作簡單，僅需使用現成的深度估計器，並利用少量配對數據進行微調。相較之下，領域隨機化 (domain randomisation) 需要精心選擇雜訊分佈，而領域自適應 (domain adaptation) 則必須針對不同的模擬環境從頭學習適配器網路。另一個優點是深度領域提供的幾何資訊極大地促進了障礙物規避的學習，進而加速訓練過程。因此，在本章中，我們展示了如何透過領域簡化 (domain simplification) 來彌合現實差距。

第 4 章探討了如何在區域導航任務中進行高效學習。我們提出了一個新穎的框架，將來自其他次優但簡單的控制器的導航知識，作為引導整合到離策 (off-policy) DRL 方法的訓練中。這是透過在導航策略的同時學習一個切換策略 (switching policy) 來實現的。根據具體情況從較佳的控制器中取樣動作。因此，底層的 DRL 方法最初可以從先驗控制器的引導中學習，一旦超越了先驗控制器的性能，隨後便能透過自我探索來改進其自身的策略。我們分別檢驗了確定性切換策略 (deterministic switching policy) 及其隨機對應版本 (stochastic counterpart)，後者被證明更為優越，因為它天生對局部最小值 (local minima) 具有更強的魯棒性。與模仿學習 (imitation learning) 相比，所提框架的主要優勢在於它不僅模仿控制器的行為，還能學習如何超越它們。在某種程度上，我們的方法可以被視為「透過啟發學習」而非模仿，從而讓學生超越老師。我們注意到，無論是簡單還是專家級的先驗控制器，都有助於提高訓練過程的效率。

針對關鍵的全域無地圖視覺導航 (global mapless visual navigation) 問題，其解決方案於第 5 章中詳述。所提出的系統僅需稀疏的視覺參考和方向引導即可進行導航，這使得策略能夠透過對環境的極簡描述，輕鬆泛化至未見過的場景。這種擬人化的行為賦予了機器人比 VT&R 更高的自主能力。我們將系統創新地構建為雙層階層式架構 (two-layer hierarchy)，實現了任務分解，這是使系統具備實用性與可實現性的關鍵步驟。與嘗試用單一策略網路學習整個任務相比，這也使得訓練更加高效。該階層架構有利於利用來自不同領域和來源的訓練數據，即外觀或深度觀測、模擬或真實的感知與動作，使學習到的策略能直接部署在真實機器人上。

未來，本論文的工作可以在以下方面進一步探索與擴展：

動態環境中的視覺避障：在第 3 章中，我們僅研究了在相對靜態、無快速移動物體環境下的導航。動態物體極大地增加了單眼視覺避障的難度，因為無模型 (model-free) 方法很難預測這些移動物體的未來位置。然而，我們提出的基於深度的感知網路可以與物件追蹤模組整合，以建模機器人與所有可觀測移動物體之間的相對速度。透過結合注意力機制 (attention mechanism)，這些資訊可以有效幫助 DRL 代理程式在高度動態的環境中學習到強健的避障策略。

階層式強化學習：第 4 章提出的框架可以同時使用底層 DRL 方法學習導航策略，並透過 REINFORCE 學習訊號學習切換策略，以便在共享獎勵函數與經驗回放記憶體 (replay memory) 的情況下，於不同的導航控制器之間進行選擇。然而，從另一個角度來看，我們可以將切換網路視為高層級指揮官，並將所有控制器視為低層級執行器，並為它們分配獨立的經驗回放記憶體與分解後的獎勵函數。這種轉變使該框架能夠處理更複雜的任務，其中每個執行器僅需根據分解後的獎勵以及記錄在各自經驗回放記憶體中的樣本，負責學習單一旦簡單的功能。接著，指揮官再學習根據整體獎勵函數，在不同情況下調度特定的執行器。

結合行為導向機器人學與 DRL：由於第 3 章與第 4 章中的任務與演算法與行為導向機器人學 (behaviour based robotics) 相關，將此領域與 DRL 方法結合將會非常有趣。這將能開發出更具智慧的行為導向演算法。例如，對於基於多個分別具有不同簡單行為之控制器的演算法，DRL 代理人可以根據不同情況下的感測輸入，學習所有行為的最佳組合策略。此外，藉由利用循環神經網路 (RNN) 作為記憶體，也有可能讓行為導向代理人更容易地執行長期導航任務。

多機器人導航：在第 5 章中，所提出的無地圖導航系統自然地產生了一種高效的通訊方式來共享導航經驗，而非傳輸詳細的點雲地圖或佔據網格 (occupancy grids)。因此，在這種稀疏引導的框架下建立多機器人導航系統是大有可為的。在此領域中，有幾個問題會立即浮現，即：1) 如何針對發布的查詢檢索每個機器人的經驗？2) 如何在整個網路中高效地傳播每個代理人有限的知識？3) 如何整合從不同代理人獲取的所有資訊，以規劃通往目的地的最佳路徑？這些都是值得進一步討論的有趣問題。

總結而言，在未來的十年中，移動機器人很可能在日常生活中扮演日益重要且無處不在的角色，例如在居家清潔、製造、運輸和保全等領域。在這種情況下，要求機器人具備手動為每台機器人定義並分配特定領域的知識與經驗是缺乏效率的。機器人將需要透過共享知識在共同的框架內進行互動，讓多個代理程式 (agents) 在無需任何人工輸入或標記的情況下探索潛在的解決方案，朝向真正的認知發展，而非僅僅是簡單的模式識別。本論文提出了在現實場景中基於 DRL 的實用機器人無地圖導航解決方案，是朝向該目標邁出的第一步。

參考文獻

- [1] Ibraheem Alhashim and Peter Wonka. High quality monocular depth estimation via transfer learning. arXiv preprint arXiv:1812.11941, 2018.
- [2] Marcin Andrychowicz, Filip Wolski, Alex Ray, Jonas Schneider, Rachel Fong, Peter Welinder, Bob McGrew, Josh Tobin, OpenAI Pieter Abbeel, and Wojciech Zaremba. Hindsight experience replay. In *Advances in Neural Information Processing Systems*, pages 5048-5058, 2017.
- [3] Antonis A Argyros, Kostas E Bekris, Stelios C Orphanoudakis, and Lydia E Kavraki. Robot homing by exploiting panoramic vision. *Autonomous Robots*, 19(1):7-25, 2005.
- [4] Ronald C Arkin. Motor schemabased mobile robot navigation. *The International journal of robotics research*, 8(4):92-112, 1989.
- [5] Karl Johan Åström and Tore Hägglund. PID controllers: theory, design, and tuning, volume 2. Instrument society of America Research Triangle Park, NC, 1995.
- [6] Martin Azizyan, Ionut Constandache, and Romit Roy Choudhury. Surroundsense: mobile phone localization via ambient fingerprinting. In *Proceedings of the 15 th annual international conference on Mobile computing and networking*, pages 261-272. ACM, 2009.
- [7] Tim Bailey, Juan Nieto, Jose Guivant, Michael Stevens, and Eduardo Nebot. Consistency of the ekf-slam algorithm. In *2006 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 3562-3568. IEEE, 2006.
- [8] Andrew G Barto, Richard S Sutton, and Charles W Anderson. Neuronlike adaptive elements that can solve difficult learning control problems. *IEEE transactions on systems, man, and cybernetics*, (5):834-846, 1983.
- [9] Herbert Bay, Tinne Tuytelaars, and Luc Van Gool. Surf: Speeded up robust features. In *European conference on computer vision*, pages 404-417. Springer, 2006.
- [10] Richard Bellman. A markovian decision process. *Journal of Mathematics and Mechanics*, 6(5):679-684, 1957.
- [11] Richard Bellman. Dynamic programming. Courier Corporation, 2013.
- [12] Richard Bellman and Robert E Kalaba. Dynamic programming and modern control theory, volume 81. Citeseer, 1965.
- [13] Cooper Bills, Joyce Chen, and Ashutosh Saxena. Autonomous mav flight in indoor environments using single image perspective cues. In *ICRA*, pages 5776-5783. IEEE, 2011.
- [14] Johann Borenstein and Yoram Koren. Obstacle avoidance with ultrasonic sensors. *IEEE Journal on Robotics and Automation*, 4(2):213-218, 1988.
- [15] Johann Borenstein and Yoram Koren. Real-time obstacle avoidance for fast mobile robots. *IEEE Transactions on systems, Man, and Cybernetics*, 19(5):1179-1187, 1989.
- [16] Johann Borenstein and Yoram Koren. The vector field histogram-fast obstacle avoidance for mobile robots. *IEEE transactions on robotics and automation*, 7(3):278-288, 1991.
- [17] Konstantinos Bousmalis, Alex Irpan, Paul Wohlhart, Yunfei Bai, Matthew Kelcey, Mrinal Kalakrishnan, Laura Downs, Julian Ibarz, Peter Pastor, Kurt Konolige, et al. Using simulation and domain adaptation to improve efficiency of deep robotic grasping. In *2018 IEEE International Conference on Robotics and Automation (ICRA)*, pages 4243-4250. IEEE, 2018.
- [18] Oliver Brock and Oussama Khatib. High-speed navigation using the global dynamic window approach. In *Proceedings 1999 IEEE International Conference on Robotics and Automation (Cat. No. 99CH36288C)*, volume 1, pages 341-346. IEEE, 1999.
- [19] Rodney Brooks. A robust layered control system for a mobile robot. *IEEE journal on robotics and automation*, 2(1):14-23, 1986.
- [20] Jake Bruce, Niko Sünderhauf, Piotr Mirowski, Raia Hadsell, and Michael Milford. Learning deployable navigation policies at kilometer scale from a single traversal. arXiv preprint arXiv:1807.05211, 2018.
- [21] Ted Camus, David Coombs, Martin Herman, and Tsai-Hong Hong. Real-time single-workstation obstacle avoidance using only wide-field flow divergence. In *Proceedings of 13th International Conference on Pattern Recognition*, volume 3, pages 323-330. IEEE, 1996.
- [22] John Canny. A computational approach to edge detection. In *Readings in computer vision*, pages 184-203. Elsevier, 1987.
- [23] Animesh Chakravarthy and Debasish Ghose. Obstacle avoidance in a dynamic environment: A collision cone approach. *IEEE Transactions on Systems, Man, and Cybernetics-Part A: Systems and Humans*, 28(5):562-574, 1998.
- [24] David L Chen and Raymond J Mooney. Learning to interpret natural language navigation instructions from observations. In *Twenty-Fifth AAAI Conference on Artificial Intelligence*, 2011.
- [25] Kyunghyun Cho, Bart Van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder-decoder for statistical machine translation. arXiv preprint arXiv:1406.1078, 2014.
- [26] Joao Cunha, Eurico Pedrosa, Cristovao Cruz, António JR Neves, and Nuno Lau. Using a depth camera for indoor robot localization and navigation. DETI/IEETA-University of Aveiro, Portugal, 2011.
- [27] Bruno Damas and José Santos-Victor. Avoiding moving obstacles: the forbidden velocity map. In *2009 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 4393-4398. IEEE, 2009.
- [28] Nguyen Xuan Dao, Bum-Jae You, and Sang-Rok Oh. Visual navigation for indoor mobile robots using a single camera. In *2005 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 1992-1997. IEEE, 2005.
- [29] Guilherme N DeSouza and Avinash C Kak. Vision for mobile robot navigation: A survey. *IEEE transactions on pattern analysis and machine intelligence*, 24(2):237-267, 2002.
- [30] Alexey Dosovitskiy, Philipp Fischer, Eddy Ilg, Philip Hausser, Caner Hazirbas, Vladimir Golkov, Patrick Van Der Smagt, Daniel Cremers, and Thomas Brox. FlowNet: Learning optical flow with convolutional networks. In *Proceedings of the IEEE international conference on computer vision*, pages 2758-2766, 2015.
- [31] Alexey Dosovitskiy, German Ros, Felipe Codevilla, Antonio Lopez, and Vladlen Koltun. Carla: An open urban driving simulator. arXiv preprint arXiv:1711.03938, 2017.
- [32] Yan Duan, Marcin Andrychowicz, Bradly Stadie, Jonathan Ho, Jonas Schneider, Ilya Sutskever, Pieter Abbeel, and Wojciech Zaremba. One-shot imitation learning. arXiv preprint arXiv:1703.07326, 2017.

- [33] Hugh Durrant-Whyte and Tim Bailey. Simultaneous localization and mapping: part i. *IEEE robotics & automation magazine*, 13(2):99-110, 2006.
- [34] Alberto Elfes. Sonar-based real-world mapping and navigation. *IEEE Journal on Robotics and Automation*, 3(3):249-265, 1987.
- [35] CDR HR Everett. Survey of collision avoidance and ranging sensors for mobile robots. *RAS*, 5(1):5-67, 1989.
- [36] Dieter Fox, Wolfram Burgard, and Sebastian Thrun. The dynamic window approach to collision avoidance. *IEEE Robotics & Automation Magazine*, 4(1):2333, 1997.
- [37] Paul Furgale and Timothy D Barfoot. Visual teach and repeat for long-range rover autonomy. *Journal of Field Robotics*, 27(5):534-560, 2010.
- [38] Dhiraj Gandhi, Lerrél Pinto, and Abhinav Gupta. Learning to fly by crashing. *arXiv:1704.05588*, 2017.
- [39] Brian P Gerkey and Kurt Konolige. Planning and control in unstructured terrain. In *ICRA Workshop on Path Planning on Costmaps*, 2008.
- [40] James J Gibson. The perception of the visual world. 1950.
- [41] Georges Giralt, Ralph Sobek, and Raja Chatila. A multi-level planning and navigation system for a mobile robot: a first approach to hilare. In *Proceedings of the 6th international joint conference on Artificial intelligence-Volume 1*, pages 335-337. Morgan Kaufmann Publishers Inc., 1979.
- [42] Alessandro Giusti, Jérôme Guzzi, Dan C Cireşan, Fang-Lin He, Juan P Rodríguez, Flavio Fontana, Matthias Faessler, Christian Forster, Jürgen Schmidhuber, Gianni Di Caro, et al. A machine learning approach to visual perception of forest trails for mobile robots. *RA Letters*, 1(2):661-667, 2016.
- [43] Clément Godard, Oisín Mac Aodha, and Gabriel J Brostow. Unsupervised monocular depth estimation with left-right consistency. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 270-279, 2017.
- [44] I. Grondman, L. Busoniu, G. A. D. Lopes, and R. Babuska. A survey of actor-critic reinforcement learning: Standard and natural policy gradients. *IEEE Trans. Syst., Man, Cybern. C*, 42(6):1291-1307, Nov 2012.
- [45] Shixiang Gu, Ethan Holly, Timothy Lillicrap, and Sergey Levine. Deep reinforcement learning for robotic manipulation with asynchronous off-policy updates. In *Robotics and Automation (ICRA), 2017 IEEE International Conference on*, pages 3389-3396. IEEE, 2017.
- [46] Saurabh Gupta, James Davidson, Sergey Levine, Rahul Sukthankar, and Jitendra Malik. Cognitive mapping and planning for visual navigation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2616-2625, 2017.
- [47] Christopher G Harris, Mike Stephens, et al. A combined corner and edge detector. In *Alvey vision conference*, volume 15, pages 10-5244. Citeseer, 1988.
- [48] Hado V Hasselt. Double q-learning. In *NPIS*, pages 2613-2621, 2010.
- [49] Matthew Hausknecht and Peter Stone. Deep recurrent q-learning for partially observable mdps. In *2015 AAAI Fall Symposium Series*, 2015.
- [50] Nicolas Heess, Jonathan J Hunt, Timothy P Lillicrap, and David Silver. Memory-based control with recurrent neural networks. *arXiv preprint arXiv:1512.04455*, 2015.
- [51] Karl Moritz Hermann, Mateusz Malinowski, Piotr Mirowski, Andras BankiHorvath, Keith Anderson, and Raia Hadsell. Learning to follow directions in street view. *arXiv preprint arXiv:1903.00401*, 2019.
- [52] Matteo Hessel, Hado van Hasselt, Joseph Modayil, and David Silver. On inductive biases in deep reinforcement learning. *CoRR*, abs/1907.02908, 2019.
- [53] Noriaki Hirose, Fei Xia, Roberto Martín-Martín, Amir Sadeghian, and Silvio Savarese. Deep visual mpc-policy learning for navigation. *arXiv preprint arXiv:1903.02749*, 2019.
- [54] Ian Horswill. Visual collision avoidance by segmentation. In *Intelligent Robots and Systems*, pages 87-99. Elsevier, 1995.
- [55] Rein Houthoofd, Xi Chen, Yan Duan, John Schulman, Filip De Turck, and Pieter Abbeel. Vime: Variational information maximizing exploration. In *Advances in Neural Information Processing Systems*, pages 1109-1117, 2016.
- [56] Ronald A Howard. Dynamic programming and markov processes. 1960.
- [57] Tadanobu Inoue, Subhajit Choudhury, Giovanni De Magistris, and Sakyasingha Dasgupta. Transfer learning from synthetic to real images using variational autoencoders for precise position detection. In *2018 25th IEEE International Conference on Image Processing (ICIP)*, pages 2725-2729. IEEE, 2018.
- [58] Rudolf Emil Kalman et al. Contributions to the theory of optimal control. *Bol. soc. mat. mexicana*, 5(2):102-119, 1960.
- [59] Mohammad Khan, Shakir Mohamed, Benjamin Marlin, and Kevin Murphy. A stick-breaking likelihood for categorical data analysis with latent gaussian models. In *Artificial Intelligence and Statistics*, pages 610-618, 2012.
- [60] Oussama Khatib. Real-time obstacle avoidance for manipulators and mobile robots. In *Autonomous robot vehicles*, pages 396-404. Springer, 1986.
- [61] Dong Ki Kim and Tsuhan Chen. Deep neural network for real-time autonomous indoor navigation. *arXiv:1511.04668*, 2015.
- [62] M. J. Kim. Thompson sampling for stochastic control: The finite parameter case. *IEEE Trans. Automat. Contr.*, 62(12):6415-6422, Dec 2017.
- [63] Young-geun Kim and Hakil Kim. Layered ground floor detection for visionbased mobile robot navigation. In *IEEE International Conference on Robotics and Automation, 2004. Proceedings. ICRA'04. 2004*, volume 1, pages 13-18. IEEE, 2004.
- [64] Diederik Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv:1412.6980*, 2014.
- [65] A Harry Klopff. Brain function and adaptive systems: a heterostatic theory. Technical report, AIR FORCE CAMBRIDGE RESEARCH LABS HANSCOM AFB MA, 1972.
- [66] Jens Kober, J Andrew Bagnell, and Jan Peters. Reinforcement learning in robotics: A survey. *The International Journal of Robotics Research*, 32(11):1238-1274, 2013.
- [67] Vijay R Konda and John N Tsitsiklis. Actor-critic algorithms. In *Advances in neural information processing systems*, pages 1008-1014, 2000.
- [68] Manikanta Kotaru, Kiran Joshi, Dinesh Bharadia, and Sachin Katti. Spotfi: Decimeter level localization using wifi. In

ACM SIGCOMM Computer Communication Review, volume 45, pages 269-282. ACM, 2015.

- [69] Bruce Krogh. A generalized potential field approach to obstacle avoidance control. In Proc. SME Conf. on Robotics Research: The Next Five Years and Beyond, Bethlehem, PA, 1984, pages 11-22, 1984.
- [70] Bruce Krogh and Charles Thorpe. Integrated path planning and dynamic steering control for autonomous vehicles. In Proceedings. 1986 IEEE International Conference on Robotics and Automation, volume 3, pages 1664-1669. IEEE, 1986.
- [71] Roman Kuc and Billur Barshan. Navigating vehicles through an unstructured environment with sonar. In Proceedings, 1989 International Conference on Robotics and Automation, pages 1422-1426. IEEE, 1989.
- [72] Ashish Kumar, Saurabh Gupta, David Fouhey, Sergey Levine, and Jitendra Malik. Visual memory for robust path following. In Advances in Neural Information Processing Systems, pages 765-774, 2018.
- [73] Ashish Kumar, Saurabh Gupta, and Jitendra Malik. Learning navigation subroutines by watching videos. arXiv preprint arXiv:1905.12612, 2019.
- [74] Young D Kwon and Jin S Lee. A stochastic map building method for mobile robot using 2-d laser range finder. Autonomous Robots, 7(2):187-200, 1999.
- [75] Iro Laina, Christian Rupprecht, Vasileios Belagiannis, Federico Tombari, and Nassir Navab. Deeper depth prediction with fully convolutional residual networks. In 3DV, pages 239-248. IEEE, 2016.
- [76] John J Leonard, Hugh F Durrant-Whyte, and Ingemar J Cox. Dynamic map building for an autonomous mobile robot. The International Journal of Robotics Research, 11(4):286-298, 1992.
- [77] Matteo Leonetti, Luca Iocchi, and Peter Stone. A synthesis of automated planning and reinforcement learning for efficient, robust decision-making. Artificial Intelligence, 241:103-130, 2016.
- [78] Bojian Liang and Nick Pears. Visual navigation using planar homographies. In Proceedings 2002 IEEE International Conference on Robotics and Automation (Cat. No. 02CH37292), volume 1, pages 205-210. IEEE, 2002.
- [79] Timothy P Lillicrap, Jonathan J Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. Continuous control with deep reinforcement learning. arXiv preprint arXiv:1509.02971, 2015.
- [80] Timothy P Lillicrap, Jonathan J Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. Continuous control with deep reinforcement learning. 2016.
- [81] Kai Lingemann, Andreas Nüchter, Joachim Hertzberg, and Hartmut Surmann. High-speed laser localization for mobile robots. Robotics and autonomous systems, 51(4):275-296, 2005.
- [82] Kai Lingemann, Hartmut Surmann, Andreas Nuchter, and Joachim Hertzberg. Indoor and outdoor localization for fast mobile robots. In Intelligent Robots and Systems, 2004.(IROS 2004). Proceedings. 2004 IEEE/RSJ International Conference on, volume 3, pages 2185-2190. IEEE, 2004.
- [83] Manolis IA Lourakis and Stelios C Orphanoudakis. Visual detection of obstacles assuming a locally planar ground. In Asian conference on computer vision, pages 527-534. Springer, 1998.
- [84] Tomas Lozano-Perez. Spatial planning: A configuration space approach. In Autonomous robot vehicles, pages 259-271. Springer, 1990.
- [85] Tomás Lozano-Pérez and Michael A Wesley. An algorithm for planning collision-free paths among polyhedral obstacles. Communications of the ACM, 22(10):560-570, 1979.
- [86] Mateusz Malinowski, Carl Doersch, Adam Santoro, and Peter Battaglia. Learning visual question answering by bootstrapping hard attention. In Proceedings of the European Conference on Computer Vision (ECCV), pages 3-20, 2018.
- [87] Maja J Mataric. Behaviour-based control: Examples from navigation, learning, and group behaviour. Journal of Experimental & Theoretical Artificial Intelligence, 9(2-3):323-336, 1997.
- [88] Yoshio Matsumoto, Kazunori Ikeda, Masayuki Inaba, and Hirochika Inoue. Visual navigation using omnidirectional view sequence. In Proceedings 1999 IEEE/RSJ International Conference on Intelligent Robots and Systems. Human and Environment Friendly Robots with High Intelligence and Emotional Quotients (Cat. No. 99CH36289), volume 1, pages 317-322. IEEE, 1999.
- [89] Yoshio Matsumoto, Masayuki Inaba, and Hirochika Inoue. Visual navigation using view-sequenced route representation. In Proceedings of IEEE International conference on Robotics and Automation, volume 1, pages 83-88. IEEE, 1996.
- [90] Chris McCarthy and Nick Bames. Performance of optical flow techniques for indoor navigation with a mobile robot. In ICRA, volume 5, pages 5093-5098. IEEE, 2004.
- [91] Hao Mei, Yantao Tian, and Linan Zu. A hybrid ant colony optimization algorithm for path planning of robot in dynamic environment. International Journal of Information Technology, 12(3):78-88, 2006.
- [92] Pierre Merriault, Yohan Dupuis, Rémi Bouteau, Pascal Vasseur, and Xavier Savatier. A study of vicon system positioning performance. Sensors, 17(7):1591, 2017.
- [93] Yishu Miao, Edward Grefenstette, and Phil Blunsom. Discovering discrete latent topics with neural variational inference. In International Conference on Machine Learning, pages 2410-2419, 2017.
- [94] Jeff Michels, Ashutosh Saxena, and Andrew Y Ng. High speed obstacle avoidance using monocular vision and reinforcement learning. In ICML, pages 593600. ACM, 2005.
- [95] Krystian Mikolajczyk and Cordelia Schmid. Scale & affine invariant interest point detectors. International journal of computer vision, 60(1):63-86, 2004.
- [96] Hua-Qing Min, Jin-Hui Zhu, and Xi-Jing Zheng. Obstacle avoidance with multiobjective optimization by pso in dynamic environment. In 2005 International Conference on Machine Learning and Cybernetics, volume 5, pages 2950-2956. IEEE, 2005.
- [97] Marvin Minsky. Steps toward artificial intelligence. Proceedings of the IRE, 49(1):8-30, 1961.
- [98] Piotr Mirowski, Matt Grimes, Mateusz Malinowski, Karl Moritz Hermann, Keith Anderson, Denis Teplyashin, Karen Simonyan, Andrew Zisserman, Raia Hadsell, et al. Learning to navigate in cities without a map. In Advances in Neural Information Processing Systems, pages 2419-2430, 2018.
- [99] Piotr Mirowski, Razvan Pascanu, Fabio Viola, Hubert Soyer, Andrew J Ballard, Andrea Banino, Misha Denil, Ross Goroshin, Laurent Sifre, Koray Kavukcuoglu, et al. Learning to navigate in complex environments. arXiv preprint arXiv:1611.03673, 2016.
- [100] Piotr Mirowski, Razvan Pascanu, Fabio Viola, Hubert Soyer, and et.al. Learning to navigate in complex environments.

In ICLR, 2017.

- [101] Volodymyr Mnih, Adria Puigdomenech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In International conference on machine learning, pages 1928-1937, 2016.
- [102] Volodymyr Mnih, Nicolas Heess, Alex Graves, et al. Recurrent models of visual attention. In Advances in neural information processing systems, pages 2204-2212, 2014.
- [103] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. Playing atari with deep reinforcement learning. arXiv preprint arXiv:1312.5602, 2013.
- [104] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. Nature, 518(7540):529, 2015.
- [105] Michael Montemerlo, Sebastian Thrun, Daphne Koller, Ben Wegbreit, et al. Fastslam: A factored solution to the simultaneous localization and mapping problem. Aaai/iaai, 593598, 2002.
- [106] Hadi Moradi, Jongmoo Choi, Eunyoung Kim, and Sukhan Lee. A real-time wall detection method for indoor environments. In 2006 IEEE/RSJ International Conference on Intelligent Robots and Systems, pages 4551-4557. IEEE, 2006.
- [107] Hans Moravec and Alberto Elfes. High resolution maps from wide angle sonar. In Proceedings. 1985 IEEE international conference on robotics and automation, volume 2, pages 116-121. IEEE, 1985.
- [108] Melissa Mozifian, Juan Camilo Gamboa Higuera, David Meger, and Gregory Dudek. Learning domain randomization distributions for transfer of locomotion policies. arXiv preprint arXiv:1906.00410, 2019.
- [109] Raul Mur-Artal, Jose Maria Martinez Montiel, and Juan D Tardos. Orb-slam: a versatile and accurate monocular slam system. IEEE Transactions on Robotics, 31(5):1147-1163, 2015.
- [110] Pushmeet Kohli Nathan Silberman, Derek Hoiem and Rob Fergus. Indoor segmentation and support inference from rgbd images. In ECCV, 2012.
- [111] Nils J Nilsson. Shakey the robot. Technical report, SRI INTERNATIONAL MENLO PARK CA, 1984.
- [112] Deepak Pathak, Parsa Mahmoudieh, Guanghao Luo, Pulkit Agrawal, Dian Chen, Yide Shentu, Evan Shelhamer, Jitendra Malik, Alexei A Efros, and Trevor Darrell. Zero-shot visual imitation. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops, pages 2050-2053, 2018.
- [113] Nick Pears and Bojian Liang. Ground plane segmentation for mobile robot visual navigation. In Proceedings 2001 IEEE/RSJ International Conference on Intelligent Robots and Systems. Expanding the Societal Role of Robotics in the the Next Millennium (Cat. No. 01CH37180), volume 3, pages 1513-1518. IEEE, 2001.
- [114] Jan Peters and Stefan Schaal. Policy gradient methods for robotics. In 2006 IEEE/RSJ International Conference on Intelligent Robots and Systems, pages 2219-2225. IEEE, 2006.
- [115] M. Pfeiffer, S. Shukla, M. Turchetta, C. Cadena, A. Krause, R. Siegwart, and J. Nieto. Reinforced imitation: Sample efficient deep reinforcement learning for mapless navigation by leveraging prior demonstrations. IEEE Robotics and Automation Letters, 3(4):4423-4430, Oct 2018.
- [116] B. Piot, M. Geist, and O. Pietquin. Bridging the gap between imitation learning and inverse reinforcement learning. IEEE Trans. Neural Netw., 28(8):1814-1826, Aug 2017.
- [117] Yuan-Qing Qin, De-Bao Sun, Ning Li, and Yi-Gang Cen. Path planning for mobile robot using the particle swarm optimization with mutation operator. In Proceedings of 2004 international conference on machine learning and cybernetics (IEEE Cat. No. 04EX826), volume 4, pages 2473-2478. IEEE, 2004.
- [118] Purushothaman Raja and Sivagurunathan Pugazhenth. Optimal path planning of mobile robots: A review. International journal of physical sciences, 7(9):1314-1320, 2012.
- [119] Anthony Remazeilles, François Chaumette, and Patrick Gros. Robot motion control from a visual memory. In IEEE International Conference on Robotics and Automation, 2004. Proceedings. ICRA'04. 2004, volume 5, pages 4695-4700. IEEE, 2004.
- [120] Edward Rosten and Tom Drummond. Machine learning for high-speed corner detection. In European conference on computer vision, pages 430-443. Springer, 2006.
- [121] Daniel J Russo, Benjamin Van Roy, Abbas Kazerouni, Ian Osband, Zheng Wen, et al. A tutorial on thompson sampling. Foundations and Trends® in Machine Learning, 11(1):1-96, 2018.
- [122] Fereshteh Sadeghi and Sergey Levine. Cad2rl: Real single-image flight without a single real image. arXiv preprint arXiv:1611.04201, 2016.
- [123] Fereshteh Sadeghi and Sergey Levine. RL: Real single-image flight without a single real image. arxiv preprint. arXiv:1611.04201, 12, 2016.
- [124] Fereshteh Sadeghi and Sergey Levine. (cad)2rl: Real single-image flight without a single real image. Robotics: Science and Systems, 2017.
- [125] José Santos-Victor, Giulio Sandini, Francesca Curotto, and Stefano Garibaldi. Divergent stereo for robot navigation: Learning from bees. In Proceedings of IEEE conference on computer vision and pattern recognition, pages 434-439. IEEE, 1993.
- [126] Davide Scaramuzza. 1-point-ransac structure from motion for vehicle-mounted cameras by exploiting non-holonomic constraints. International journal of computer vision, 95(1):74-85, 2011.
- [127] Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver. Prioritized experience replay. arXiv preprint arXiv:1511.05952, 2015.
- [128] Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver. Prioritized experience replay. In International Conference on Learning Representations, Puerto Rico, 2016.
- [129] Florian Schroff, Dmitry Kalenichenko, and James Philbin. Facenet: A unified embedding for face recognition and clustering. In Proceedings of the IEEE conference on computer vision and pattern recognition, pages 815-823, 2015.
- [130] Alexander Scott, Lynne E Parker, and Claude Touzet. Quantitative and qualitative comparison of three laser-range mapping algorithms using two types of laser scanner data. In Smc 2000 conference proceedings. 2000 IEEE international conference on systems, man and cybernetics. 'cybernetics evolving to systems, humans, organizations, and their complex

interactions'(cat. no. 0, volume 2, pages 1422-1427. IEEE, 2000.

- [131] Souvik Sen, Jeongkeun Lee, Kyu-Han Kim, and Paul Congdon. Avoiding multipath to revive inbuilding wifi localization. In Proceeding of the 11th annual international conference on Mobile systems, applications, and services, pages 249-262. ACM, 2013.
- [132] Souvik Sen, Božidar Radunovic, Romit Roy Choudhury, and Tom Minka. You are facing the mona lisa: spot localization using phy layer information. In Proceedings of the 10th international conference on Mobile systems, applications, and services, pages 183-196. ACM, 2012.
- [133] Rico Sennrich, Barry Haddow, and Alexandra Birch. Improving neural machine translation models with monolingual data. arXiv preprint arXiv:1511.06709, 2015.
- [134] Jayaram Sethuraman. A constructive definition of dirichlet priors. Stat. Sin., pages 639-650, 1994.
- [135] David Silver, Aja Huang, Chris J Maddison, Arthur Guez, Laurent Sifre, George Van Den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. Mastering the game of go with deep neural networks and tree search. nature, 529(7587):484, 2016.
- [136] David Silver, Guy Lever, Nicolas Heess, Thomas Degris, Daan Wierstra, and Martin Riedmiller. Deterministic policy gradient algorithms. In Proceedings of the 31st International Conference on International Conference on Machine Learning - Volume 32, ICML'14, pages I-387-I-395. JMLR.org, 2014.
- [137] David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, et al. Mastering the game of go without human knowledge. Nature, 550(7676):354, 2017.
- [138] Kahlouche Souhila 與 Achour Karim. Optical flow based robot obstacle avoidance. IJARS, 4(1):2, 2007.
- [139] Gregory J Stein 與 Nicholas Roy. Genesis-rt: Generating synthetic images for training secondary real-world tasks. 於 2018 IEEE International Conference on Robotics and Automation (ICRA), 第 7151-7158 頁. IEEE, 2018.
- [140] Richard S Sutton. Learning to predict by the methods of temporal differences. Machine learning, 3(1):9-44, 1988.
- [141] Richard S Sutton、Andrew G Barto 等人. Introduction to reinforcement learning, 第 135 卷. MIT press Cambridge, 1998.
- [142] Richard S Sutton, David A McAllester, Satinder P Singh, and Yishay Mansour. Policy gradient methods for reinforcement learning with function approximation. In Advances in neural information processing systems, pages 1057-1063, 2000.
- [143] Tristan Swedish and Ramesh Raskar. Deep visual teach and repeat on path networks. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops, pages 1533-1542, 2018.
- [144] Lei Tai, Shaohua Li, and Ming Liu. A deep-network solution towards model-less obstacle avoidance. In IROS, pages 2759-2764. IEEE, 2016.
- [145] Lei Tai and Ming Liu. Towards cognitive exploration through deep reinforcement learning for mobile robots. arXiv:1610.01733, 2016.
- [146] Lei Tai, Giuseppe Paolo, and Ming Liu. Virtual-to-real deep reinforcement learning: Continuous control of mobile robots for mapless navigation. In Intelligent Robots and Systems (IROS), 2017 IEEE/RSJ International Conference on, pages 31-36. IEEE, 2017.
- [147] Ashit Talukder, S Goldberg, Larry Matthies, and Adnan Ansar. Real-time detection of moving objects in a dynamic scene from moving robotic vehicles. In Proceedings 2003 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS 2003)(Cat. No. 03CH37453), volume 2, pages 1308-1313. IEEE, 2003.
- [148] Ashit Talukder and Larry Matthies. Real-time detection of moving objects from moving vehicles using dense stereo and optical flow. In 2004 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)(IEEE Cat. No. 04CH37566), volume 4, pages 3718-3725. IEEE, 2004.
- [149] Edward L Thorndike. The law of effect. The American journal of psychology, 39(1/4):212-222, 1927.
- [150] Sebastian Thrun, Wolfram Burgard, and Dieter Fox. A probabilistic approach to concurrent mapping and localization for mobile robots. Autonomous Robots, 5(3-4):253-271, 1998.
- [151] Sebastian Thrun and Michael Montemerlo. The graph slam algorithm with applications to large-scale mapping of urban structures. The International Journal of Robotics Research, 25(5-6):403-429, 2006.
- [152] Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. In 2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pages 23-30. IEEE, 2017.
- [153] X. Truong and T. D. Ngo. Toward socially aware robot navigation in dynamic and crowded environments: A proactive social motion model. IEEE Trans. Automat. Sci. Eng., 14(4):1743-1760, Oct 2017.
- [154] Iwan Ulrich and Illah Nourbakhsh. Appearance-based obstacle detection with monocular color vision. In AAAI/IAAI, pages 866-871, 2000.
- [155] Prahlad Vadakkepat, Kay Chen Tan, and Wang Ming-Liang. Evolutionary artificial potential fields and their application in real time robot path planning. In Proceedings of the 2000 congress on evolutionary computation. CECOO (Cat. No. 00TH8512), volume 1, pages 256-263. IEEE, 2000.
- [156] Hado Van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double q-learning. In AAAI, pages 2094-2100, 2016.
- [157] Ayzaan Wahid, Alexander Toshev, Marek Fiser, and Tsang-Wei Edward Lee. Long range neural navigation policies for the real world. arXiv preprint arXiv:1903.09870, 2019.
- [158] Sen Wang, Hongkai Wen, Ronald Clark, and Niki Trigoni. Keyframe based large-scale indoor localisation using geomagnetic field and motion pattern. In IEEE/RSJ International Conference on Intelligent Robots and Systems, 2016.
- [159] Xin Wang, Qiuyuan Huang, Asli Celikyilmaz, Jianfeng Gao, Dinghan Shen, Yuan-Fang Wang, William Yang Wang, and Lei Zhang. Reinforced cross-modal matching and self-supervised imitation learning for vision-language navigation. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pages 6629-6638, 2019.
- [160] Ziyu Wang, Nando de Freitas, and Marc Lanctot. Dueling network architectures for deep reinforcement learning.

CoRR, abs/1511.06581, 2015.

[161] Ziyu Wang, Tom Schaul, Matteo Hessel, Hado Hasselt, Marc Lanctot, and Nando Freitas. Dueling network architectures for deep reinforcement learning. In Maria Florina Balcan and Kilian Q. Weinberger, editors, Proceedings of The

第 33 屆國際機器學習會議 (International Conference on Machine Learning), Proceedings of Machine Learning Research 第 48 卷, 第 1995-2003 頁, 美國紐約州紐約市, 2016 年 6 月 20-22 日. PMLR.

[162] Ziyu Wang, Tom Schaul, Matteo Hessel, Hado Van Hasselt, Marc Lanctot, and Nando De Freitas. Dueling network architectures for deep reinforcement learning. arXiv preprint arXiv:1511.06581, 2015.

[163] Christopher John Cornish Hellaby Watkins. Learning from delayed rewards. 博士論文, 劍橋大學國王學院, 1989.

[164] Ronald J Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. Machine learning, 8(3-4):229-256, 1992.

[165] Oliver Wulf and Bernardo Wagner. Fast 3d scanning methods for laser measurement systems. In International conference on control systems and computer science (CSCS14), pages 2-5. Citeseer, 2003.

[166] Jing Xiao, Zbigniew Michalewicz, Lixin Zhang, and Krzysztof Trojanowski. Adaptive evolutionary planner/navigator for mobile robots. IEEE transactions on evolutionary computation, 1(1):18-28, 1997.

[167] Zhuoling Xiao, Hongkai Wen, Andrew Markham, and Niki Trigoni. Robust pedestrian dead reckoning (r-pdr) for arbitrary mobile device placement. In Indoor Positioning and Indoor Navigation (IPIN), 2014 International Conference on, pages 187-196. IEEE, 2014.

[168] Linhai Xie, Sen Wang, Andrew Markham, and Niki Trigoni. Towards monocular vision based obstacle avoidance through deep reinforcement learning. arXiv preprint arXiv:1706.09829, 2017.

[169] Linhai Xie, Sen Wang, Stefano Rosa, Andrew Markham, and Niki Trigoni. Learning with training wheels: speeding up training with a simple controller for deep reinforcement learning. In 2018 IEEE International Conference on Robotics and Automation (ICRA), pages 6276-6283. IEEE, 2018.

[170] Zhaoming Xie, Glen Berseth, Patrick Clary, Jonathan Hurst, and Michiel van de Panne. Feedback control for cassie with deep reinforcement learning. In 2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pages 1241-1246. IEEE, 2018.

[171] Ali Yahya, Adrian Li, Mrinal Kalakrishnan, Yevgen Chebotar, and Sergey Levine. Collective robot reinforcement learning with distributed asynchronous guided policy search. In IEEE/RSJ International Conference on Intelligent Robots and Systems, pages 79-86. IEEE, 2017.

[172] Shichao Yang, Sandeep Konam, Chen Ma, Stephanie Rosenthal, Manuela Veloso, and Sebastian Scherer. Obstacle avoidance through deep networks based intermediate perception. arXiv:1704.08759, 2017.

[173] Z. Yang, K. Merrick, L. Jin, and H. A. Abbass. Hierarchical deep reinforcement learning for continuous action control. IEEE Trans. Neural Netw., 29(11):51745184, Nov 2018.

[174] Cang Ye, N. H. C. Yung, and Danwei Wang. A fuzzy controller with supervised learning assisted reinforcement learning algorithm for obstacle avoidance. IEEE Trans. Syst., Man, Cybern. B, 33(1):17-27, Feb 2003.

[175] Jingwei Zhang, Lei Tai, Peng Yun, Yufeng Xiong, Ming Liu, Joschka Boedecker, and Wolfram Burgard. Vr-goggles for robots: Real-to-sim domain adaptation for visual control. IEEE Robotics and Automation Letters, 4(2):1148-1155, 2019.

[176] Li Zhang and Bijoy K Ghosh. Line segment based map building and localization using 2d laser rangefinder. In Proceedings 2000 ICRA. Millennium Conference. IEEE International Conference on Robotics and Automation. Symposia Proceedings (Cat. No. 00CH37065), volume 3, pages 2538-2543. IEEE, 2000.

[177] Zhengyou Zhang. Microsoft kinect sensor and its effect. IEEE multimedia, 19(2):4-10, 2012.

[178] Oleksii Zhelo, Jingwei Zhang, Lei Tai, Ming Liu, and Wolfram Burgard. Curiosity-driven exploration for mapless navigation with deep reinforcement learning. arXiv preprint arXiv:1804.00456, 2018.

[179] Chao Zhou, Yucheng Wei, and Tieniu Tan. Mobile robot self-localization based on global visual appearance features. In 2003 IEEE International Conference on Robotics and Automation (Cat. No. 03CH37422), volume 1, pages 1271-1276. IEEE, 2003.

[180] Jin Zhou and Baoxin Li. Homography-based ground detection for a mobile robot platform using a single camera. In Proceedings 2006 IEEE International Conference on Robotics and Automation, 2006. ICRA 2006., pages 4100-4105. IEEE, 2006.

[181] Yuke Zhu, Roozbeh Mottaghi, Eric Kolve, Joseph J Lim, Abhinav Gupta, Li Fei-Fei, and Ali Farhadi. Target-driven visual navigation in indoor scenes using deep reinforcement learning. In Robotics and Automation (ICRA), 2017 IEEE International Conference on, pages 3357-3364. IEEE, 2017.

¹ http://wiki.ros.org/stage_ros

² http://wiki.ros.org/gazebo_ros_pkgs

¹ <https://en.wikipedia.org/wiki/Kinect>

² <https://www.turtlebot.com/turtlebot2/>

³ <https://github.com/zainmehdi/Optical-Flow-based-Obstacle-Avoidance>

¹ http://wiki.ros.org/move_base

² http://wiki.ros.org/move_base

³ <http://wiki.ros.org/stage>

⁴ http://wiki.ros.org/gazebo_ros_pkgs

⁵ 請注意, 此處提到的變異量 (variance) 是指平滑處理後學習曲線的變異量。

⁶ <http://wiki.ros.org/amcl>

⁷ <http://wiki.ros.org/gmapping>

¹ http://wiki.ros.org/gazebo_ros_pkgs

² <http://wiki.ros.org/navigation>

³ <https://www.turtlebot.com/turtlebot2/>

⁴ <https://en.wikipedia.org/wiki/Kinect>